Diffusion Scores for Markov Data

A technical compendium

Exact inference on the noised chain, estimation of the transition kernel, and comparison with neural denoisers

Giovanni Mantovani
Department of Computing Sciences, Bocconi University
giovanni.mantovani@studbocconi.it

September 2, 2026

Status. Work in progress, circulated internally. This document is the long-form companion to the internal note *Diffusion scores for Markov data*. Where the note states a result, this document derives it. Chapters covering experiments still running are marked [PENDING] and will be completed as those results land; nothing marked pending is relied on elsewhere.

Who this is for. A reader who knows undergraduate probability and linear algebra and nothing about diffusion models, graphical models, or expectation maximisation. Every concept used anywhere in the project is built here from its definition. Nothing is cited in place of an explanation. Where a step is standard but non-obvious, it is done in full rather than asserted, and where an assumption is doing real work, there is a box saying what breaks without it.

Contents

Part I

Model and exact identities

Chapter 1

Setting, conventions, and the three scores

This chapter fixes what the rest of the document assumes. It is deliberately compact: the material here is developed from first principles in the thesis (Hamiltonian and statistical mechanics, stochastic calculus, the diffusion literature), and repeating that development would turn a technical supplement into a second textbook. What is kept is what later chapters actually cite — the estimation-theoretic facts that Proposition 6.1 rests on, the coordinatewise structure of the channel that makes the whole project possible, and the distinction between three different objects that the word "score" is used for.

1.1 Conditioning and the Bayes-optimal estimator

Bayes' rule, in the form used throughout:

$$P(a \mid x) \;=\; \frac{P(x \mid a)\, P(a)}{P(x)} \;\propto\; \underbrace{P(x \mid a)}_{\text{likelihood}}\; \underbrace{P(a)}_{\text{prior}}. \tag{1.1}$$

Proposition 1.1 (Orthogonality of the posterior mean)**.** *For any estimator u that is a function of x alone,*

$$\mathbb{E}\big[\|a-u\|^2 \,\big|\, x\big] \;=\; \mathbb{E}\big[\|a-\langle a\rangle_{x,t}\|^2 \,\big|\, x\big] \;+\; \|u-\langle a\rangle_{x,t}\|^2. \tag{1.2}$$

Proof. Expand $a-u = (a-\langle a\rangle_{x,t}) + (\langle a\rangle_{x,t}-u)$. The cross term vanishes because $\langle a\rangle_{x,t}-u$ is x-measurable and $\mathbb{E}[a-\langle a\rangle_{x,t} \mid x] = 0$. □

Two consequences are used constantly. The posterior mean minimises the conditional squared risk, so it is the reference every estimator in this project is scored against. And the first term is an irreducible floor common to all estimators, which is why every table reports error *against* $\langle a\rangle_{x,t}$ rather than raw risk — subtracting the floor is what makes the comparison informative.

Optimality here is with respect to squared error specifically. A different loss selects a different functional of the posterior — absolute error would select the median. Nothing in the project depends on squared loss being the right choice for any application; it is the choice that makes the target computable and the comparison well posed.

1.2 Linear estimators

Restricting to estimators linear in x gives the LMMSE estimator, which for zero-mean (a, x) is

$$\widehat{\mathbf{a}}_{\mathrm{LMMSE}} \;=\; \mathrm{Cov}(a,x)\;\mathrm{Cov}(x)^{-1}x. \tag{1.3}$$

Proposition 1.2 (Linear optimality)**.** *Among estimators of the form $u = Bx$,* (1.3) *minimises* $\mathbb{E}\|a - u\|^2$*. When (a, x) is jointly Gaussian and zero-mean it coincides with $\langle a\rangle_{x,t}$, since the conditional expectation is then linear.*

The zero-mean hypothesis is load-bearing rather than cosmetic: the general form carries $\mathbb{E}[a] + \mathrm{Cov}(a, x)\,\mathrm{Cov}(x)^{-1}(x - \mathbb{E}[x])$, and dropping the mean terms when $\mathbb{E}[a] \neq 0$ produces a genuinely different estimator. Chapter 6 measures the size of that difference.

1.3 Two things a distribution can be

A model is fitted to samples, not to a distribution. Writing the empirical measure

$$\widehat{P}_0 \;=\; \frac{1}{M}\sum_{\mu=1}^{M} \delta_{a^\mu}, \tag{1.4}$$

the distinction between P_0 and $\widehat{P}_0$ is the one §2.3 turns into the project's motivation. It is not a technicality about estimation error: the two have genuinely different scores, and the reverse process driven by one does something categorically different from the reverse process driven by the other.

1.4 The forward process

The Ornstein–Uhlenbeck channel,

$$\frac{\mathrm{d}x}{\mathrm{d}t} \;=\; -x + \sqrt{2}\,\eta(t), \qquad x(0) = a, \tag{1.5}$$

with η standardised Gaussian white noise, so the $\sqrt{2}$ is carried explicitly. Solving by integrating factor gives

$$x(t) \;=\; e^{-t}a + \sqrt{\Delta_t}\,z, \qquad \Delta_t = 1 - e^{-2t}, \qquad z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})\,, \tag{1.6}$$

and hence the noised distribution as a Gaussian convolution,

$$P_t(x) \;=\; \int \mathrm{d}a\; P_0(a)\,\frac{1}{(2\pi\Delta_t)^{L/2}}\exp\!\left[-\frac{\|x - e^{-t}a\|^2}{2\Delta_t}\right]. \tag{1.7}$$

Two properties are used repeatedly: $\Delta_t \to 0$ as $t \to 0$, so $P_t \to P_0$; and $\mathrm{Var}(x_t) = 1$ for unit-variance data at every t, so initialising the reverse process at $\mathcal{N}(\mathbf{0}, \mathbf{I})$ incurs a bias of only $e^{-2t_{\max}}$.

1.5 Why the coordinatewise structure matters

This section is short and is the reason the project exists. The forward noise is independent across coordinates, so the likelihood factorises sitewise:

$$P(x \mid a, t) = \prod_{i=1}^{L} \frac{1}{\sqrt{2\pi\Delta_t}}\exp\!\left[-\frac{(x_i - e^{-t}a_i)^2}{2\Delta_t}\right] \;=\; \prod_{i=1}^{L} \ell_t(x_i \mid a_i). \tag{1.8}$$

Assumption at work

Each factor $\ell_t(x_i \mid a_i)$ has degree one among the latent variables. Attaching a degree-one factor to a graph adds a leaf and cannot create a cycle — so conditioning on x reweights

node potentials without creating edges among the a_i, and the posterior factor graph is whatever the prior's was.
Conditioning usually creates dependence. It does not here, precisely because each observation is generated from a single latent variable. The construction fails the moment the forward noise is correlated across coordinates: then $P(x \mid a)$ does not factorise, the likelihood becomes a factor touching several a_i, and the posterior graph acquires edges the prior did not have. Everything in Chapters 5–8 rests on this one property.

1.6 Reversing it

By the time-reversal theorem for diffusions [1, 2], a forward SDE $\mathrm{d}x = f\,\mathrm{d}t + g\,\mathrm{d}W$ has a time-reversed process with drift $f - g^2\nabla \log p_t$ and the same diffusion coefficient. With $f = -x$ and $g = \sqrt{2}$, writing $\tau = t_{\max} - t$ so that increasing τ means decreasing t,

$$\frac{\mathrm{d}x}{\mathrm{d}\tau} \;=\; x + 2\,S(x,\tau) + \sqrt{2}\,\eta(\tau), \qquad S(x,t) := \nabla_x \log P_t(x), \tag{1.9}$$

which is the only object the reverse process needs. Chapter 2 shows it is a rescaling of the posterior mean, and therefore that generating and inferring are the same problem.

1.7 Map of the code

Everything referenced in this document lives under `research/nongaussian-bp/`.

Path	Contents
`src/priors.py`	The data-generating models: `GaussianAR1`, `LaplaceAR1`, `StudentTAR1`, `UniformAR1`, `GaussianMixtureAR1`. Each provides `sample` and `log_transition_matrix`.
`src/bp_grid.py`	The inference recursion. `make_grid`, `grid_bp` (one sequence, with evidence), `grid_bp_batch` (many sequences, device-parameterised).
`src/bp_gaussian.py`	The closed-form Gaussian recursion in information form.
`src/exact_scores.py`	Closed-form Gaussian posterior mean and score, used as ground truth.
`src/em.py`	The expectation step, the sufficient statistic, and the estimation driver.
`src/kernels.py`	Parametrised transition families and their maximisation steps.
`src/denoiser.py`	A common interface over inference-based and network-based denoisers, plus the network training loop.
`src/reverse.py`	Reverse-time integration: SDE and probability-flow ODE.
`src/sample_metrics.py`	Statistics for comparing generated data against the truth.
`src/backend.py`	CPU/GPU array-module dispatch for the recursion.
`src/ring.py`	The rotating-ring model of Chapter 10: gauge, the closed-form ψ-conditional density, and the blind per-frame likelihood.
`experiments/frozen_config.py`	The single configuration behind every number in the note and the workshop paper. Imported, never re-specified.
`experiments/`	One file per experiment, each self-describing. `exp_28` is the ring.
`tools/check_paper.sh`	The production gate: page limits, no code references, no unfilled placeholders, shared sections still shared.
`tools/check_replicates.py`	Parses each experiment's settings and fails on a replicate count the frozen config cannot reach. See §9.10.3.
`tests/`	Validation. Several tests here are the evidence for claims made in this document and are cited as such.

Chapter 2

The score is the posterior mean

This chapter proves the identity that makes the whole project possible: the score, which is what the reverse process needs, is a rescaling of the posterior mean, which is what inference computes. They are the same problem.

2.1 The derivation

Start from (1.7) and differentiate with respect to x. The only x-dependence is in the Gaussian factor, and

$$\nabla_x \exp\left[-\frac{\|x - e^{-t}a\|^2}{2\Delta_t}\right] = -\frac{x - e^{-t}a}{\Delta_t} \exp\left[-\frac{\|x - e^{-t}a\|^2}{2\Delta_t}\right],$$

so

$$\nabla_x P_t(x) = \int \mathrm{d}a\ P_0(a) \left(-\frac{x - e^{-t}a}{\Delta_t}\right) \frac{1}{(2\pi\Delta_t)^{L/2}} \exp\left[-\frac{\|x - e^{-t}a\|^2}{2\Delta_t}\right]. \tag{2.1}$$

Now divide by $P_t(x)$. By Bayes' rule (1.1), the quantity

$$\frac{P_0(a)\,(2\pi\Delta_t)^{-L/2} \exp\left[-\|x - e^{-t}a\|^2/2\Delta_t\right]}{P_t(x)}$$

is exactly the posterior density $P(a \mid x, t)$: numerator is prior times likelihood, denominator is the evidence. So (2.1) becomes

$$S(x,t) = \frac{\nabla_x P_t(x)}{P_t(x)} = \int \mathrm{d}a\ P(a \mid x, t) \left(-\frac{x - e^{-t}a}{\Delta_t}\right) = -\frac{x}{\Delta_t} + \frac{e^{-t}}{\Delta_t} \int \mathrm{d}a\ P(a \mid x, t)\, a,$$

and the remaining integral is the posterior mean. Hence

$$\boxed{S(x,t) \;=\; \nabla_x \log P_t(x) \;=\; -\frac{x - e^{-t}\,\langle a\rangle_{x,t}}{\Delta_t}.} \tag{2.2}$$

When is the differentiation legitimate? Interchanging ∇_x and $\int \mathrm{d}a$ requires a dominating function. The integrand's gradient is bounded in absolute value by

$$P_0(a)\,\frac{|x - e^{-t}a|}{\Delta_t}\,\frac{e^{-\|x-e^{-t}a\|^2/2\Delta_t}}{(2\pi\Delta_t)^{L/2}},$$

and for x in a neighbourhood of any fixed point this is dominated by $C(1 + \|a\|)P_0(a)$ for a constant C depending on Δ_t and the neighbourhood, since the Gaussian factor is bounded and decays. That dominating function is integrable exactly when P_0 has a finite first moment.

Dominated convergence then applies. All the priors used here have moments of every order, so the condition is not restrictive.

The formula says something concrete. Rearranged, $\langle a\rangle_{x,t} = e^t(x + \Delta_t\, S(x,t))$: to guess the clean signal, take the observation, step along the score by an amount proportional to how noisy things are, and undo the shrinkage. At small t, Δ_t is small and the correction is tiny — the observation is already almost the answer. At large t, $\Delta_t \to 1$ and the correction does all the work.

Implementation

`src/bp_grid.py` $\to$ `score_from_posterior_mean` and `src/denoiser.py` $\to$ `score_from_mean` both implement (2.2). Every arm in every comparison converts a posterior mean to a score through one of these, so no arm can gain an advantage from a different convention.

2.2 Consequences worth stating

2.2.1 Estimating the score and denoising are one problem

There is no separate "score model". A denoiser and a score model are the same object in two coordinate systems. This is why diffusion models can be trained by a regression loss on clean signals and then used to drive a differential equation — the training target and the sampling requirement are related by (2.2).

2.2.2 The two error measures differ by a known factor

Applying (2.2) to two different denoisers $\widehat{m}$ and $m^\star$ and subtracting, the x terms cancel:

$$\widehat{S} - S^\star = \frac{e^{-t}}{\Delta_t}\,(\widehat{m} - m^\star)\,, \qquad \text{so} \qquad \big\|\widehat{S} - S^\star\big\| = \frac{e^{-t}}{\Delta_t}\,\big\|\widehat{m} - m^\star\big\|. \tag{2.3}$$

Score error and posterior-mean error are therefore *the same measurement* up to the factor e^{-t}/Δ_t. That factor is not mild: it runs from about 6.0 at $t = 0.08$ to about 0.09 at $t = 2.4$, a spread of roughly 65.

Assumption at work

Reporting only a score error silently weights the low-noise end of the schedule about sixty-five times more heavily than the high-noise end, relative to reporting a posterior-mean error. Two methods can therefore be ranked differently by the two metrics without either metric being wrong. This project reports both, and states the factor, so that a reader can convert. Earlier work here reported score error alone.

Implementation

`src/metrics.py` $\to$ `score_mean_identity_residual` checks (2.3) holds to machine precision for any pair of denoisers — a large residual means an implementation bug, not a modelling difference. `src/sample_metrics.py` $\to$ `pointwise_ladder` emits `score_reweighting` next to every error it reports. Verified in `tests/test_sample_metrics.py` $\to$ `test_score_reweighting_matches_the_tweedie_factor` and `test_reweighting_spans_the_claimed_range_across_the_schedule`.

2.2.3 The Gaussian case, as a check

If $P_0 = \mathcal{N}(0, \Sigma_0)$ then x is Gaussian with covariance $\Sigma_t = e^{-2t}\Sigma_0 + \Delta_t I$, and the posterior mean is linear:

$$\langle a\rangle_{x,t} = e^{-t}\Sigma_0\Sigma_t^{-1}x, \qquad S(x,t) = -\Sigma_t^{-1}x. \tag{2.4}$$

The second follows by differentiating $\log P_t(x) = -\frac{1}{2}x^\top \Sigma_t^{-1} x + \text{const}$ directly, and consistency with (2.2) is a one-line check that we use as a test of the whole pipeline.

Implementation

`src/exact_scores.py` → `exact_gaussian_posterior_mean` and `src/exact_scores.py` → `precision_matrix_score` implement (2.4). These are the ground truth against which the general machinery of Chapter 5 is validated; see `tests/test_score_mean_error_identity.py`.

2.3 Three scores, not one

Equation (2.2) defines the score of *a* distribution. Which distribution one has in mind changes what the object is, and three different choices appear in the literature under the same word.

The population score. Take P_0 to be the true data distribution. This is the score that the reverse process would need in order to generate genuinely new data distributed as P_0. Nobody has it, because nobody has P_0.

The empirical score. Take $\widehat{P}_0$ from (1.4), the sum of point masses at the training examples. Its noised version is a Gaussian mixture centred on those examples, and its posterior mean is a softmax-weighted average of them:

$$\widehat{m}(x,t) = \frac{\sum_\mu a^\mu \exp\left[-\|x - e^{-t}a^\mu\|^2/2\Delta_t\right]}{\sum_\nu \exp[-\|x - e^{-t}a^\nu\|^2/2\Delta_t]}. \tag{2.5}$$

Note what this returns as $t \to 0$: the nearest training example. Exact reversal under this score reproduces the training set and nothing else.

The learned score. What a network with finite capacity, finite data and a particular optimiser actually produces.

Assumption at work

The chain of reasoning that matters, stated carefully:

1. Training minimises an average over the observed samples — the *empirical* objective.
2. Its unrestricted minimiser over all measurable functions is the *empirical* posterior mean (2.5), by Proposition 1.1 applied under $\widehat{P}_0$.
3. Exact reversal under that score returns $\widehat{P}_0$ — the training set.
4. Therefore a model that perfectly attained the optimum of its own training objective would be useless as a generative model.

That trained models nonetheless generalise is a statement about what stops them from attaining that optimum — capacity, architecture, optimisation, regularisation — not about the objective. It is *not* correct to say the population objective is minimised by the empirical score; the population objective is minimised by the population posterior mean.

This is the tension that motivates wanting a setting where the *population* score is computable. If one can compute it, one can ask how far a trained network is from it, and separate "the network is bad" from "this target is hard". That is what the Markov chain of Chapter 3 provides.

Sources

The identity between the score of a Gaussian-convolved density and the posterior mean is classical in empirical Bayes, due to Robbins [3] and Miyasawa [4], and is usually called Tweedie's formula after the exposition in Efron [5]. Its use as a training objective for generative models is Vincent [6], building on the score-matching estimator of Hyvärinen [7]; the generative-modelling application is Song and Ermon [8].

Chapter 3

The data model: a Markov chain

3.1 Definition

A sequence $a = (a_1, \dots, a_L)$ is a *first-order Markov chain* if, given the present, the future is independent of the past:

$$P(a_i \mid a_{i-1}, a_{i-2}, \dots, a_1) \;=\; P(a_i \mid a_{i-1}) \qquad \text{for all } i. \tag{3.1}$$

Applying the chain rule of probability and then (3.1) repeatedly,

$$\boxed{P_0(a) \;=\; \mu(a_1) \prod_{i=2}^{L} K(a_i \mid a_{i-1}),} \tag{3.2}$$

where μ is the law of the first site and K is the *transition kernel.* We take K homogeneous — the same rule at every step — which will matter in Chapter 5 and again when we estimate it.

> The factorisation is the whole point. A general density on $\mathbb{R}^L$ is an unmanageable object. Equation (3.2) says this one is built from $L-1$ copies of a single two-variable function. Everything tractable about the model descends from that.

3.2 The specific family used here

We use additive transitions with a linear autoregression,

$$a_i \;=\; \rho\, a_{i-1} + \varepsilon_i, \qquad \varepsilon_i \sim \varphi \text{ i.i.d.}, \qquad K(a' \mid a) = \varphi(a' - \rho a), \tag{3.3}$$

with $|\rho| < 1$, and φ an *innovation density* of mean zero and variance q.

3.2.1 Covariance stationarity

Suppose a_1 is drawn with $\mathrm{Var}(a_1) = \sigma^2$ and ask when that variance is preserved along the chain. Taking variances in (3.3) and using independence of ε_i from a_{i-1},

$$\sigma^2 = \rho^2\sigma^2 + q \qquad \Longrightarrow \qquad \sigma^2 = \frac{q}{1-\rho^2}.$$

Choosing $q = 1 - \rho^2$ therefore gives unit marginal variance. For the covariance, multiply (3.3) by a_{i-k} and take expectations; since ε_i is independent of everything earlier,

$$\mathrm{Cov}(a_i, a_{i-k}) = \rho\ \mathrm{Cov}(a_{i-1}, a_{i-k}) = \cdots = \rho^k\sigma^2,$$

so with the normalisation above

$$\boxed{(\Sigma_0)_{ij} \;=\; \mathrm{Cov}(a_i, a_j) \;=\; \rho^{|i-j|}.} \tag{3.4}$$

Correlation decays geometrically with separation, with ρ setting the range. At $\rho = 0.85$, sites five apart still share $\rho^5 \approx 0.44$ of their variation; sites twenty apart share 0.04. There is a well-defined correlation length, and it will reappear in Chapter 6 as the scale controlling how much context a denoiser needs.

3.2.2 The crucial degree of freedom

Here is the feature that makes this family a good experimental probe, and it is worth dwelling on.

Equation (3.4) depends on φ *only through its variance* q. Everything else about the innovation — whether it is Gaussian, heavy-tailed, bounded, bimodal — leaves no trace in the covariance whatsoever.

So one can build a family of data distributions that are

- *identical* in every second-order statistic, and
- *different* in every higher-order one.

family	innovation	excess kurtosis	covariance
`GaussianAR1`	$\mathcal{N}(0, q)$	0	$\rho^{|i-j|}$
`LaplaceAR1`	Laplace$(0, b)$, $2b^2 = q$	$+3$	$\rho^{|i-j|}$
`StudentTAR1`	scaled t_ν	$6/(\nu - 4)$	$\rho^{|i-j|}$
`UniformAR1`	uniform on $[-c, c]$	-1.2	$\rho^{|i-j|}$
`GaussianMixtureAR1`	symmetric two-component	< 0, bimodal	$\rho^{|i-j|}$

This is the experimental design in one table. Any method that uses only second-order information — and Chapter 6 shows that a very standard approximation does exactly that — cannot tell these five apart, by construction. Any difference in performance across the row is therefore attributable to higher-order structure and nothing else. There is no confound to argue about.

Implementation

`src/priors.py` → `GaussianAR1`, `LaplaceAR1`, `StudentTAR1`, `UniformAR1`, `GaussianMixtureAR1`. Each is a frozen dataclass holding only ρ, with `q`, `sample`, `log_transition_matrix` and `innovation_excess_kurtosis` as properties. The variance matching of §3.2.2 is built into each class rather than imposed by the caller, so the families cannot accidentally be compared at mismatched covariance.

3.3 Why this is a reasonable model of anything

Two defences, and it is worth being honest that they are defences rather than derivations.

Sequential data has temporal coherence. Diffusion models are increasingly applied to video and other sequential data, where consecutive frames are strongly dependent. A first-order Markov chain is the simplest non-trivial model of that dependence. It is not a model *of* video; it is a model of the property that makes video different from a bag of independent images.

It is a controllable probe. The stronger justification is methodological. The set of distributions for which the population score is computable is small, and every member of it is a modelling compromise. This one buys genuine nonlinearity of the score — which Gaussians do not have — at the price of a strong structural assumption. Given that the aim is to study how estimators exploit structure, having structure one can dial is the point.

Assumption at work

The model assumes *exact* first-order Markov structure. If the data has longer-range dependence, an estimator built on (3.2) is misspecified, and no amount of data repairs a misspecification. This is the single largest limitation of everything that follows, and it is not mitigated anywhere in this document.

3.4 Sampling from the chain

Generating data is direct: draw a_1 from μ, then iterate (3.3). There is no rejection, no burn-in, no approximation.

Implementation

Each prior's `sample(rng, n)` returns *one* sequence of length `n`, built by the recursion above with a_1 drawn standard normal. Batches are formed by stacking, e.g. `experiments/exp_16_sampling_validation.py` $\to$ `sample_chains`. The single-sequence signature is a deliberate simplicity, not an oversight; batching is the caller's business.

Remark 3.1 (Covariance-stationary, not strictly stationary). The computation above establishes exactly one thing: with $q = 1 - \rho^2$ and $\mathrm{Var}(a_1) = 1$, *second* moments propagate unchanged, so $\mathrm{Var}(a_i) = 1$ for every i and $\mathrm{Cov}(a_i, a_j) = \rho^{|i-j|}$, whatever the shape of φ. The chain is **covariance-stationary** (equivalently second-order stationary).

It is **not** strictly stationary in distribution unless φ is Gaussian. The invariant law of $a_i = \rho a_{i-1} + \varepsilon_i$ is the law of $\sum_{k\geq 0} \rho^k \varepsilon_{-k}$, an infinite convolution which for non-Gaussian ε is not Gaussian; drawing $a_1 \sim \mathcal{N}(0, 1)$ therefore starts the chain off its invariant law, and the marginal of a_i drifts towards that law over a burn-in of order $1/\log(1/\rho) \approx 6$ sites at $\rho = 0.85$. Choosing $q = 1 - \rho^2$ alone does not buy strict stationarity, and this document does not claim it does.

Nothing in the comparisons depends on the difference. Every family shares the same a_1 law and the same covariance, so they still differ only beyond second moments, which is the property the controlled probe needs. But the alternative — sampling a_1 from each family's invariant law, which needs a fixed-point computation per family — has not been run, so the claim that it would change nothing is a prediction and not a measurement. It is on the list in §9.7.

Part II

Functional inference

Chapter 4

Graphical models and factor graphs

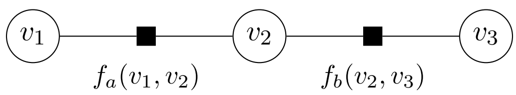
Chapter 3 gave a factorised density. This chapter turns factorisation into a picture, because the picture is what makes the key result of Chapter 5 obvious rather than surprising.

4.1 Factor graphs

A *factor graph* represents a density that is a product of local terms,

$$P(v_1, \ldots, v_N) \;\propto\; \prod_{\alpha} f_\alpha(v_{\partial\alpha})\,, \tag{4.1}$$

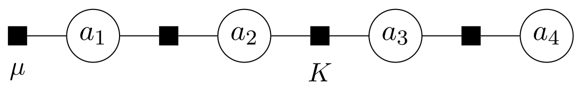
where each factor f_α depends on a subset $\partial\alpha$ of the variables. The graph is bipartite: a circle for each variable, a square for each factor, and an edge between them whenever the variable appears in that factor.



> The graph records *which variables interact directly.* Two variables not sharing a factor influence each other only through intermediaries. That is the entire content of the representation, and it is what will let us reorganise an intractable integral.

4.2 The Markov chain as a factor graph

For (3.2), the factors are $\mu(a_1)$ and one $K(a_i \mid a_{i-1})$ per consecutive pair. Each transition factor touches exactly two variables, adjacent ones, so the graph is a path:



4.3 Trees, and why they are special

Definition 4.1. A factor graph is a *tree* if it contains no cycles: between any two nodes there is exactly one path.

A path graph is a tree. This is the structural fact everything rests on.
The reason trees are special is the *separator* property.

Proposition 4.2 (Separation)**.** *Let the graph be a tree and let a_i be a variable node. Removing a_i disconnects the graph into components. Conditional on a_i, the variables in different components are independent.*

Proof. Because the graph is a tree, every factor involves variables lying in $\{a_i\}$ together with exactly one component — if a factor touched two components it would create a second path between them, hence a cycle. Group the factors by component: $P \propto \prod_c g_c\big(a_i, v^{(c)}\big)$ where $v^{(c)}$ are the variables of component c. Fixing a_i, the density in the remaining variables factorises across components, which is independence. $\square$

On a chain, this says: if you know a_i, then knowing anything to its left tells you nothing further about anything to its right. All the influence between the two sides is routed through a_i. That is why one can summarise everything to the left of site i in a single function of a_i — there is nothing else about the left that the right needs to know.

Assumption at work

The proof used acyclicity twice. On a graph with a cycle, a factor *can* join two components, the density does not factorise on conditioning, and the summary function is no longer sufficient. Belief propagation applied anyway ("loopy BP") double-counts information travelling around the cycle: a message leaving a node returns to it having been treated as independent evidence. It may still work well in practice, but it is no longer exact, and none of the guarantees in Chapter 5 survive.

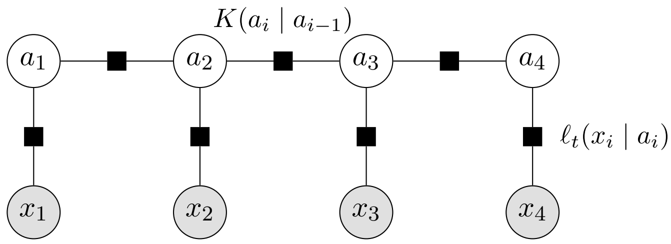
4.4 Conditioning on the observations

Now the step that makes the project work.

We observe x, related to a by (1.6). By Bayes' rule the posterior is prior times likelihood, and by (1.8) the likelihood factorises over sites:

$$\boxed{P(a \mid x, t) \;\propto\; \underbrace{\mu(a_1)\prod_{i=2}^{L} K(a_i \mid a_{i-1})}_{\text{prior: chain}} \cdot \underbrace{\prod_{i=1}^{L} \ell_t(x_i \mid a_i)}_{\text{likelihood: unary}}\;.} \tag{4.2}$$

Every likelihood factor touches *one* latent variable. In factor-graph terms it is a pendant node hanging off a_i and connected to nothing else.



Proposition 4.3. *The factor graph of the posterior* (4.2) *is a tree.*

Proof. The latent variables and transition factors form a path, which is acyclic. Each likelihood factor has degree one among latent variables, so attaching it adds a leaf and cannot create a cycle. The observations are fixed, not variables. $\square$

This is the whole trick, and it is worth saying plainly why it is not obvious. Conditioning usually *creates* dependence — explaining away is the standard example. Here it does not, because each observation is generated from a single latent variable independently of all others. So conditioning reweights the node potentials and leaves the edge set alone. The posterior is a different chain, not a non-chain.

Assumption at work

This fails the moment the noise is correlated across coordinates. Then $P(x \mid a)$ does not factorise, the likelihood becomes a factor touching several a_i simultaneously, and the posterior graph acquires edges — generally becoming dense and losing tractability entirely. The coordinatewise structure of the OU noise (§1.5) is doing essential work here.

4.5 The cost that has been removed

Marginalising (4.2) naively to get $P(a_i \mid x, t)$ means integrating over $L - 1$ variables — exponential in the sequence length if done by brute force on a discretisation. Proposition 4.3 says the posterior has the structure that makes this cost linear instead. Chapter 5 carries that out.

Sources

Factor graphs and the sum-product algorithm in the form used here are Kschischang et al. [9]; the graphical-model formulation and the separation properties are Pearl [10]. For the statistical-physics reading of the same objects, including exactness on trees, see Mézard and Montanari [11]; for the exponential-family and variational view, Wainwright and Jordan [12].

Chapter 5

Belief propagation

Chapter 4 showed the posterior is a tree. This chapter turns that into an algorithm and proves it exact.

5.1 The problem

We want $P(a_i \mid x, t)$ for each i, from

$$P(a \mid x,t) \;\propto\; \mu(a_1)\prod_{i=2}^{L} K(a_i \mid a_{i-1})\prod_{i=1}^{L} \ell_t(x_i \mid a_i). \tag{5.1}$$

By definition this means integrating out the other $L-1$ variables. Discretising each variable to N_g values and summing naively costs N_g^{L-1} operations — at $N_\mathrm{g} = 401$, $L = 32$ that is about 10^{80}, so the naive route is not merely slow but impossible.

5.2 The idea: factor out what does not depend on the summation variable

Consider a small case, $L = 3$, and write $\psi_i(a_i) := \ell_t(x_i \mid a_i)$. We want $P(a_3 \mid x)$, so we integrate over a_1 and a_2:

$$P(a_3 \mid x) \propto \psi_3(a_3)\int \mathrm{d}a_2 \int \mathrm{d}a_1\; \mu(a_1)\psi_1(a_1)\, K(a_2 \mid a_1)\, \psi_2(a_2)\, K(a_3 \mid a_2).$$

The inner integral over a_1 involves only the first three factors — $K(a_3 \mid a_2)$ does not contain a_1, so it can be pulled out. Define

$$\overrightarrow{m}_2(a_2) := \int \mathrm{d}a_1\; \mu(a_1)\,\psi_1(a_1)\,K(a_2 \mid a_1),$$

a function of a_2 alone. Then

$$P(a_3 \mid x) \propto \psi_3(a_3)\int \mathrm{d}a_2\; \overrightarrow{m}_2(a_2)\,\psi_2(a_2)\,K(a_3 \mid a_2) =: \psi_3(a_3)\,\overrightarrow{m}_3(a_3).$$

Two integrals became two *nested* integrals, each over one variable. The saving is the whole algorithm: instead of a single sum over a grid of size N_g^2, we do two sums of size N_g each, reusing the first result. That is dynamic programming, and it works because the factorisation lets the summand split.

5.3 The general recursion

Generalising, define the *forward* (left) and *backward* (right) messages

$$\boxed{\begin{aligned} \overrightarrow{m}_i(a_i) &\propto \int \mathrm{d}a_{i-1}\, K(a_i \mid a_{i-1})\, \psi_{i-1}(a_{i-1})\, \overrightarrow{m}_{i-1}(a_{i-1}), && \overrightarrow{m}_1 := \mu, \qquad (5.2)\\ \overleftarrow{m}_i(a_i) &\propto \int \mathrm{d}a_{i+1}\, K(a_{i+1} \mid a_i)\, \psi_{i+1}(a_{i+1})\, \overleftarrow{m}_{i+1}(a_{i+1}), && \overleftarrow{m}_L :\equiv 1. \qquad (5.3)\end{aligned}}$$

The base cases matter. $\overrightarrow{m}_1 = \mu$ is the only place the initial law enters; without it the prior on the first site would be silently dropped. $\overleftarrow{m}_L \equiv 1$ says there is no evidence to the right of the last site — an improper flat function, which is fine because it is always multiplied by something integrable.

Proposition 5.1 (Exactness)**.** *With* (5.2)–(5.3)*,*

$$P(a_i \mid x, t) \;\propto\; \overrightarrow{m}_i(a_i)\, \psi_i(a_i)\, \overleftarrow{m}_i(a_i), \tag{5.4}$$

and the pairwise marginal on the edge $(i-1, i)$ *is*

$$P(a_{i-1}, a_i \mid x, t) \;\propto\; \overrightarrow{m}_{i-1}(a_{i-1})\, \psi_{i-1}(a_{i-1})\; K(a_i \mid a_{i-1})\; \psi_i(a_i)\, \overleftarrow{m}_i(a_i). \tag{5.5}$$

Proof. By induction. Unrolling (5.2),

$$\overrightarrow{m}_i(a_i) \propto \int \mathrm{d}a_{1:i-1}\; \mu(a_1) \prod_{j=2}^{i} K(a_j \mid a_{j-1}) \prod_{j=1}^{i-1} \psi_j(a_j),$$

which is exactly the joint density of everything at or left of site i, with the left variables integrated out and ψ_i *not* yet included. Symmetrically $\overleftarrow{m}_i(a_i)$ is the joint over everything strictly right of i, integrated out, given a_i. By Proposition 4.2 these two are conditionally independent given a_i, so their product times the local evidence $\psi_i(a_i)$ is proportional to the marginal. The pairwise statement is the same argument with the separator taken to be the edge rather than a node. □

Assumption at work

Note where ψ_i appears. In (5.2) the local evidence at site $i-1$ is absorbed *before* the transition, so $\overrightarrow{m}_i$ carries $\psi_1, \ldots, \psi_{i-1}$ and $\overleftarrow{m}_i$ carries $\psi_{i+1}, \ldots, \psi_L$. The marginal (5.4) therefore contains each ψ_j exactly once. Absorbing the local evidence *after* the transition instead would put ψ_i into both messages and square it — the belief would be sharper than the truth, and every posterior variance too small. Getting this wrong produces plausible-looking output, which is why the convention is pinned by a test rather than left to a comment.

Implementation

`src/bp_grid.py` → `grid_bp` implements (5.2)–(5.4) for one sequence, returning the beliefs and the log-evidence. `src/bp_grid.py` → `grid_bp_batch` does the same for a batch, sharing the site loop so each message update is a single $(N_\mathrm{g} \times N_\mathrm{g}) \cdot (N_\mathrm{g} \times B)$ matrix product. The initialisations are literally `L[0] = exp(log_mu)` and `R[-1] = 1.0`. Pairwise statistics (5.5) are accumulated in `src/em.py` → `e_step`.

5.4 Cost

Each message update integrates one variable against the kernel: on a grid of N_g points that is a matrix–vector product, $O(N_\mathrm{g}^2)$. There are $2L$ updates, so one sequence costs

$$O(LN_\mathrm{g}^2) \tag{5.6}$$

instead of $O(N_\mathrm{g}^L)$. At $L = 32$, $N_\mathrm{g} = 401$ this is about 5×10^6 operations — microseconds — against the 10^{80} of the naive route.

The exponential became linear in the sequence length. Nothing was approximated to achieve it; the saving is entirely from reorganising the order of integration, which the tree structure permits and a general graph does not.

5.5 Reading off the answer

From the belief $b_i(a_i) := P(a_i \mid x, t)$ we get the posterior mean by one more integral,

$$\langle a_i \rangle_{x,t} = \int \mathrm{d}a_i \; a_i \, b_i(a_i), \tag{5.7}$$

and then the score from (2.2). That is the complete pipeline: messages, beliefs, mean, score.

Implementation

`src/denoiser.py` → `bp_posterior_mean` is the single entry point used by every experiment — it takes anything exposing `log_transition_matrix`, which covers both the true priors of `src/priors.py` and the estimated kernels of `src/kernels.py`. That shared interface is what makes "exact inference under the true kernel" and "inference under a learned kernel" the same code path with a different argument, so no comparison between them can be confounded by an implementation difference.

5.6 Evidence, and why it is worth carrying

The normalising constant of (5.4) is the *evidence* $P_t(x)$. Messages are renormalised at each step to prevent underflow, and if the discarded constants are accumulated, their product recovers $\log P_t(x)$ exactly.

This is not bookkeeping for its own sake. The evidence is the objective that the estimation of Chapter 8 maximises, and having it in closed form is what makes it possible to check a gradient against a finite difference of the thing being optimised.

Implementation

`src/bp_grid.py` → `grid_bp` returns `log_evidence` alongside the beliefs. `src/exact_scores.py` → `gaussian_log_evidence` gives the closed form in the Gaussian case, which is what the general routine is tested against.

5.7 The relationship to forward–backward and to Kalman

Equations (5.2)–(5.3) are the forward–backward algorithm of hidden Markov models, written for a continuous state. If the latent state is discrete the integrals become sums and this is exactly Baum's algorithm; if everything is Gaussian, messages are determined by two numbers each and the recursion becomes the Kalman filter and RTS smoother.

> These are not three algorithms but one, specialised by what the messages are. Discrete: a vector. Gaussian: a mean and a variance. General continuous: a function, which must be represented somehow — and *that* representation is the only approximation in the whole scheme. Chapter 7 is about it.

5.8 Validation

Two independent checks establish that the implementation computes what this chapter claims.

Against brute force. For a short chain and a coarse grid, the discretised posterior can be enumerated exhaustively — all N_{g}^{L} configurations — and marginalised by summation, using no messages at all. This shares no code with the recursion. Agreement is at machine precision: posterior means to 1.6×10^{-14}, log-evidence to 1.8×10^{-15}, and the pairwise statistic to 3.9×10^{-16}, with the total pairwise mass equal to the edge count exactly.

Against a closed form. On a Gaussian chain the posterior mean is known analytically (2.4), and the recursion reproduces it to 9.2×10^{-15} relative at the working resolution.

Implementation

`tests/test_em_bp.py` contains the enumeration check; `tests/test_grid_convergence.py` and `tests/test_score_mean_error_identity.py` contain the Gaussian and identity checks. These tests are the evidence for the numbers above and are the reason those numbers can be quoted at all.

Sources

The recursions of this chapter are the forward–backward algorithm for hidden Markov models [13], of which Rabiner [14] is the standard exposition. In the linear-Gaussian case they specialise to the Kalman filter [15] and the Rauch–Tung–Striebel smoother [16]. Cappé et al. [17] treats the continuous-state case in the generality needed here. That U-Net architectures can be read as approximate belief propagation on a hierarchical model is argued by Mei [18].

Chapter 6

The Gaussian case and the LMMSE baseline

The Gaussian chain plays two roles: it is the case where everything is computable in closed form, hence the yardstick for validating the general machinery; and it is the model implicitly assumed by the most common approximation, hence the baseline everything is measured against.

6.1 Everything in closed form

Take $\varphi = \mathcal{N}(0, q)$. Then a is jointly Gaussian with covariance $\Sigma_0 = \rho^{|i-j|}$ from (3.4), and by (1.6) so is x, with

$$\Sigma_t \; = \; e^{-2t}\Sigma_0 + \Delta_t I. \tag{6.1}$$

For jointly Gaussian variables the conditional mean is linear, giving (2.4): $\langle a \rangle_{x,t} = e^{-t}\Sigma_0\Sigma_t^{-1}x$ and $S = -\Sigma_t^{-1}x$.

The score is a fixed linear map. No inference is required at all — one matrix solve. This is exactly why Gaussian data is a poor probe of how models exploit structure: the answer is linear regression, and every method that can do linear regression is optimal.

6.1.1 The precision matrix is tridiagonal

A useful structural fact. The clean precision $Q_0 = \Sigma_0^{-1}$ of an AR(1) chain is *tridiagonal*: writing the density as $P_0(a) \propto \exp[-\frac{1}{2q}\sum_i (a_i - \rho a_{i-1})^2]$ and expanding, only terms a_i^2 and $a_i a_{i-1}$ appear. So

$$(Q_0)_{ii} = \frac{1+\rho^2}{q} \text{ (interior)}, \qquad (Q_0)_{i,i\pm 1} = -\frac{\rho}{q}, \qquad (Q_0)_{ij} = 0 \text{ for } |i-j| > 1.$$

The posterior precision adds the likelihood's contribution, $J = (e^{-2t}/\Delta_t)I + Q_0$, and stays tridiagonal at every t.

Tridiagonal precision is the algebraic image of the Markov property: zero in the precision matrix means conditional independence given everything else. The *covariance* Σ_t is dense — every site is correlated with every other — while the precision is banded. Being Markov is a statement about the precision, not the covariance.

6.2 The moment closure

For a non-Gaussian innovation the messages of Chapter 5 are genuinely non-Gaussian functions. A natural approximation keeps only two numbers per message: replace each message by the

Gaussian with the same mean and variance. This is cheap, requires no grid, and is what one would write first.

Under the transition $a_i = \rho a_{i-1} + \varepsilon_i$ with ε of mean 0 and variance q, a message with moments (m, v) maps to

$$\text{forward:} \quad (m, v) \mapsto (\rho m,\ \rho^2 v + q); \qquad \text{backward:} \quad (m, v) \mapsto (m/\rho,\ (v + q)/\rho^2), \tag{6.2}$$

the backward map being the forward one inverted, because that recursion integrates over the *output* of the transition rather than its input. It requires $\rho \neq 0$.

Proposition 6.1. *For the chain* (3.3) *with* $\mathbb{E}[a] = 0$ *and Gaussian unary likelihoods, the moment-closed recursion returns*

$$\widehat{\mathbf{a}}_{\text{LMMSE}} \;=\; e^{-t}\Sigma_0 \left(e^{-2t}\Sigma_0 + \Delta_t I\right)^{-1} x, \tag{6.3}$$

the LMMSE estimator of the covariance-matched Gaussian model, for every *innovation law with those two moments.*

Proof. Both maps in (6.2) depend on φ only through $(0, q)$, and the likelihood absorption is a Gaussian–Gaussian product, which is exact. So the recursion returns the same function of x for every innovation law with those moments — in particular the same as on the covariance-matched *Gaussian* chain. On that chain every message is genuinely Gaussian, so the projection is the identity, the recursion is exact, and it returns the true posterior mean. For jointly Gaussian zero-mean (a, x) the posterior mean is linear and coincides with the LMMSE estimator of Proposition 1.2. ☐

Assumption at work

The zero-mean hypothesis is load-bearing. The general LMMSE estimator is $\mathbb{E}[a] + \mathrm{Cov}(a, x)\,\mathrm{Cov}(x)^{-1}(x - \mathbb{E}[x])$, and dropping the mean terms is only valid when both means vanish. With a prior mean of 1.3, (6.3) departs from the true LMMSE by 0.65 in maximum absolute value.

This is a sharper statement than "the Gaussian approximation is approximate". The closure does not merely lose some information about the innovation — it loses *all* of it beyond the variance. Two chains from the matched family of §3.2.2, one Gaussian and one heavily bimodal, are given identical treatment. The approximation is blind by construction, not by degree.

A consequence worth stating: at the single-Gaussian level, "error from representing messages badly" and "error from assuming the wrong model" are the same number. They are different objects only for richer message families.

Implementation

`src/bp_gaussian.py` → `gaussian_chain_bp` implements the closure in information form, carrying precision λ and information h rather than mean and variance. That parametrisation matters: a message conveying no information is $\lambda = 0$ exactly, whereas in moment form it is a variance of $+\infty$.

Assumption at work

An earlier version of this project implemented the closure by evaluating each Gaussian message *on the grid*, pushing it through the exact update, and re-fitting moments. At weakly informative t the outgoing message is nearly flat, and moment-matching a flat function *truncated* to $[-A, A]$ returns a variance of $(2A)^2/12$ rather than ∞ — an invented precision of order $3/A^2$. The error fed back through the recursion, driving messages towards the boundary. Measured maximum absolute error in the posterior mean at $t = 1.8$: $9.1\,\times$

10^{-1}, against 4.4×10^{-16} for the information form. The lesson is that the closure must be done analytically; representing an unnormalisable object on a finite grid is not a neutral implementation detail.

6.3 Where the approximation costs something

Since the closure is exactly LMMSE, the gap to exact inference is the value of the higher-order structure. Measured on a Laplace chain at $\rho = 0.85$, the relative deviation of the closure's score from the exact one falls monotonically from 0.198 at $t = 0.08$ to 9.9×10^{-5} at $t = 2.4$.

The direction may be surprising: the approximation is *worst at low noise.* The reason is that at large t the likelihood is weak and the posterior is dominated by the smoothing effect of the noise, which Gaussianises everything — so a Gaussian approximation of a nearly Gaussian object is nearly exact. At small t the posterior is sharply peaked and its shape is inherited from the prior, whose non-Gaussianity is then fully visible.

Whether that matters for *generation* is a separate question, because errors at small t enter the reverse process where the denoiser is barely acting. That question is what the experiments of Chapter 9 address, and the answer is not the obvious one.

Implementation

`experiments/exp_02_laplace_gaussian_message_error.py` → produces the numbers above; `experiments/exp_03_nongaussian_innovation_sweep.py` → extends them across the innovation families of §3.2.2.

Sources

The linear-minimum-mean-square-error estimator and its orthogonality characterisation are standard [19, 20]. The Gaussian message recursions coincide with Kalman filtering and RTS smoothing [15, 16].

Part III

Numerical representation

Chapter 7

Representing messages: the numerical layer

Chapter 5 gave an exact algorithm on functions. A computer cannot store a function, so an implementation must choose a representation — and that choice is the *only* approximation in the whole scheme. This chapter is about it, because conflating this approximation with the exactness of the previous chapter is the easiest way to overstate what the method does.

7.1 Three statements that must be kept apart

1. **Sum-product on the posterior chain is exact**, as a theorem about tree-structured factorisations (Proposition 5.1). This is a mathematical statement about functional messages and involves no computer.
2. **The discretised recursion is exact for the discretised model.** Restricting messages to a grid defines a different, finite-state model; on that model the recursion is exact up to floating-point arithmetic.
3. **The discretised model approximates the continuous one**, with an error that must be *measured*.

Assumption at work

Writing simply "our method is exact" collapses (1) and (3) and claims something false. Throughout this project we write *exact tree inference* for (1) and *exact for the discretised model* for (2), and never abbreviate. The distinction is not pedantry: (1) holds for any innovation law whatsoever, while the error in (3) depends strongly on the smoothness of that law — as §7.3.2 shows.

7.2 The grid

Messages are represented by their values on N_{g} equally spaced points on $[-A, A]$, and integrals become composite trapezoidal sums,

$$\int \mathrm{d}a\, f(a) \;\approx\; \sum_{k=1}^{N_{\mathrm{g}}} w_k f(u_k), \qquad w_k = \Delta u \text{ (interior)}, \quad \tfrac{1}{2}\Delta u \text{ (endpoints)}. \tag{7.1}$$

Each message update (5.2) becomes a matrix–vector product with the matrix $K(u_l \mid u_k)$, which is formed once per model and reused at every site and every sequence.

Implementation

`src/bp_grid.py` → `make_grid` builds the points and trapezoidal weights. The transition matrix comes from each prior's `log_transition_matrix(grid)` (`src/priors.py`) or each estimated kernel's (`src/kernels.py`) — the same interface, which is why the true and learned models are interchangeable in every experiment.

7.3 Two error sources

7.3.1 Truncation

Probability mass outside $[-A, A]$ is discarded rather than transported. The diagnostic is the mass in the boundary cells: if it is not negligible, A is too small and refining N_g will not help, because the error is in the domain rather than the resolution.

This diagnostic used to be computed inside the recursion and thrown away, so no aggregate could be quoted. It is now recorded (`outputs/frozen/exp_18/boundary.csv`, `experiments/exp_18_revision_diagnostics.py`). The quantity is the maximum over all 32 sites of a forward sweep on a Laplace chain at $\rho = 0.85$ — which makes it a *statistic of a sampled trajectory*, not a bound. It is therefore reported over 256 chains per cell and at three quantiles, taking in each case the worst value over the twelve-level noise schedule:

A	worst chain	90th pct.	median
4	2.3×10^{-3}	2.7×10^{-4}	3.8×10^{-5}
6	1.6×10^{-4}	2.5×10^{-6}	3.8×10^{-7}
8	1.2×10^{-6}	1.5×10^{-8}	3.0×10^{-9}
10	3.3×10^{-8}	7.8×10^{-11}	1.8×10^{-11}

The value is essentially independent of N_g across $\{201, 401, 801\}$, which is the point: truncation responds to the domain and not to the resolution, and a grid-refinement sweep at fixed A therefore cannot certify it. At the working $A = 8$ the discarded mass is around 10^{-6} in the worst chain, four orders of magnitude below the smallest effect this project reports and still safe — but the spread between the rows is why all three are given. The worst chain carries 84× the 90th percentile and 390× the median, so a figure taken from one trajectory lands near the median and understates the tail by two to three orders of magnitude.

Assumption at work

The frozen configuration justified $A = 8$ with "the truncation residual is 1.4×10^{-8}" for several weeks. That number is the 90*th percentile*, not a bound: the worst chain is 1.2×10^{-6}. The conclusion did not change — both are far below every reported effect — but the sentence claimed a guarantee it did not have, which is the same category of error as reading one trajectory's worst site as a bound. Quote the statistic you computed, and say which statistic it is.

Assumption at work

This failure is invisible unless one looks for it. An earlier package in this project ran at $A = 8$ with a boundary mass of 7.4×10^{-7}, above its own tolerance, and the symptom was not an error message but a slow drift in the posterior means at large t. Widening the domain at preserved spacing reduced it to 5.3×10^{-10}.

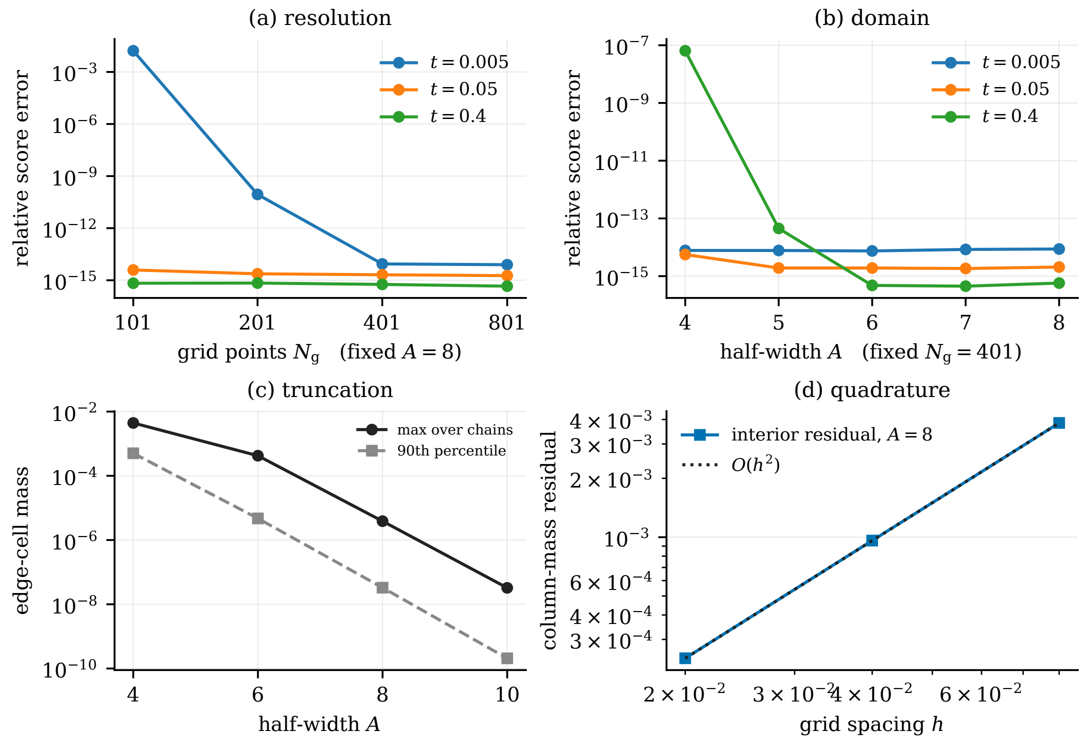


Figure 7.1: The two error sources, measured separately. (a) Relative score error against the number of grid points at fixed $A = 8$, and (b) against the half-width at fixed $N_g = 401$; both saturate at the floating-point floor $\sim 10^{-14}$, beyond which the curves measure arithmetic rather than discretisation. (c) Worst boundary mass over 32 sites against A: the truncation diagnostic, near-independent of N_g. (d) Interior column-mass residual of the transition matrix against spacing at $A = 8$: the quadrature diagnostic, following $O(h^2)$ as the trapezoidal rule predicts. Panels (a)–(b) from `outputs/exp_01_grid_validation/grid_heatmap.csv`, (c)–(d) from `outputs/exp_18/boundary.csv`.

7.3.2 Quadrature

The trapezoidal rule has error $O(\Delta u^2)$ for integrands with a bounded second derivative. Whether that bound applies depends on the innovation.

- **Gaussian innovation.** The integrand is smooth and convergence is fast.
- **Laplace innovation.** $\varphi(e) \propto e^{-|e|/b}$ has a discontinuous derivative at $e = 0$. The second derivative is not bounded there, the standard estimate does not apply, and convergence is second order at best.

Assumption at work

It is tempting — and this project did it in an earlier draft — to describe the discretisation as "spectrally accurate" on the strength of the Gaussian measurements. That generalisation is not available: spectral accuracy requires smoothness the Laplace kernel does not have. The accompanying note now claims second-order behaviour for that family and nothing more.

7.3.3 A resolution condition, and what really happens at small t

At small t the likelihood $\ell_t(x_i \mid a_i)$ is a narrow Gaussian of width $\sqrt{\Delta_t}/e^{-t}$. If that width is comparable to the grid spacing, the integrand is under-resolved and the quadrature is meaningless however fine the rest looks. The working condition used here is $\sqrt{2t} \gtrsim 3\Delta u$.

Everything gets harder as $t \to 0$, and it is worth being precise about why, because the usual one-line explanation is wrong. It is often said that the score diverges as $\Delta_t \to 0$ because (2.2) carries a factor $1/\Delta_t$. It generally does not diverge: the numerator $x - e^{-t}\langle a\rangle_{x,t}$ vanishes at the same order, and for a P_0 with a continuously differentiable density $S(x,t) \to \nabla \log P_0(x)$, a finite limit. Four separate effects are at work, and only the first is a genuine mathematical singularity.

1. **A real singularity, for one family only.** The uniform-innovation chain has a compactly supported P_0, so $\nabla \log P_0$ is unbounded at the support boundary. The Laplace chain has a density with a kink, so its score has a jump discontinuity but stays bounded. The Gaussian and mixture chains have scores bounded on compacta.

2. **Numerical cancellation.** The implementation forms a difference of two nearly equal quantities and divides by $\Delta_t \approx 2t$, so relative error is amplified by $O(1/t)$ no matter how accurate the posterior mean is.

3. **Under-resolution.** The likelihood factor has width $\sqrt{\Delta_t}$; once that is comparable to h the quadrature integrates it with $O(1)$ points. That is the condition stated just above.

4. **Integrator stiffness.** The reverse drift's Jacobian scales like $1/\Delta_t$, so the step size must shrink as $t \to 0$. This is why `src/reverse.py` → `time_grid` is geometric rather than uniform.

The practical conclusion — stop at $t_{\min} > 0$, and treat the small-t end of every table with the most suspicion — is unchanged. The reason for it is not a divergence of the score.

7.4 Stability

Messages are products of many small numbers and underflow quickly. Two standard devices: work in the log domain where possible with per-row maximum subtraction, and renormalise each message to unit quadrature mass after every step, accumulating the discarded constants so the evidence is recovered exactly.

Implementation

Both appear in `src/bp_grid.py` → `grid_bp`: `log_ell -= log_ell.max(...)` before exponentiating, and an explicit mass check that raises `FloatingPointError` if a message loses all its mass rather than silently returning garbage.

7.5 The stopping-time problem

Because integration stops at $t_{\min} > 0$, what a sampler produces is a draw from $P_{t_{\min}}$, not from P_0. This sounds like a technicality and is not.

Independent Gaussian noise of variance v added to a quantity of variance q multiplies its excess kurtosis by $(q/(q+v))^2$. At $\rho = 0.85$ and $t_{\min} = 0.02$ this predicts a generated excess kurtosis of 1.910 where the clean chain has 3.0.

Assumption at work

This project measured exactly 1.91 for the reference sampler and initially read it as a failure of the integrator to converge. It was not: the sampler was reproducing $P_{t_{\min}}$ correctly and being compared against the wrong target.

The obvious repair — apply a final denoising step and report $\langle a\rangle_{x,t}$ instead of x — *inverts* the bias rather than removing it. The posterior mean of a heavy-tailed prior is a shrinkage operator: it pulls small values towards zero harder than large ones, so the law of the posterior mean is *more* peaked than P_0. Measured overshoot: 3.73, 4.34, 4.74 at $t_{\min} = 0.02, 0.05, 0.10$ against a true 3.0 — and identical at $N_{\mathrm{g}} = 401$ and $N_{\mathrm{g}} = 801$, which rules out grid resolution as the cause.

The correct treatment is to compare like with like: draw from P_0, noise it forward to the same $t_{\min}$, and compare. Both sides are then samples of the same law, no estimator sits between the sampler and the metric, and the target moments follow in closed form.

Implementation

`experiments/exp_16_sampling_validation.py` → `reference_at` builds the forward-noised reference; `target_innovation_moments` gives the closed-form target. The docstrings there record both failed designs, since the reasoning is easier to follow than to rediscover.

7.6 Running on a GPU

The recursion is a sequence of dense matrix products, which suits a GPU. Rather than write a second implementation, `src/backend.py` parameterises the existing one by its array module, so CPU and GPU execute the same code.

> Two implementations of an exact algorithm is a bad trade. They agree on the easy cases and drift where nobody is looking, and the exactness claims of §5.8 would then hold for one device and be assumed for the other.

Implementation

`tests/test_backend_parity.py` gates every GPU number: device agreement, the closed-form Gaussian check re-run on device, float64 preservation, and invariance to batch width. Measured on an NVIDIA H200: posterior means agree to 1.8×10^{-16}–8.0×10^{-16}, variances to 9.3×10^{-15}, and the GPU reproduces the closed-form Gaussian score to 4.2×10^{-16}. `hpc/bocconi_gpu.sbatch` → runs that suite before any sweep and exits if it fails.

Assumption at work

The availability probe originally tested that a GPU array could be *allocated*. Allocation succeeds without cuBLAS loaded, so on a misconfigured node the check passed and every matrix product then failed inside the recursion. It now probes an actual matrix product. A capability check must exercise the operation that will be used, not a cheaper proxy.

Part IV

Learning the transition

Chapter 8

Estimating the transition kernel

Everything so far assumed the chain is known. This chapter removes that assumption: we observe only noised sequences and must estimate K from them, never seeing a clean sample.

8.1 The objective

Write θ for the kernel parameters. The observed-data log-likelihood is

$$\mathcal{L}(\theta) \;=\; \sum_{\mu=1}^{M} \log P_{t_\mu,\theta}(x^\mu), \qquad P_{t,\theta}(x) = \int \mathrm{d}a\; P_0^\theta(a)\, P(x \mid a, t). \tag{8.1}$$

The integral is the same marginalisation Chapter 5 made tractable — it is the evidence, and BP already returns it.

This is a latent-variable problem in the textbook sense: the clean sequence a is the latent variable, the noised x is observed, and the parameters live in the prior. What is unusual is that the latent structure is a tree, so the quantities EM needs are available *exactly* rather than by approximation — which is not the typical situation.

8.2 Expectation maximisation

EM alternates two steps. Given a current $\theta^{(r)}$, form

$$\mathcal{Q}(\theta \mid \theta^{(r)}) = \sum_{\mu} \mathbb{E}_{\theta^{(r)}}\left[\log P_\theta(a, x^\mu) \mid x^\mu\right], \tag{8.2}$$

then set $\theta^{(r+1)} = \arg\max_\theta \mathcal{Q}$.

Proposition 8.1 (Monotone ascent)**.** $\mathcal{L}(\theta^{(r+1)}) \geq \mathcal{L}(\theta^{(r)})$.

Proof. For any θ, write $\log P_\theta(x) = \mathcal{Q}(\theta \mid \theta^{(r)}) - H(\theta \mid \theta^{(r)})$ with $H(\theta \mid \theta^{(r)}) = \mathbb{E}_{\theta^{(r)}}[\log P_\theta(a \mid x) \mid x]$. By Gibbs' inequality $H(\theta \mid \theta^{(r)}) \leq H(\theta^{(r)} \mid \theta^{(r)})$ for every θ, since the difference is a Kullback–Leibler divergence. Choosing $\theta^{(r+1)}$ to increase $\mathcal{Q}$ therefore increases $\mathcal{L}$ by at least as much. □

Monotonicity is the reason EM is trustworthy without a step size. It is also the sharpest available test of an implementation: an error anywhere — in the recursion, the accumulation, or the maximisation — generically breaks it. In every run reported in this project the violation is exactly zero.

8.3 The expectation step is exact

In a general latent-variable model (8.2) is intractable and one resorts to a variational or sampled approximation, losing monotonicity with it. Here the posterior is a tree, so Proposition 5.1 gives the required expectations exactly.

The complete-data log-likelihood is a sum over edges, so the *θ-dependent part* of $\mathcal{Q}$ — it also contains a site-one term and a likelihood term, both θ-free and therefore additive constants — depends on the data only through

$$\boxed{\Xi_{kl} \;=\; \sum_{\mu=1}^{M}\sum_{i=2}^{L} P(a_{i-1}=u_k,\; a_i=u_l \,|\, x^{\mu})\,,} \tag{8.3}$$

the expected transition mass between grid points, accumulated from the pairwise marginals (5.5).

Ξ is the continuum analogue of the expected transition counts of Baum–Welch. In the discrete case it literally counts how often the chain was inferred to move from one state to another; here it is a mass on pairs of grid points.

Assumption at work

Sufficiency holds only under four conditions: a *homogeneous* kernel shared across sites and sequences, a fixed grid, an initial law excluded from θ, and θ-free likelihood factors. The last is what licenses observations at several noise levels to *add* into one Ξ. Relax any of them and (8.3) is no longer sufficient.
Note also what sufficiency does and does not buy. The maximisation step never revisits the data, so it costs $O(PN_\mathrm{g}^2)$ independently of M. But *forming* Ξ costs $O(MLN_\mathrm{g}^2)$ per iteration and $O(N_\mathrm{g}^2)$ storage. "The whole E-step is one matrix" is a statement about sufficiency, not about inference being cheap.

Implementation

`src/em.py` $\to$ `e_step` accumulates (8.3); `src/em.py` $\to$ `ExpectedStatistics` holds it; `src/em.py` $\to$ `fit_em` is the driver with the monotonicity trace. `src/em.py` $\to$ `clean_statistics` is the $t \to 0$ limit, which needs no BP at all — the clean-data case Marc's original suggestion referred to.

8.4 Fisher's identity: a gradient without differentiating the recursion

There is a second route to the same information. Fisher's identity states

$$\boxed{\nabla_\theta \mathcal{L}(\theta) = \sum_{\mu} \mathbb{E}_\theta[\nabla_\theta \log P_\theta(a, x^{\mu}) \,|\, x^{\mu}] = \big\langle \Xi(\theta),\; \nabla_\theta \log K_\theta \big\rangle.} \tag{8.4}$$

Proof. $\nabla \log P(x) = \nabla P(x)/P(x)$ and $\nabla P(x) = \int \mathrm{d}a\, \nabla P(a,x) = \int \mathrm{d}a\, P(a,x) \nabla \log P(a,x)$; dividing by $P(x)$ turns $P(a,x)/P(x)$ into the posterior. The second equality follows because $\log P_\theta(a,x)$ depends on θ only through the edge terms. $\square$

This says something practically important. BP is not a computation graph to be backpropagated through — it is the oracle that supplies the conditional expectation. The exact gradient of the *marginal* likelihood comes out of one forward–backward pass. Advice received elsewhere on this project held that EM here would require moving the recursion into an automatic-differentiation framework; that is not so, and the entire implementation is plain array arithmetic.

Assumption at work

Fisher's identity is *not* EM. It licenses gradient ascent on $\mathcal{L}$; EM maximises $\mathcal{Q}$ at each step. The two are different algorithms and neither dominates. Measured here: on the smooth Gaussian kernel gradient ascent reaches EM's optimum to 2×10^{-9} nats while needing a tuned step size (at too large a step it diverges outright); on the Laplace kernel it attains a *higher* likelihood than EM, because there the exact maximiser of $\mathcal{Q}$ is a lattice artefact of the discretisation.

Implementation

`src/em.py` → `q_gradient` implements (8.4); `experiments/exp_08_gradient_vs_exact_mstep.py` → `exp_08` is the comparison. The identity is checked against a central finite difference of the exact evidence in `tests/test_em_bp.py`, agreeing to a relative 1.5×10^{-9}.

8.5 The kernel families

class	free parameters	maximisation step
`GaussianAR1Kernel`	(ρ, q), 2	closed form: belief-weighted Yule–Walker
`LaplaceAR1Kernel`	(ρ, b), 2	closed form, via a weighted median
`MixtureInnovationKernel`	ρ and C triples, $3C$	inner ECM sweeps
`MDNKernel`	network weights	gradient, with backtracking

The mixture kernel is the one used for the headline results,

$$K_\theta(a' \mid a) = \sum_{c=1}^{C} \pi_c \mathcal{N}\big(a' - \rho a;\ \nu_c,\ s_c^2\big)\,, \tag{8.5}$$

because it keeps the linear autoregression while leaving the innovation law essentially free. It has never been shown the density it is asked to recover.

Counting the parameters. At $C = 4$: ρ, four weights, four locations, four scales — thirteen numbers, subject to $\sum_c \pi_c = 1$, hence **twelve free**.

Assumption at work

Two subtleties in that count. First, the zero-mean condition $\sum_c \pi_c \nu_c = 0$ is *not* enforced: imposing it by reprojection is not a maximisation step and broke monotone ascent in testing, so it is monitored as a diagnostic instead (it lands at the 10^{-3} level). A reader who assumed the model class of §3.2.2 literally would compute eleven. Second, and consequently, Proposition 6.1 applies to the *true* kernel but not automatically to the estimated one, whose innovation is not exactly centred.
Third: the mixture M-step is not the exact maximiser of $\mathcal{Q}$. The complete-data problem carries a second latent variable — the mixture label — so the step is itself iterative, run as inner conditional-maximisation sweeps. That makes the algorithm *generalised* EM: monotone ascent is preserved, exact maximisation is not. It was for some time labelled "exact block M-step" in the source, which it never was.

Assumption at work

The inner sweep count was fixed at four, and four sweeps is not enough to settle the innovation *shape* when the outer loop is short. Measured at $C = 8$, $M = 256$, $t = 0.22$,

$N_g = 401$, holding the outer count at 40 and varying only the inner budget (the truth is $\kappa_4 = 3$):

inner sweeps	2	4	8	16	64
fitted κ_4	0.64	1.68	2.06	2.14	2.16
$\widehat{\rho}$	0.81838	0.81865	0.81977	0.82006	0.82005
$\mathcal{L}$/edge	−1.25791	−1.25717	−1.25705	−1.25705	−1.25707

The shape moves by 28% between four and sixteen sweeps while ρ moves in the fourth decimal — the same rate asymmetry the outer loop shows, where ρ is flat from about iteration 25 and the innovation shape needs of order a hundred, reappearing one level down. The objective is also non-monotone in this budget: 64 sweeps lands below 16. So this is a third instance of the pattern that runs through the whole project, after EM's outer budget and Rung 4a's gradient ascent: *the optimisation budget is a regularisation knob, and more ascent is not more accuracy.*

The scope of the effect matters, and the first version of this note overstated it by generalising from the 40-iteration column. At 60 outer iterations the inner budget stops mattering, because the two loops partially substitute — four sweeps get there too, just slower. At $M = 128$ the fitted κ_4 is 3.22 at four sweeps against 3.34 at sixteen; at $M = 512$ it is 2.97 against 2.93, i.e. non-monotone and within noise. In the efficiency run, whose outer count is selected on validation from a grid reaching 400, the median selection is 120 and 70% of cells sit at 60 or above — so most of that run is in the regime where this does not bite, and 30% of it is not.

The default is now sixteen, which removes the interaction rather than asking a reader to work out which regime a cell was in.

Assumption at work

The same measurements settle a resolution question that had been open. A mixture component narrower than a grid cell can raise the quadrature likelihood without corresponding to anything in the innovation law, so an under-resolved fit must never count as evidence for higher capacity. At $C = 8$, $N_g = 401$ the narrowest fitted component is $s_{\min}/h \approx 9.4$–10.0: nearly ten cells wide, five times the threshold below which a component is a lattice artefact. The production fits are resolved with room to spare. This is now recorded per cell by the efficiency experiment rather than spot-checked, and `tools/make_tab_efficiency.py` refuses to build the table from cells that fail it.

Implementation

`src/kernels.py` → `MixtureInnovationKernel` — see its `m_step`, whose `n_inner` is the ECM sweep cap (now 16) and `inner_tol` the $\mathcal{Q}$-based inner criterion; the returned kernel carries `inner_sweeps`, `inner_converged` and `inner_q_gain` so a starved inner loop is visible. `theta` exposes the flat parameter vector, which is what the parameter count is read from: $1+3C$ numbers with π simplex-constrained, hence $3C$ free — twelve at $C = 4$, and **twenty-four** at the $C = 8$ the frozen config actually uses. The efficiency table said twelve for longer than it should have.

8.6 Identifiability

Two claims must be separated, and confusing them overstates what is proved.

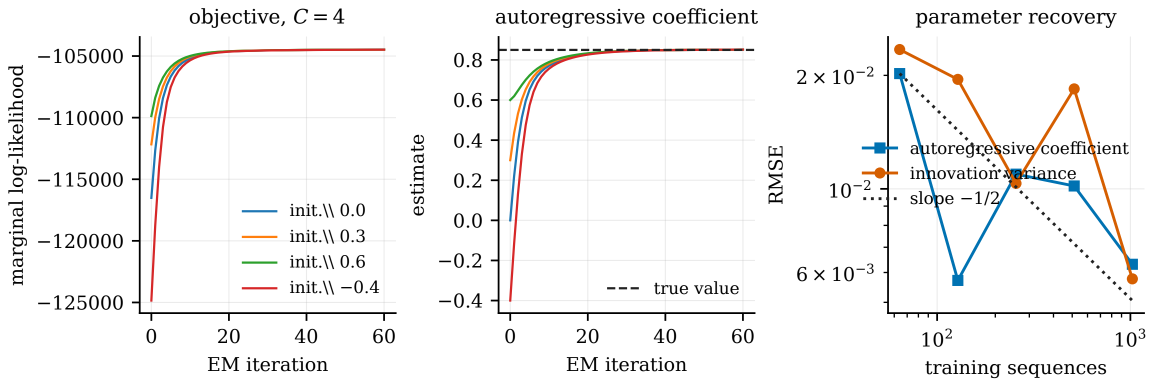


Figure 8.1: Left: the marginal log-likelihood by iteration at $C = 4$, $M = 512$, from four initialisations of ρ; the largest decrease across iterations is zero in all four runs, and in the four $C = 8$ runs not shown. Centre: the estimated autoregressive coefficient over the same runs, reaching $[0.8517, 0.8520]$ against a truth of 0.85 from initialisations spanning $[-0.4, 0.6]$ — which is the direct evidence that ρ is estimated rather than supplied. (The eight $C \in \{4, 8\}$ runs together span $[0.8517, 0.8529]$.) Right: RMSE of $\widehat{\rho}$ and of the innovation variance against the number of training sequences, with an $M^{-1/2}$ reference. Sources `outputs/exp_18/em_trace.csv`, `outputs/exp_06_em_parameter_recovery/em_rate.csv`.

Distribution level. Gaussian convolution at finite t is injective. In characteristic-function terms $\varphi_{t,\theta}(\xi) = \varphi_\theta(e^{-t}\xi)\, e^{-\Delta_t\|\xi\|^2/2}$; the Gaussian factor has no zeros so it divides out, and $e^{-t} > 0$ makes $\xi \mapsto e^{-t}\xi$ a bijection. So P_t determines P_0.

Parameter level. That P_0 is determined does *not* by itself determine K. That additionally requires: the initial law fixed or separately identifiable; the transition family identifiable as a parametric family; the mixture treated modulo label permutation; the mixture parameters in the interior — all $\pi_c > 0$ with the (ν_c, s_c) distinct, which an over-specified C need not respect; and μ of full support, since K is pinned down only there. We assume these rather than prove them.

Injective is not the same as stable. This is Gaussian deconvolution, whose inverse is unbounded, so identifiability survives at any finite t while the *information* degrades rapidly — and unevenly. Measuring that unevenness needs care about what is being compared. In the Gaussian model with parameters (ρ, q) the innovation *scale* q falls by a factor 142 between $t = 0.05$ and $t = 1.6$ while the correlation falls by 26, but both of those are second-order quantities: q is a scale, not a shape, so the ratio says nothing about higher-order structure.

A genuine shape coordinate is needed. On a generalised-Gaussian innovation $\varphi_\beta(e) \propto \exp(-(|e|/s)^\beta)$ with s chosen so the variance is exactly q, β moves the fourth moment at fixed second moment. Reporting the *efficient* information for each coordinate — projecting out the other two, since the diagonal ignores their coupling — the decay across the same range is 56–87 for the correlation, 235–522 for the variance and 8.7×10^3–6.8×10^4 for the shape. Higher-order structure is therefore not five times more fragile but 112 to 1222 times, which is exactly the structure this project cares about.

Implementation

`experiments/exp_06_em_parameter_recovery.py` → measures the information decay (`price_of_noising.csv`) and the recovery of an unknown innovation law from noisy data alone.

Sources

EM is Dempster et al. [21]; the convergence analysis, including what monotone ascent does and does not guarantee, is Wu [22]. The generalised and conditional-maximisation variants used here — which is what the mixture M-step actually is — are Meng and Rubin [23]. Fisher's identity for the gradient of the marginal likelihood goes back to Fisher [24] and is given in the latent-variable form used here by Louis [25] and Cappé et al. [17]. The transition-count statistic $\boldsymbol{\Xi}$ is the continuum analogue of the one in Baum et al. [13]. Identifiability of finite mixtures, which Assumption 1 of the note leans on, is Teicher [26] and Yakowitz and Spragins [27].

Part V

The experiment ledger

Chapter 9

Claim audit: what is settled, and what was withdrawn

The experimental record and the correction record, in one place. They were two chapters, which meant a claim's current status lived in one and the reason it changed lived in the other, and checking a number meant knowing to look twice.

Everything below comes from a single configuration, defined once in `experiments/frozen_config.py` and imported by every experiment. The previous version of this record stitched together runs at three EM iteration budgets, three replicate counts and two values of ρ; several conclusions turned out to be artefacts of those settings rather than of the model.

The ledger

One line per claim. **Settled** means it can be cited without qualification; **conditional** means it holds under a stated restriction that must travel with it; **censored** means the run hit a budget rather than a conclusion; **withdrawn** means it was claimed and is not any more.

claim	status	where it stands
Grid BP reproduces the Gaussian closed form	settled	9.2×10^{-15} at $N_{\mathrm{g}} = 401$, $A = 8$; a configuration-specific agreement, not a bound for arbitrary kernels
Truncation and quadrature are separately controlled	settled	§9.2; the boundary statistic is a quantile over chains, not a bound
Moment closure = LMMSE of the covariance-matched Gaussian	settled	proved, not observed; the gap measures non-Gaussianity
Fisher's identity gives the exact gradient	settled	$\sim 10^{-9}$ against finite differences of the discretised evidence
EM ascends the exact evidence	settled	monotonicity violation exactly 0 across every reported run
ρ and q are recovered from noised sequences	settled	§9.3
The innovation *density* is recovered	conditional	under the declared density metric and at a certified budget; the shape needs ∼120 iterations where ρ needs ∼25
Structure is worth an order of magnitude against networks	conditional	§9.4; conditional on the arms being asymmetric in supplied information *by construction*, which is the point of the measurement and does not go away. The budget objection, which did, was closed by tripling both caps at $M = 2048$: the ratio moves -0.16 ± 0.18
Against a weight-shared (not merely unstructured) baseline the margin is 2.3–6.1×	settled	`exp_31_structured_baseline.py`, sixteen seeds, all-site scored, provenance-clean; the weight-shared window head beat two architectures able to propagate further across the chain in the screening that selected it, so the confirmed baseline is the simplest one tested, not the strongest conceivable one
Local score estimation is near-optimal away from mid t	settled	the windowed estimator is the exact conditional expectation under strictly stationary initialisation; the initial-law mismatch was measured at a median ratio of 1.0038 and does not account for the effect. One exception: uniform innovations at $\rho = 0.5$, $t = 0.05$
Higher-order structure is more fragile to noising	conditional	*qualitatively* yes, on a family where β moves the fourth moment at exactly fixed variance and after projecting out (ρ, q). The *ratios* are not reportable: they run to 10^4 and depend on the parametrisation, the nuisance projection and the endpoints. Lead with the analytical statement, not the table
Single-frame marginals are blind to the ring's rotation	settled	theorem. Only one-frame marginals are blind; two-frame and larger joints are exactly where ψ is identifiable, and the claim was previously stated over "marginals" without that qualifier. The numerical control is a machine-precision zero against a density function that takes no ψ argument, so it checks consistency, not the theorem
Non-Markov violations are not interchangeable	conditional	two violations probed, not a general theory
The *shape-based* capacity effect is a convergence artefact	withdrawn	§9.10.1; most shape coordinates had not settled at the budget used, and ∼96% of the apparent effect is rate. What is withdrawn is the capacity *attribution* for anything resting on fitted or generated shape
Held-out evidence saturates near eight components	withdrawn	*reversed* by `exp_32` (sixteen paired seeds, two sizes, fits run to plateau, predeclared equivalence region). No capacity improves on $C = 1$ at either size, and the C=8 vs C=16 step resolves *against* the region at $M = 128$ with $C = 16$ worse. The six-seed, forty-iteration design that reported the

This chapter is the experimental record. It replaces an earlier version that reported results from runs at three different EM iteration budgets, three different replicate counts and two values of ρ — numbers that could not go in one table, and several of which turned out to be artefacts of those settings rather than of the model. Part 9.10 of this chapter records what those were and how they were caught.

Everything below comes from a single configuration, defined once in `experiments/frozen_config.py` and imported by every experiment.

9.1 The configuration

$\rho = 0.85$	one value everywhere; the earlier 0.8 runs are retired
$L = 32$	chain length, which is also the ambient dimension
$N_g = 401,\ A = 8$	read off the grid/domain sweep, §9.2
twelve noise levels	log-spaced on $[0.05, 3.0]$
sixteen replicates	everywhere
$M \in \{32, \ldots, 4096\}$	seven doublings
256 held-out sequences	drawn from a seed disjoint from every training seed
EM stopping	relative 10^{-9} on the log-evidence; **never** on ρ. The cap is whatever the caller passes — `exp_07` a literal 120, `exp_32` 1200 — *not* the `em_max_iters` declared here, which nothing reads (see the box below)
ring	$T = 12$, so $L = 24$

Assumption at work

A frozen configuration is only as good as its reach, and this one has now failed that test twice. Four experiments carried private replicate knobs that never read `n_seeds`, and every one of them reported `is_frozen: true` while running at three, four or eight replicates (§9.10.3). The same disease then turned up in the iteration budget: `em_max_iters` and `em_shape_tol` are declared in the config and read by *nothing*, while `exp_07` passed a literal 120 (§9.10.1). Both were invisible to the provenance stamp, which compares the config against itself. `tools/check_replicates.py` now parses the settings dictionaries for both classes of knob.

The general lesson is worth stating once: a frozen configuration is a claim about what the experiments read, and only a reader-side check can test that claim. Declaring a value is not the same as consuming it.

9.2 Settled: the numerical foundation

The discretisation is controlled on two axes, separately. Truncation — mass outside $[-A, A]$ discarded rather than transported — responds to the domain and not the resolution: the worst-chain boundary mass falls from 2.3×10^{-3} at $A = 4$ to 1.2×10^{-6} at $A = 8$, essentially independently of N_g (the 90th percentile falls $2.7 \times 10^{-4} \to 1.5 \times 10^{-8}$ over the same range; §7.3.1 explains why both are quoted). Quadrature error falls by a factor of four per halving of h, the predicted second order for the trapezoidal rule. Neither sweep alone certifies the other, which is why both are reported.

The pipeline is verified where the answer is known. At $N_g = 401$, $A = 8$ the discretised recursion reproduces the Gaussian closed form to 9.2×10^{-15} relative, and agrees with brute-force enumeration of the discretised posterior — summing over all N_g^L configurations on a short chain, sharing no code with the message passing — to 1.6×10^{-14} on posterior means and 1.8×10^{-15} on the log-evidence. Two independent routes, not one self-consistent one.

What moment closure costs. By Proposition 6.1 the gap between moment-projected and full sum-product is exactly the price of replacing the prior by its covariance-matched Gaussian. Measured on the Laplace chain, the median relative score error falls from 0.241 at $t = 0.05$ to 7.3×10^{-6} at $t = 3$ — a factor of 3.3×10^4 across the schedule, as noising Gaussianises the posterior and the innovation's shape stops mattering.

9.3 Settled: learning the kernel

Both optimisation routes reach the same place. On the Gaussian kernel, gradient ascent via Fisher's identity reaches the exact M-step's optimum to 0.021 nats at its best step size, with slightly better parameter errors (0.0040 against 0.0053 on ρ). The difference is robustness, not accuracy: EM's monotonicity violation is 0.0 exactly and it needs no step size, while gradient ascent at $\eta = 2.0$ diverges (ρ error 0.41, monotonicity violation 1641) and at $\eta = 8.0$ is unstable. That is the whole content of the comparison, and it is why every rung above uses EM.

The gradient is exact. Fisher's identity is verified against finite differences of the discretised evidence to $\sim 10^{-9}$. This check matters more than it looks: a fixed-point residual near zero is evidence of a stationary point only if the derivative defining it is right, and a sign error in exactly that derivative once survived every internal diagnostic (§9.10.6).

9.4 Settled: what the structure buys

Relative denoising error against grid BP under the true kernel, averaged over the noise schedule, with *both* arms' optimisation budgets chosen on a disjoint validation bundle (`629985`):

M	network	EM–BP (12 free)	ratio
32	0.3953 ± 0.0023	$\mathbf{0.0546 \pm 0.0027}$	7.3
64	0.3285 ± 0.0026	$\mathbf{0.0481 \pm 0.0041}$	7.9
128	0.2744 ± 0.0012	$\mathbf{0.0328 \pm 0.0018}$	8.8
256	0.2268 ± 0.0010	$\mathbf{0.0243 \pm 0.0014}$	10.1
512	0.1878 ± 0.0006	$\mathbf{0.0214 \pm 0.0011}$	9.3
1024	0.1569 ± 0.0006	$\mathbf{0.0162 \pm 0.0011}$	12.6
2048	0.1326 ± 0.0005	$\mathbf{0.0134 \pm 0.0008}$	15.7

All sixteen seeds, all seven doublings, $\pm$ one standard error *aggregated per seed and then across seeds* — the twelve noise levels within a seed share one training set and one set of fitted models, so they are not twelve independent observations. The network's (parameterisation, training length) choice agrees with the test-set optimum in 82.2% of the 1,344 cells and EM–BP's iteration count in 79.5%, so the selection is signal rather than noise. EM–BP is more accurate in 1,342 of 1,344 cells; the two exceptions are both at $M = 32$, $t = 3.0$, at ratios 0.92 and 0.96. The network at $M = 2048$ (0.133) has not reached the error the estimator attains at $M = 32$ (0.055), and no crossing is observed, so no data-equivalence factor is quoted.

Assumption at work

"Both arms select their cap" was read as "the ratio is bounded by the cap". It is not, and the difference is measurable. This box used to say that the largest ratio was the least trustworthy number in the table, on the grounds that at $M = 2048$ the network selects the 20,000-step cap in 33% of cells and EM–BP the 400-iteration cap in

61%, and to advise quoting 7.3–12.6 rather than the endpoint.
That inference does not follow. An arm selects its last checkpoint whenever validation error is still falling, at whatever rate; cap-selection therefore reports the *sign* of the remaining gain and says nothing about its *size*. The size was measured directly: array `631467` reruns $M = 2048$ with the EM cap tripled (400 → 1200) and the network's tripled (20,000 → 60,000), on the same sixteen seeds and the same validation and test bundles, so the comparison pairs seed by seed — necessary here, because the seed-to-seed spread in the ratio (8.97–36.33) is an order of magnitude larger than the effect sought.
The paired difference is -0.16 ± 0.18 ($t = -0.87$ on 15 df) and the network's error moves by −0.3%. Under the raised caps the network's cap-selection falls from 33% to 6.8% — it genuinely stops being budget-constrained — while EM–BP still selects its cap in 48% of cells, so the caps are still *selected*; they simply do not *bind*. The endpoint is quotable and the advice above is withdrawn.
The generator enforces this rather than trusting the prose: it recomputes the paired difference on every run and refuses to emit the table if it exceeds 1.0.

$M = 4096$, and why it is in the table after being excluded. `frozen_config` originally dropped 4096 from `efficiency_sizes` citing exactly the reasoning the paragraph above retracts. With that reasoning gone, the size was run: arrays `631496/631497`, sixteen seeds on H200, at a budget raised to match (2400 EM iterations, 60,000 steps). All 192 cells are resolution-certified (minimum $s_{\min}/h = 5.07$), and EM–BP is more accurate in 192 of 192 — the ratio is 17.60 ± 1.21, continuing the rise with M, at a mean transition Hellinger of 0.0457.

Because that row runs at a different budget from the rest of the table, the $M = 2048$ rerun above does double duty: it is the calibration that makes the two protocols comparable, and it is disclosed as such in the caption.

Assumption at work

A cell-level split that was really a seed-level one. At $M = 4096$ the cells where the EM stopping rule fires look worse than the censored ones (0.0110 against 0.0089 in score error), which reads as the stopping rule cutting fits short. It is not: `stop_reason` is constant within a seed — part 5 fits EM once per seed and evaluates it at all twelve noise levels — so the split is four seeds against twelve, not 48 cells against 144. At the seed level the gap is not significant (Welch $t = +0.95$ on score error, +1.49 on Hellinger, −0.64 on the ratio). This is the same error as the replicate-count episode in §9.10.3, in a new costume.

What the symmetric protocol cost, and why it was worth it. The previous version of this table (`629091`) selected EM–BP's iteration count on validation while training the network to a fixed 20,000 steps. Correcting that moved every ratio down — by 32% at $M = 32$, 36% at 128, and only 5% at 2048 — because early stopping helps the network most where data is scarcest. Measured on each arm separately, running to the configured budget instead of the selected one costs the network +42%, +52%, +52% at $M = 32, 64, 128$ against EM–BP's +39%, +32%, +14%. Pinning one arm's budget was therefore not a neutral bookkeeping choice; it was worth up to a third of the headline number, in our favour.

The ratio is not monotone and that shape should not be over-read. Both arms improve with M; the estimator starts so far ahead that the ratio is a quotient of two moving quantities, and nothing here identifies a mechanism. A side observation, recorded but not claimed: the selected network training length scales as $M^{0.84}$ ($r = 0.99$) against EM–BP's $M^{0.53}$ ($r = 1.00$), so the network needs disproportionately more optimisation as data grows — but both slopes are lower bounds, since both arms hit their caps at $M = 2048$.

Assumption at work

These sixteen seeds ran at 120 EM iterations, because `exp_07` hard-coded that budget while `FROZEN.em_max_iters` said 400 and nothing read it (§9.10.1). The rerun at the configured budget is Bocconi array `628943`.

The obvious prediction about that rerun was wrong, and the way it was wrong is the interesting part. The reasoning was: the shape settles at a median of 229 updates, so a run stopped at 120 is short of convergence, so the estimator is handicapped and the ratios are a lower bound. On the first six seeds the direction *reverses with sample size*. Held-out score error of the estimator, same six seeds, 120 against 400 iterations:

M	EM at 120	EM at 400	
32	0.0585	0.0673	worse by 15%
64	0.0501	0.0626	worse by 25%
256	0.0283	0.0295	worse by 4%
512	0.0223	0.0200	better by 10%
2048	0.0158	0.0125	better by 21%

Running the optimiser *longer* makes the estimator *worse* at small M and better at large M, so no fixed budget is neutral between the two arms of the comparison.

The mechanism, measured rather than inferred. An instrumented run (`experiments/exp_29_em_overfitting.py`) records the training marginal log-evidence and the held-out score error on the *same* iterates, which is what separates overfitting from a bad optimum: overfitting means the training objective keeps improving while generalisation turns around, whereas a bad optimum would show both stalling.

M	training $\mathcal{L}$ monotone ↑	held-out best at	cost of running to 400
32	yes	iteration 130	+2.8%
64	yes	iteration 40	+46.0%
2048	yes	iteration 250	+8.6%

The training log-evidence increases monotonically to the last iterate at every size — as EM guarantees it must — while the held-out error has an *interior minimum* at every size. That is overfitting, and the diagnosis is now a measurement rather than a plausible story. Note what the third row says: the interior optimum is at 250 even at $M = 2048$. The effect is **not** confined to small samples, and the earlier reading — "120 wins below the crossover, 400 wins above" — was an artefact of comparing two arbitrary points either side of an optimum that moves. A cap of 400 overshoots at every size tested.

Two consequences. First, "run it to convergence" is not automatically the honest choice. The iteration count is a hyperparameter with a bias–variance trade-off like any other, and fixing it a priori quietly favours one arm; picking it on the test set would be exactly the sin §9.10.4 records. It is now chosen on the validation bundle, per noise level, symmetrically with the network's parameterisation, which is what `629091` runs.

Second — and this is why it is in this chapter rather than a footnote — this is the memorisation–generalisation question of §2.3 appearing in a model with twelve free parameters and no network anywhere. The training objective and the generalisation objective come apart, early stopping is doing implicit regularisation, and all of it is visible in a setting where the target score is computable in closed form. That is precisely what this setting was built for, and it is the most promising thing found in this batch.

Still not claimed. That the optimal stopping point scales with M in any particular way. The three sizes here give 130, 40 and 250, which is not monotone, and three points chosen for a diagnostic are not a curve. Reading a scaling law off them would repeat §9.10.4 exactly.

9.5 Settled: structure the marginals cannot see

Chapter 10 in full. The result in one line: across 1,344 cells the per-frame arm's estimate of $p(\psi)$ moves 0.0000 in total variation from its uniform initialisation, and across the 480 cells of the single-rotation variant its profile likelihood in ψ has range 0.00×10^0 nats — a machine-precision zero, which is what Theorem 10.1 requires. The joint arm recovers the rotation to 0.15° at $t = 0.05$, degrading to 7.09° at $t = 1.43$ and returning to the null by $t = 3$.

9.6 Settled: where the Markov assumption fails

Two contamination mechanisms, both with an exact reference. `ratio_to_em` is the baseline's relative score error over EM–BP's, so > 1 means the estimator wins.

Table 9.1: **Provenance: not certified.** The run behind this table was produced with `src/em.py` uncommitted, so its recorded commit does not reconstruct the program that produced these numbers. A clean-deployment rerun of the same protocol is what should stand here. Two controlled violations of the Markov assumption, each against an exact reference for the contaminated model. The first two rows of each block are the baseline's relative score error divided by EM–BP's, averaged over the noise schedule, so a value above 1 means the structured estimator wins and a bold value means it loses. The third row is the fitted correlation minus the Chow–Liu correlation of the true contaminated law: the distance between the chain that was fitted and the best chain there is. The fourth is EM–BP's own relative score error at its worst noise level, which says how much of a change in the ratio is the estimator degrading rather than the baseline improving.

	global latent, rank-one, strength β				
	0.00	0.10	0.25	0.50	1.00
Gaussian, vs CNN	16.41	14.84	6.68	3.21	2.16
Gaussian, vs MLP	29.34	22.48	11.28	4.00	1.67
$\widehat{\rho} - \rho_{\mathrm{CL}}$	−0.0025	−0.0008	+0.0008	+0.0051	+0.0144
EM–BP error, worst t	0.019	0.025	0.055	0.134	0.167
	long-range precision coupling, strength γ				
	0.00	0.05	0.10	0.20	0.40
Gaussian, vs CNN	16.07	1.24	**0.97**	**0.88**	**0.75**
Gaussian, vs MLP	27.84	1.66	1.19	**0.84**	**0.70**
$\widehat{\rho} - \rho_{\mathrm{CL}}$	+0.0018	+0.0376	+0.0557	+0.0806	+0.0976
EM–BP error, worst t	0.016	0.298	0.402	0.490	0.579

Assumption at work

The table above replaces a hand-typed one, sourced from run `627165`, whose numbers could not be reproduced from any committed output under any aggregation — and whose baseline had trained for 8,000 steps against the frozen protocol's 20,000, so every ratio in it was inflated by an undertrained comparator. It is now generated by `tools/make_tab_nonmarkov.py`

from the frozen-budget rerun (array `633406`, `em_iters=400`, `net_steps=20000`).
The correction moves the numbers in the direction the defect predicts: at $\beta = 0$ the Gaussian ratio against the CNN falls from 18.3 to 16.41, and against the MLP from 37.0 to 27.84. A comparison against an undertrained baseline is not a conservative error, and the earlier version of this section stated a scope conclusion on top of one.

The estimator survives rank-one contamination essentially intact — still winning at $\beta = 1$, where half the marginal variance is a shared constant — and **inverts at** $\gamma \approx 0.10$ under long-range coupling, where it now loses to both baselines rather than to the CNN alone. The mechanism is exactly what the structure predicts: a global latent makes the score residual rank one, which a chain absorbs; long-range coupling is not low-rank and cannot be represented. This is the honest scope statement, and it is backed by a frozen-budget run.

Assumption at work

These runs predate `frozen_config.py` and are *not* at the frozen settings. They are reported here, in the development document, and are deliberately not quoted in the note or the workshop paper. The qualitative conclusion — rank-one survivable, long-range not — is robust to the settings; the specific ratios are not to be pooled with §9.4.

9.7 In progress

Four of the five reruns listed in the previous version have landed and moved into the sections above; what follows is what they said and what is still out.

Landed: the parametric rate (`628576`). At four replicates the fitted log–log slopes were -0.196 for ρ and -0.700 for q against a predicted -0.500, with a non-monotone ρ curve — not a confirmation of $M^{-1/2}$, and the claim was pulled from the note rather than caveated. At sixteen the variance slope is -0.519 ($r = -0.99$, monotone), which is the rate; ρ is monotone but shallower at -0.273. Half the claim is restored, half is not, and the note says so rather than averaging them into a verdict.

Landed: clean against channel (`628600`). Two slownesses, and they are independent. *Optimisation*: coordinates of one kernel settle at very different rates — median 80 updates for ρ, 68 for q, 229 for the excess kurtosis (`628601`, below). *Statistics*: fitting through the channel costs rate, not merely a constant.

log–log slope, sixteen replicates	ρ	q
clean transition pairs	-0.52	-0.50
through the channel	-0.44	-0.36

The clean arm sits on the parametric rate on both coordinates, which is the check that the estimator is not itself the bottleneck. It is also flatly incompatible with the withdrawn discretisation-floor reading of §9.10.4: the clean arm falls by a factor of 3.4 on ρ and 4.3 on q across the range, so there is no floor to see.

Landed: shape convergence at sixteen seeds (`628601`). The settling table above. Zero monotonicity violations across all sixteen traces. The kurtosis figure has a long tail — up to 638 updates in the slowest seed — which is what makes a 400 cap a decision rather than a formality.

Landed: sample efficiency, both budgets selected (`629985`, sixteen seeds). §9.4. Supersedes `628571` (fixed budgets both sides) and `629091` (EM-BP's budget selected, the network's pinned). This is the version the paper carries, with the $M = 2048$ caveat recorded there.

Landed, partly: the confining potential (Rung 4a, `629193`). Three attempts, and the third has now landed in full, so this is worth setting out in order.

Result, complete (`629962`: 480 cells, eight seeds × five sizes × six noise levels, balanced). The joint arm recovers the well width better than the marginal arm — median relative error 0.277 against 0.404 — and its accuracy improves monotonically with the number of trajectories:

M	128	512	1024	2048	4096
median $\lvert\hat{\lambda} - \lambda\rvert/\lambda$	0.656	0.367	0.260	0.192	0.161

The convergence fix is what made this readable. Under the previous absolute tolerance the marginal arm hit its iteration cap in 87.8% of cells against the joint arm's 56.7%, so the two arms were being compared at different amounts of optimisation; only 13 of 400 pairs had both converged. With the per-sequence tolerance and a raised cap the rates are 44.2% and 21.2%, and the comparison is between estimators rather than between budgets.

Where the advantage lives, and where it reverses. Resolved by noise level, the ordering is not schedule-wide and the crossover is sharp:

t	0.050	0.105	0.222	0.467	0.982	2.068
joint	**0.086**	**0.137**	**0.127**	**0.359**	0.860	0.936
marginal	0.141	0.170	0.447	0.410	**0.754**	**0.838**

Assumption at work

Two caveats travel with the headline, and the schedule-wide median hides both.
The joint arm wins at $t \leq 0.467$ and *loses* at $t \geq 0.982$. Past the crossover the channel has destroyed most of the rotation information, both arms are near the null, and the joint likelihood's extra freedom costs more than it buys. Quoting 0.277 against 0.404 without the breakdown would imply a uniform advantage that the data does not show.
And the errors are large in absolute terms. At the largest sample size the joint arm is still 16% off the true well width. What this rung supports is the *ordering* at low noise and the *scaling* in M — not a precision claim about λ.

`628434_5` was killed by the six-hour wall having written nothing — 960 gradient fits at $\sim$0.2 s per step is roughly ten hours, and the experiment writes once at the end. Sharding by seed fixed that. `628942` then completed and produced nothing usable, for a reason that had nothing to do with the queue: `part_potential` generated two species at $\pm\omega$ while `fit_potential` scores every trajectory at the single scalar ψ it is handed, so half the data was evaluated under the wrong rotation. The estimator's escape was to collapse the ring. At $r_* \to 0$ the ring is rotation-invariant and so pays nothing for a wrong ψ; r_* pinned to the lower clip in 51.2% of cells, and the sweep reported the *marginal* arm beating the joint arm, which is the reverse of what this rung exists to show.

`629193` (one species, 600 steps, sixteen seeds, 960 cells) repairs that:

	λ rel. err. (med.)	r_* abs. err. (med.)	pinned at clip
joint, two species (`628942`)	1.844	0.950	51.2%
joint, one species (`629193`)	0.229	0.020	4.8%
marginal, one species	0.429	0.039	4.4%

The joint arm's error falls monotonically with sample size — 0.618, 0.336, 0.222, 0.181, 0.118 at $n = 128$ to 4096 — and that trend is reported here because it survives the caveat below.

Assumption at work

The arm *comparison* from `629193` is not reported, because the two arms were not held to the same convergence standard. At $t \leq 1$ the marginal arm hit the 600-step cap in 95% of cells against the joint arm's 57%, so "joint beats marginal" partly means "joint converged and marginal did not". The obvious control does not rescue it: only 13 of 400 pairs have both arms converged.

The cause was the stopping rule. `plateau_tol` was an *absolute* tolerance on an objective that sums over sequences, hence eight times stricter at n=4096 than at n=512: a convergence criterion varying along the very axis this rung reports its scaling on. It is now per-sequence, pinned by `tests/test_ring.py::test_stopping_rule_does_not_depend_on_sample_size`, which replicates one dataset fourfold and asserts the fit stops at the same step.

But more optimisation is not more accuracy, and that is the deeper problem. Twelve cells were refit to 2400 steps with the plateau test disabled, to see what the budget was costing. The first cell inspected showed exactly the expected picture — at n=512, t=0.2216, the joint arm is identical to eight significant figures from step 600 onward while the marginal arm's λ error falls from 0.482 to 0.372 to 0.335 — and it would have been easy to stop there and report that the joint arm was converged and the marginal arm cut short. Across all twelve cells that is not what happens:

cell	λ rel. err. at 600/1200/2400			log-lik. monotone
n=512, t=0.2216, joint	0.078	0.078	0.078	yes
n=512, t=0.4665, joint	0.029	0.057	0.060	yes
n=512, t=0.2216, marginal	0.177	0.066	0.207	yes
n=2048, t=0.2216, marginal	0.265	0.075	0.149	yes

The objective rises monotonically in all twelve cells — backtracking guarantees it — while the parameter error is non-monotone in four, and in two joint-arm cells the estimate at 2400 steps is *worse* than at 600. Ascending the likelihood harder does not monotonically improve recovery, because the finite-sample maximiser is not at the truth. This is the same phenomenon as the EM overfitting of §9.4, in a different estimator: the optimisation budget is a regularisation knob, not a convergence detail.

So `629962` should be read for what it is. Holding both arms to one n-independent criterion removes a defect that was certainly there; it does not deliver two converged estimators, and the honest fix — selecting each arm's stopping point on held-out data, as §9.4 now does for EM-BP and the network — is not implemented for this rung. Any joint-versus-marginal margin from `629962` carries that caveat.

Landed: the structure-aware baseline (`experiments/exp_31_structured_baseline.py`, jobs `641509`–`641514`, sixteen seeds, all three lanes complete, provenance-clean at commit `569a67c`). The measurement an outside review identified as the outstanding one: what survives of the headline ratio against a baseline that can represent chain inference, rather than the fixed window that closes most of it. Screened three architectures on development seeds — the weight-shared window head, a dilated convolutional stack, and a bidirectional message-passing network, the last two able in principle to propagate across the whole chain — and **the window head won**, at radius 4, width 64. Confirmed at the headline protocol on the full sixteen frozen seeds, all-site scored: EM–BP is more accurate than the window head in every one of 192 scored cells — 16 seeds $\times$ 4 sizes $\times$ 3 regions, not the 128 a two-column table suggests — by a factor rising from 2.34$\times$ at $M = 32$ to 6.14$\times$ at $M = 2048$. *The radius was wrong in three documents until 1 September.* It was typed as 2, read off the all-site aggregation rather than the bulk one the selection rule actually uses, and stood in the thesis, the paper's appendix and this chapter until `tools/make_tab_screening.py` recomputed it from `screening.csv`. It is now a macro. The increase with M is recorded as a description of four measurements and no longer offered as evidence that the residual is non-architectural: an approximation floor produces the same signature, and the raw risks — EM–BP falling 4.5$\times$ across the range against the window head's 1.6$\times$ — are consistent with one. This replaces the withdrawn `exp_12_scaled` comparison in every document and closes the review's largest single open item. The table generator is `tools/make_tab_structured.py`, rewritten for this run's schema and refusing to run if the screened architecture ever changes or provenance ever fails, rather than silently reporting a stale assumption.

Still out.

- **Rung 4a at a matched convergence budget** (`629962`), eight seeds split by sample size into sixteen shards, at a 3000-step cap under the per-sequence tolerance. Until it lands, the joint-versus-marginal margin above stays out of every document.

Landed: capacity equivalence, and it reverses the claim it was built to test. (`experiments/exp_32_capacity_equivalence.py`.) Predeclared equivalence region, `em_cap` raised from 400 to 1200. The raise is itself a measurement rather than a convention: at the old cap every one of the 320 fits was still censored, and a representative $C = 16$ fit was moving thirty times faster than $C = 8$ over the last hundred iterations — the same capacity-versus-convergence-rate confound, one level up. Run to 2000 iterations, the per-edge gain falls to $\sim 6 \times 10^{-9}$ by 800 and then plateaus, so 1200 captures essentially all the real movement without pretending the strict 10^{-9} tolerance is reachable. Jobs `641515`–`641520` plus the resume shard `643778`; lane b had reached 39/50 when two consecutive six-hour shards ran out of wall time, and `part_sweep`'s per-cell flush and on-disk resume check made a third shard safe against the same output directory rather than destructive to the first thirty-nine cells. Complete at 160/160: sixteen seeds, five capacities, two sizes.

The result is not the one the design was hoping to be able to state. Paired against $C = 1$ on identical data, *no* capacity improves held-out evidence or schedule-level score risk at either size — $C = 1$ wins on 13/16 seeds at $M = 128$ and 14/16 at $M = 512$ — and the predeclared region refuses the C=8 versus C=16 comparison as well, resolving against it at $M = 128$ with $C = 16$ measurably worse. The six-seed, forty-iteration table it supersedes reported the opposite sign and was read as capacity beginning to pay at $C = 4$; that does not reproduce. The likely mechanism, offered but not tested, is that the short budget was regularising the larger mixtures. Two confounds remain and both disfavour high capacity: every cell at $C \geq 4$ stopped at the cap rather than at tolerance, and the share of fits whose narrowest component falls under the grid's resolution floor climbs from 0/32 at $C = 1$ to 11/32 at $C = 16$. Separating them needs a finer grid at fixed capacity and a budget ladder at fixed capacity, neither run.

One artefact from this rerun must not be used. A contrast job (`639099`) fired early because its `afterany` dependency was satisfied by the *cancellation* of a restructured upstream shard — `afterany` fires on any terminal state and cannot distinguish "the work finished" from "something happened to that job id". It computed the contrast on seeds 0–6 alone, roughly 40% of the design. The file is renamed `DO_NOT_CITE_premature_contrasts_job639099.csv` on the cluster so that a merge script globbing the ordinary name finds nothing; it is kept rather than deleted because its visibly over-wide $M = 512$ interval is the demonstration of why the full seed count matters.

9.8 Withdrawn, and not replaced

Recorded in full in Chapter 9; listed here so that nothing in this chapter is read as still standing.

- **The capacity attribution.** That enlarging the innovation mixture buys generative fidelity. Roughly 96% of the apparent effect was convergence rate at a fixed iteration budget. The rerun that would settle it lost all but one of its array tasks, so one cell survives and the claim stands withdrawn and unreplaced (§9.10.1).
- **The discretisation floor.** That the clean-arm variance error was floored by the grid. It was under-replicated, not floored, and the grid contributes a constant 4.4×10^{-4} — an order of magnitude too small to produce the effect (§9.10.4).
- **Everything downstream of the superseded generation protocol**, including the four-family comparison at matched covariance (§9.10.5).

9.9 Not started

The architectural question. Sum-product on a chain suggests a network that is narrow but as deep as twice the chain, unlike the tree-structured case where the natural depth is the tree's [18, 28]. Which architectures exploit Markovianity, and at what depth, is what this setting was built to measure, and is the obvious continuation. The receptive-field sweep that would begin it exists (`exp_12`) but selected its radius on the evaluation set, so its numbers are exploratory and are not reported anywhere.

9.10 Corrections: what was wrong, and how it was caught

Each entry below is a claim that was made and then withdrawn or narrowed. They are kept because the mechanism that produced each one is still live in the code, and because the pattern across them is itself a finding.

This chapter exists so that the note and the workshop paper do not have to carry it. A reader of those documents should meet claims, not retractions of claims; a reader of this one should be able to see what was believed, why it was wrong, and — the part that matters for the next mistake — what diagnostic caught it.

Every entry follows the same shape: the claim as it was made, the correction, and the check that now exists so it cannot recur silently. Several of these were live in circulated drafts.

9.10.1 The iteration budget that manufactured a capacity effect

The claim. At $C = 4$ the estimator was an order of magnitude better pointwise than the convolutional baseline and yet generated a worse residual law; varying C improved both axes monotonically, which identified the deficit as a **capacity** limit of the innovation mixture.

What was wrong. Every run behind that account used `em_iters` $= 40$. EM on this kernel converges on ρ far faster than on the innovation *shape*: at $\rho = 0.85$, $M = 512$, $C = 8$ the fitted excess kurtosis reads 0.84 at 30 iterations, 1.85 at 60, 2.30 at 120 and 2.29 at 400, while

ρ is flat from iteration $\sim$25. Monitoring ρ — which is the natural thing to plot — gives a converged-looking trace over a kernel that is nowhere near converged. Enlarging the mixture at a fixed iteration budget buys convergence *rate*, not representational capacity, and roughly 96% of the apparent capacity benefit was that.

What replaced it. Nothing yet, and this is worth stating plainly. The rerun that would settle it was submitted as Bocconi array `627164`, and **only array task 2 ever ran** — `sacct` lists `627164_2` and nothing else. One cell survives, $C = 2$ at one seed, in which the estimator is consistent with the reference (1.837 ± 0.061 against 1.894 ± 0.096, a gap of 1.20σ). That is suggestive that the capacity reading was indeed a convergence artefact — a two-component mixture would not have sufficed under the old account — but one cell at one capacity cannot replace a withdrawn claim. **The capacity attribution remains withdrawn and unreplaced.**

The check that was supposed to exist, and did not. This section previously claimed that `experiments/frozen_config.py` defines a stopping rule on the *shape* — EM stopping when $|\Delta\kappa_4| < 10^{-3}$, with a cap of 400 iterations. **That was false when written and stayed false for a month.** The config declares `em_shape_tol` and `em_max_iters`, but a search of the tree finds *no reader of either*: the rule actually implemented in `src/em.py` → `fit_em` is a relative tolerance of 10^{-9} on the log-evidence, and the cap is whatever the caller passes. `exp_07` — the headline experiment — passed a literal 120, in four places.

This is the replicate defect of §9.10.3 in another costume, and it is worse in one respect: there the compendium recorded the gap once it was found, whereas here the compendium asserted a safeguard that did not exist. A document describing its own checks is evidence about those checks only if the description is itself checked.

Two mitigating facts, neither of which excuses it. The tolerance that *is* implemented is far stricter than the one advertised: replayed against the 800-update traces of `exp_27`, it fires in one of sixteen seeds, so in practice runs go to the cap rather than stopping early on a false plateau. And the direction of the exp_07 error is conservative — an under-iterated EM makes the estimator look worse, so the efficiency ratio was understated, not inflated.

The check that now exists, and what it immediately found. `exp_07` reads `FROZEN.em_max_iters`, so the budget is the configured one. `tools/check_replicates.py` is extended from replicate counts to iteration budgets, and — this is the part that matters — from settings dictionaries to *call sites*, since `n_iters=120` is a keyword argument and the dict-only version could never have seen it.

Turned on, it reports sixteen fixed budgets across three paper experiments. Sorting them by whether the run reports a coordinate the budget can actually reach:

experiment	sites	budget	reports
`exp_06`	6	80–120	ρ, q only — settle at 80, 68: adequate
`exp_06`	6	120	innovation *moments* — kurtosis settles at 229
`exp_18`	1	60	innovation moments
`exp_18`	1	60	ρ only: adequate

So the defect is not confined to `exp_07`. Eight of the sixteen budgets are defensible because the coordinates they report settle well inside them; **eight are not**, and every number downstream of those is a kernel stopped roughly half-way to its shape. None of them is quoted in the note or the workshop paper — the shape-dependent claims there come from `exp_07`, which is being rerun — but the fitted-density material in this document does depend on them, and is to be read as provisional until they are rerun at the frozen budget.

Assumption at work

An iteration budget is not a global property of a run. It is adequate or not *relative to the slowest coordinate that run reports*, and the same 120 iterations are ample for ρ and insufficient for a fourth cumulant. This is why `exp_27` exists and why its settling table is a prerequisite for reading any other experiment here, rather than a curiosity about convergence.

`experiments/exp_27_shape_convergence.py` records the whole diagnostic set at every evaluated state, which is what the original experiment never did.

9.10.2 Pointwise accuracy against generative fidelity

The claim. That pointwise denoising accuracy and generative fidelity *dissociate*: that an estimator can be an order of magnitude better at predicting $\mathbb{E}[a \mid x]$ and simultaneously worse at generating samples with the right innovation shape. This was given as the central finding of the results chapter in circulated drafts of both the thesis and the note.

What was wrong. The same 40-iteration budget as §9.10.1, applied to the one statistic that needed more. The fitted innovation *shape* settles at a median of 229 updates and as late as 638, while ρ settles at a median of 80 (the settling table above), and the generative metric is a shape statistic. So the comparison scored converged estimators against unconverged ones on precisely the coordinate that had not converged, and the ranking it produced is not attributable to the estimators.

This is a confound rather than a refutation — the dissociation may well be real — but the experiment as run cannot distinguish it from the iteration budget, and that is enough to withdraw the claim. It is withdrawn and **unreplaced**: no corrected version has been run at a budget that would settle the question.

The numbers, for the record. Generated-sample residual excess kurtosis against the closed-form target of 1.9098 derived in §9.10.5, mean over four seeds per cell, under the corrected forward-noising protocol (the two network-free arms are bit-identical to the earlier protocol, and both trained networks improve under it).

M	EM/BP	CNN	MLP
32	0.491	0.728	14.803 (2/4 seeds unstable)
128	0.814	1.293	1.441
512	0.897	**1.601**	0.167
2048	0.902	**1.572**	**0.046**

The within-arm version was the sharpest form of the claim, and is worth recording because it needs no comparison between estimators: at $M = 2048$ the global MLP's four per-seed generated-kurtosis values were 0.097, 0.069, 0.019 and -0.002 — all indistinguishable from a Gaussian generated law — while its own pointwise error improved monotonically over the same range. One estimator disagreeing with itself across two notions of correctness as its training data grows. The 40-iteration confound applies to the arms it is compared against, and the proposed mechanism (a better L_2 fit to the posterior mean is smoother, and a smoother denoiser drives the reverse SDE toward a more Gaussian generated law) was never measured directly.

The check that would have caught it. Any convergence diagnostic on the reported statistic itself. Monitoring ρ — the natural thing to plot, and what was plotted — certifies a run whose shape is still moving, because the two coordinates converge at rates differing by a factor of eighty. The lesson generalises past this claim: a convergence check must be run on the quantity being reported, not on the parameter that is easiest to watch.

9.10.3 The replicate counts the frozen configuration could not reach

The claim. That a frozen configuration file, imported by every experiment, was sufficient to guarantee that every number in the paper came from one protocol.

What was wrong. `frozen_config.py` exposes `n_seeds`. Four experiments carried their own replicate knobs and read none of it:

experiment	knob	was	protocol
`exp_06`	`n_rep`	10	16
`exp_06`	`n_rep_rate`	4	16
`exp_06`	`n_rep_clean`	3	16
`exp_27`	`n_seeds`	8	16
`exp_07`	`seed`	one per run, four run	16

All five runs reported `is_frozen: true`, because the provenance check compares the config against itself and cannot see a knob the experiment never asked about. *A configuration that cannot reach the parameter that matters is not freezing anything.*

The consequences were real and different in each case. The rate experiment at four replicates gives fitted log–log slopes of -0.196 for ρ and -0.700 for q against a predicted -0.500, with a non-monotone ρ curve — that is not a confirmation of the parametric rate, and the claim was pulled from the note rather than reported with a caveat. The clean-versus-noised comparison at three replicates is the same count that produced the withdrawn flatness reading of §9.10.4.

The check that now exists. `tools/check_replicates.py` parses each experiment's settings dictionaries and fails on any literal replicate count in a paper experiment, allowing deviations only with an explicit `# frozen-exempt: <reason>` comment. It is worth recording that this check was *first written as a shell grep*, and the grep missed `exp_27` entirely — a line-based search cannot tell a `quick` smoke dictionary from a `full` one. The AST version found it immediately. A check that is approximately right about where to look is not a check.

9.10.4 The flat curve that was under-replicated, not floored

The claim. On clean chains the innovation-variance error was flat in M $(0.0041, 0.0039, 0.0059, 0.0046, 0.0040$ from $M = 64$ to $1024)$, and the flatness was a discretisation floor imposed by the grid.

What was wrong. Both halves. At three replicates an RMSE is estimated from three numbers and its own sampling error is comparable to the effect being read off it; at sixteen the curve is not flat, running $0.0136, 0.0073, 0.0056, 0.0042, 0.0032$, close to the $M^{-1/2}$ reference. The $M = 64$ entry alone moves by a factor of three between the two replication levels.

Nor was the grid ever a plausible culprit. Removing it entirely — exact clean-data OLS on the raw transition pairs, no binning — gives $0.0135, 0.0071, 0.0057, 0.0043, 0.0032$, statistically indistinguishable from the binned fit, with a paired grid-minus-raw discrepancy of a constant 4.4×10^{-4} across all budgets. A quantity that small cannot produce a floor at 4×10^{-3}.

The lesson, which generalises. The failure mode was reading a *mechanism* into a pattern that was noise, and then reaching for the most technically interesting available explanation. The discretisation floor was a satisfying story; it was also never tested against the alternative that three points are not enough to see a trend.

9.10.5 Two wrong ways to compare a generated law

The claim. Successive versions compared the generated sample against clean data, and then applied a denoising read-out to it.

What was wrong. Both, in opposite directions. Reverse integration stops at $t_{\min} > 0$, so what each estimator produces is a sample of $P_{t_{\min}}$, not of P_0; comparing it against clean data measures the stopping point rather than the score. Concretely, independent Gaussian noise of variance v on an innovation of variance q and excess kurtosis κ gives excess kurtosis $\kappa(q/(q+v))^2$, since fourth cumulants add and the variance grows. The second version applied a denoising read-out instead, which inverted the bias through shrinkage.

What replaced it. Draw the reference from P_0 and noise it *forward* to the same $t_{\min}$, so both sides are samples of the same law and the target follows in closed form. At $\rho = 0.85$, $t_{\min} = 0.02$ the residual innovation noise is $v = \Delta_t(1 + \rho^2)$ and the predicted target is 1.9098 against a clean 3.0. The closed-form target is what identified both errors: neither was visible as an anomaly, and both produced plausible numbers.

9.10.6 A sign error in a gradient that a fixed point could not see

What was wrong. `src/kernels.py` → `MixtureInnovationKernel.grad_log_transition_matrix` carried a sign error in the ρ derivative.

Why it mattered more than it looks. A fixed-point residual near zero is evidence of a stationary point only if the derivative defining it is right. Every convergence diagnostic downstream of that function inherited the error, so the runs looked converged for a reason unrelated to whether they were.

The check that now exists. Fisher's identity is verified against finite differences of the discretised evidence to $\sim 10^{-9}$ — a check that compares two independent routes to the same quantity, rather than one that asks a quantity whether it is happy with itself.

9.10.7 The ring normaliser, twice

Recorded in full in §10.4 and its surrounding boxes; summarised here because it is the most recent and the most instructive.

The ring density is written unnormalised, and its normaliser depends on the parameters of the confining well. Omitting it is harmless while only ψ is being estimated — the term is constant and cancels in every responsibility — and silently fatal the moment λ is estimated, because a wider well contains more mass and the likelihood grows without bound.

Two things are worth extracting. First, the bug was *introduced by a correct-looking port*: the ported code agreed with its audited original bit for bit, because the original only ever compared ψ at fixed λ and never needed the term. Agreement with a reference only certifies the cases the reference was used for. Second, fixing it in `src/ring.py` → `log_pt_conditional` left it unfixed in `src/ring.py` → `log_pt_marginal`, where the count differs — once per frame rather than once — and that second instance was caught not by a test but by an experiment diverging to $\hat{\lambda} = 3.5 \times 10^6$. The test suite now pins both functions and both directions.

9.10.8 What these have in common

Five of the six were failures of *measurement*, not of derivation. No theorem was wrong. In each case a quantity was computed correctly and then read as evidence for something it could not support: an iteration budget read as a capacity limit, three replicates read as a floor, a stopping point read as a score error, a fixed point read as convergence, a constant read as absent.

Two habits follow, and they are the reason this project's checks look the way they do.

Prefer two independent routes to one self-consistent one. The gradient sign error survived every internal diagnostic and died immediately against finite differences. The generation comparison survived two revisions and died against a closed-form target. Grid BP is checked against a closed form *and* against brute-force enumeration sharing no code with it.

Make the failing direction fail. It is not enough for a test to pass when the code is right; where a bug's signature is known, assert it. `tests/test_ring.py` asserts that the profile likelihood peaks at the truth *and* that it becomes monotone when the normaliser is removed, so deleting the term cannot make the suite greener. Most of the entries above would have been caught a good deal sooner by a test of that shape.

Part VI

Case studies alongside the main line

Chapter 10

The rotating ring: structure the marginals cannot see

Every model so far has been the AR(1) chain, where what is being estimated — the autoregressive coefficient, the innovation density — shows up in the data's low-order statistics. Look at a histogram of a_i and of $a_i a_{i-1}$ and you have most of it. This chapter develops a model where that is false, and where it is false for a reason one can prove rather than observe.

The model is Problem 1 of the three panels on Mézard's board, developed in `research/board-3problems`. What follows is self-contained; the port into this project's package is `src/ring.py`, and §10.7 records how it was checked against the independently audited original.

10.1 The model

Two clocks, and keeping them apart is the whole setup. *Internal* time $u = 0, \dots, T-1$ runs along a trajectory. *Diffusion* time t is the noise level. One sample is a **whole trajectory**, so the ambient dimension is $L = 2T$ and the channel hits every frame at once.

Three independent latent variables:

$$r_0 \sim p_\lambda(\rho) \propto \rho\, e^{-(\rho - r_*)^2/2\lambda}, \qquad \theta_0 \sim \mathrm{Unif}[0, 2\pi), \qquad \psi \sim p(\psi), \tag{10.1}$$

and then a rotating random walk in the plane,

$$z_0 = r_0(\cos\theta_0, \sin\theta_0), \qquad z_{u+1} = R_\psi z_u + \sigma\, \eta_u, \qquad \eta_u \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_2) \text{ i.i.d.} \tag{10.2}$$

The factor ρ in p_λ is the two-dimensional area element, not part of the physics: the density of the *point* $z_0 \in \mathbb{R}^2$ is $\propto e^{-(\|z\| - r_*)^2/2\lambda}$, and converting to a radial marginal picks up the Jacobian. This is why the mode of p_λ sits slightly outside r_*.

(10.2) is the discrete form of the pair written on the board, $\mathrm{d}r = (r_* - r)\,\mathrm{d}u + \Gamma\,\mathrm{d}B_u$ together with $\dot\theta = \omega$ — that is, a **quadratic confining potential** $V(r) = (r - r_*)^2/2\lambda$ holding the radius near r_*, plus a steady rotation. So the model has two kinds of parameter, and the point of this chapter is that they behave completely differently: λ and r_* describe the well, and ψ describes the dynamics.

10.2 The rotation is a gauge

Let $U_\psi = \mathrm{blockdiag}(R_{-u\psi})_{u=0}^{T-1}$, which rotates frame u backwards by $u\psi$. It is orthogonal, so it commutes with the isotropic channel: if $\mathbf{x} = e^{-t}\mathbf{a} + \sqrt{\Delta_t}\,\xi$ then $U_\psi \mathbf{x} = e^{-t} U_\psi \mathbf{a} + \sqrt{\Delta_t}\, U_\psi \xi$ and $U_\psi \xi \overset{d}{=} \xi$. Applying it to (10.2) turns the rotating walk into a driftless one, so

$$P_t(\mathbf{x} \mid \psi) = P_t^0(U_\psi \mathbf{x}), \tag{10.3}$$

with P_t^0 the $\psi = 0$ density. At known ψ the rotation costs nothing statistically — it is a change of coordinates.

This is why the model is tractable at any T despite being genuinely non-Gaussian. One does not need a separate calculation per rotation angle; one gauges the data and evaluates a single density.

10.3 The $\psi = 0$ density in closed form

Conditionally on z_0 the trajectory is Gaussian with a z_0-independent covariance, so P_0 is a two-parameter *location* mixture of one Gaussian. Write $X \in \mathbb{R}^{T\times 2}$ for the gauged trajectory, $K_{uv} = \min(u, v)$,

$$A_t = e^{-2t}\sigma^2 K + \Delta_t \mathbf{I}_T, \qquad g = A_t^{-1}\mathbf{1}, \qquad \kappa = \mathbf{1}^\top g, \qquad \beta = \|X^\top g\|.$$

Then

$$\log P_t^0(X) = -\tfrac{1}{2}\operatorname{tr}\Big(X^\top A_t^{-1} X\Big) \;+\; \log \mathcal{Z}_t(\beta) \;-\; \log|A_t| \;-\; \log C(\lambda, r_*), \tag{10.4}$$

with the radial integral and its normaliser

$$\mathcal{Z}_t(\beta) = \int_0^\infty \rho\, e^{-(\rho-r_*)^2/2\lambda}\, e^{-e^{-2t}\kappa\rho^2/2}\, I_0\big(e^{-t}\rho\beta\big)\, \mathrm{d}\rho, \qquad C(\lambda, r_*) = \int_0^\infty \rho\, e^{-(\rho-r_*)^2/2\lambda}\, \mathrm{d}\rho. \tag{10.5}$$

A quadratic form plus one one-dimensional Bessel integral. No network, no Monte Carlo, no curse of dimension — which is what makes this a reference rather than a benchmark.

Assumption at work

$C(\lambda, r_*)$ is **not optional**, and omitting it fails silently. The ring density in (10.1) is written unnormalised, and its normaliser depends on both parameters of the well. If one estimates ψ at fixed (λ, r_*), the term is a constant and cancels in every responsibility — which is exactly why it can sit missing for a long time without anything going wrong. The moment one estimates λ, it stops being a constant: a wider well simply contains more mass, so the likelihood increases without bound in λ and the estimate pins to the top of whatever grid it is given. Measured on this implementation, with the truth at $\lambda = 0.05$:

λ	0.02	0.03	0.05	0.08	0.11	0.15
without C	−1870	−1435	−911	−476	−218	**0**
with C	−43.0	−13.9	**−0.6**	−35.3	−96.0	−190

The upper row is smooth, confident and wrong. `tests/test_ring.py` pins both directions: that the profile likelihood peaks at the truth, *and* that it does not when the normaliser is removed — so deleting the term cannot make the suite greener.

Assumption at work

The count differs between the two likelihoods, and this is not a slip. The *joint* model invokes the ring density once, for z_0, and propagates; it subtracts $\log C$ once. The per-frame *marginal* model treats the frames as independent, so it invokes the ring density T times and must subtract $T \log C$. The first version of this code fixed the joint likelihood and left the marginal one wrong, and the error surfaced only when the blind arm of the potential experiment returned $\hat{\lambda} = 3.5 \times 10^6$.

10.4 Marginal blindness

Theorem 10.1 (Marginal blindness)**.** *For the model* (10.1)–(10.2) *with uniform initial phase, every single-frame marginal is independent of ψ, at every noise level. Consequently a model built from per-frame marginals carries exactly zero Fisher information about the rotation.*

Proof. Unrolling (10.2),

$$z_u = R_\psi^u z_0 + \sigma \sum_{k=1}^{u} R_\psi^{u-k} \eta_k.$$

Each η_k is isotropic Gaussian and a rotation of an isotropic Gaussian has the same law, so $R_\psi^{u-k}\eta_k \overset{d}{=} \eta_k$, independently across k. And θ_0 is uniform on the circle, so z_0 has a rotationally invariant law and $R_\psi^u z_0 \overset{d}{=} z_0$. The two terms are independent, hence

$$z_u \overset{d}{=} z_0 + \sigma\sqrt{u}\,\xi, \qquad \xi \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_2)\,, \tag{10.6}$$

in which ψ does not appear. The channel acts coordinatewise and independently of ψ, so the noised frame is $x_u \overset{d}{=} e^{-t}(z_0 + \sigma\sqrt{u}\,\xi) + \sqrt{\Delta_t}\,\xi'$, again free of ψ. Therefore $\partial_\psi \log P_t(x_u) \equiv 0$ for every u and every t, so each frame's Fisher information about ψ is exactly zero. A sum of per-frame log-densities cannot contain information that no term contains. □

It is worth being precise about what this does *not* say. The marginals are not uninformative in general — they determine λ, r_* and σ perfectly well, and the per-frame law is a visible, non-Gaussian ring whose radius grows as $\sqrt{u}$ by (10.6). They are blind only to the *rotation*. Nor does the theorem say the joint is easy: the joint carries the information, and extracting it is what the estimator does.

The sharp form of the statement is that this is not a matter of degree. A per-frame model is not *less* informative about the dynamics, or informative only at low noise. It carries zero, exactly, at every noise level. Radii are not evidence.

10.5 Three estimation targets on one model

The chapter's design follows from the two preceding sections. The same model carries a parameter the marginals can see and a parameter they provably cannot, so running both through the same estimator isolates the effect of the structure rather than of the method.

	target	free parameters	visible to marginals?
4a	the confining potential (λ, r_*)	two	**yes**
4b	a single rotation ψ	one	**no** (Theorem 10.1)
4c	a law over rotations $p(\psi)$	a distribution	**no**

Each is run in two arms differing only in which likelihood scores a trajectory: the joint (10.4), or the product of per-frame marginals. That makes the rung a 3×2 design whose diagonal is the claim.

4a, the potential. Gradient ascent on $(\log\lambda, r_*)$ from a random initialisation, with a numerical gradient — honest for two parameters, and it avoids a hand-derived derivative that could be wrong in the same silent way the normaliser was. This is the *easy* case, and it is easy for a reason one can state: by (10.6) frame u's radial law is the well convolved with a Gaussian of variance $\sigma^2 u$, so every frame carries information about it.

Assumption at work

The iteration budget is doing real work here, exactly as it does for EM on the chain (Chapter 8). At 25 gradient steps this fit reports $\lambda = 0.075$ against a truth of 0.05 and looks perfectly well-behaved doing it; the likelihood plateaus near 150 steps and the estimate settles by 300 ($\lambda = 0.0493$, $r_* = 0.9966$ at $n = 512$). The experiment therefore runs to a plateau criterion with 300 as a cap, and records a `hit_cap` flag so that a fit which stopped because it ran out of budget cannot be read as one that converged.

4b, one rotation. Profile the likelihood over a grid of angles and refine parabolically through the peak, so the estimate is not limited by the grid. One scalar — fewer parameters than the potential — and by Theorem 10.1 no marginal carries any information about it.

4c, a law over rotations. EM for a mixture whose components are indexed by the rotation, on a truth of two species $p^*(\psi) = \frac{1}{2}\delta_{+\omega} + \frac{1}{2}\delta_{-\omega}$:

$$\text{E:} \quad r_\mu(\psi_k) \propto p(\psi_k)\, P_t(\mathbf{x}^\mu \mid \psi_k), \qquad \text{M:} \quad p(\psi_k) \leftarrow \frac{1}{M}\sum_\mu r_\mu(\psi_k).$$

The same sufficiency that makes $\mathbf{\Xi}$ a one-shot statistic on the chain appears here. The ψ-conditional log-likelihood does not depend on $p(\psi)$ — the thing being estimated — so the $M \times |\psi|$ matrix of log-densities is computed *once* and every EM iteration is a reweighting of it. The E-step touches the data one time and the M-step never revisits it.

The blind arm is not a strawman baseline; it is the theorem's own control. Its likelihood is constant in ψ, so its responsibilities equal $p(\psi)$ exactly, its M-step is the identity, and $p(\psi)$ can never leave its uniform initialisation. If the run shows anything else, the implementation is wrong rather than the theorem.

10.6 What the frozen batch found

Sixteen seeds, seven budgets, twelve noise levels, from `outputs/frozen/exp_28_ring_em`.

The blind arm returns exactly the null. Across the 1,344 cells of 4c the estimate moves 0.0000 in total variation from its uniform initialisation, and across the 480 cells of 4b the profile likelihood in ψ has range 0.00×10^0 nats. Not a small number — a machine-precision zero, everywhere, which is what a theorem predicting exactly no information requires. Reported as $\hat{\omega}$, the blind arm returns 90.00°, the value a uniform $p(\psi)$ gives, at every budget and every noise level.

The joint arm recovers the rotation, and loses it to the channel. Estimating a single angle, the error is $2.1 \pm 0.7°$ pooled over the whole schedule. Recovering the two-species law, the median error in ω against a truth of 30° and a null of 90°:

t	0.05	0.32	0.68	0.98	1.43	3.00
median $\lvert\hat{\omega} - \omega\rvert$	0.15°	0.24°	0.90°	2.86°	7.09°	58.8°

Information about the rotation survives until $t \approx 1$ and is gone by $t = 3$, where the estimate has returned to the null. That is the channel doing what a channel does, and it is visible here in a way it is not on the chain, because the null is known exactly.

The potential. [PENDING] — the 4a sweep runs 300 gradient steps per cell and had not completed when this was written. The preliminary result is that both arms recover (λ, r_*), which is the expected contrast with 4b: the well is visible to the marginals and the rotation is not.

10.7 The port, and how it was checked

`src/ring.py` is a port of code in `research/board-3problems`, which is audited there against independent polar quadrature (check `P1d`, the ψ-posterior against quadrature at 10^{-9}). A port is exactly the kind of change that can agree on the easy cases and drift somewhere nobody looks, so it is checked rather than assumed:

- Before the normalisation fix, the two implementations agreed **bit for bit**: maximum absolute difference 0.0 over a batch of 400 trajectories.
- After it, they differ by exactly $\log|A_t| + \log C$ — constant across the batch to 3.6×10^{-15}, and equal to the intended constant to 2.7×10^{-15}. So the change is provably the normalisation and nothing else.
- Marginal blindness is confirmed numerically as a distributional statement rather than a pathwise one: per-frame second moments agree across $\psi \in \{0, \pm 30°, 90°\}$ within four standard errors at $n = 40{,}000$, and match the closed form $\mathbb{E}\|z_0\|^2 + 2\sigma^2 u$.

Implementation

`src/ring.py` → `log_pt_conditional` is (10.4); `src/ring.py` → `log_pt_marginal` is the blind model, and takes no ψ argument because there is no ψ to take. `experiments/exp_28_ring_em.py` runs the three targets as four parts; `tests/test_ring.py` carries eleven checks including both directions of the normaliser.

One performance note, because it changed the experiment's feasibility rather than only its speed: `src/ring.py` → `log_pt_marginal` needs T different values of κ, one per frame, and originally rebuilt the whole Bessel table for each. Since I_0 depends only on $z = e^{-t}\beta\rho$ and not on κ, the expensive part can be computed once and shared across frames. That is an 11× speed-up (522 ms → 48 ms per call) with identical output. The tell that the cost was in the quadrature and not the data was that the marginal likelihood took the same time at $n = 512$ and $n = 4096$.

Part VII

Background, at full length

Chapter 11

Derivations from Statistical Mechanics

These are the derivations underlying Chapter 14. They are placed here so that the body carries the argument and the appendix carries the work; nothing in them is omitted.

11.1 Hamiltonian mechanics and phase space

Consider a mechanical system with n degrees of freedom, described by generalised coordinates $q = (q_1, \ldots, q_n)$ and conjugate momenta $p = (p_1, \ldots, p_n)$. Its dynamics is generated by a single scalar function, the *Hamiltonian* $H(q, p)$, which for the systems considered here has the mechanical form

$$H(q,p) \;=\; \underbrace{\sum_{i=1}^{n} \frac{p_i^2}{2m_i}}_{\text{kinetic}} \;+\; \underbrace{U(q_1,\ldots,q_n)}_{\text{potential}}, \tag{11.1}$$

and equals the total energy of the system. The state of the system at any instant is the pair (q, p), a point in the $2n$-dimensional *phase space* $\Gamma \subseteq \mathbb{R}^{2n}$. The evolution is given by *Hamilton's equations*,

$$\dot{q}_i \;=\; \frac{\partial H}{\partial p_i}, \qquad \dot{p}_i \;=\; -\frac{\partial H}{\partial q_i}, \qquad i = 1, \ldots, n. \tag{11.2}$$

Two structural properties of (11.2) are the pillars on which statistical mechanics rests.

Energy conservation. Along any trajectory,

$$\frac{\mathrm{d}H}{\mathrm{d}t} = \sum_{i=1}^{n} \left(\frac{\partial H}{\partial q_i}\,\dot{q}_i + \frac{\partial H}{\partial p_i}\,\dot{p}_i \right) = \sum_{i=1}^{n} \left(\frac{\partial H}{\partial q_i}\frac{\partial H}{\partial p_i} - \frac{\partial H}{\partial p_i}\frac{\partial H}{\partial q_i} \right) = 0. \tag{11.3}$$

The motion is therefore confined to the *energy shell* $\Gamma_E = \{(q,p) : H(q,p) = E\}$, a $(2n-1)$-dimensional hypersurface.

Incompressibility of the phase flow. The phase-space velocity field

$$v(q,p) \;=\; (\dot{q}, \dot{p}) \;=\; \left(\frac{\partial H}{\partial p}, -\frac{\partial H}{\partial q} \right) \tag{11.4}$$

has identically vanishing divergence:

$$\nabla \cdot v \;=\; \sum_{i=1}^{n} \left(\frac{\partial \dot{q}_i}{\partial q_i} + \frac{\partial \dot{p}_i}{\partial p_i} \right) \;=\; \sum_{i=1}^{n} \left(\frac{\partial^2 H}{\partial q_i \partial p_i} - \frac{\partial^2 H}{\partial p_i \partial q_i} \right) \;=\; 0. \tag{11.5}$$

Hamiltonian flow behaves like an incompressible fluid in phase space: it can stretch and fold a region, but never change its volume. This identity is the seed of the probabilistic construction that follows.

11.2 Liouville's theorem

A single trajectory is useless for macroscopic physics: for a gas, $n \sim 10^{23}$ and neither the initial condition nor the solution of (11.2) is accessible. The move that founds statistical mechanics is to describe our *ignorance* of the microstate by a probability density $\rho(q,p,t)$ on phase space: $\rho(q,p,t)\,\mathrm{d}q\,\mathrm{d}p$ is the probability that the system is in the infinitesimal cell around (q,p) at time t. Liouville's theorem states how this density evolves.

Theorem 11.1 (Liouville)**.** *Under the Hamiltonian flow* (11.2)*, the phase-space density obeys*

$$\frac{\partial \rho}{\partial t} = -\{\rho, H\} = \sum_{i=1}^{n}\left(\frac{\partial H}{\partial q_i}\frac{\partial \rho}{\partial p_i} - \frac{\partial H}{\partial p_i}\frac{\partial \rho}{\partial q_i}\right), \tag{11.6}$$

where $\{\cdot,\cdot\}$ is the Poisson bracket. Equivalently, the density is constant along trajectories: $\mathrm{d}\rho/\,\mathrm{d}t = 0$*.*

Toolbox 11.1 — Proof of Liouville's theorem, step by step

The density is locally conserved (probability mass is neither created nor destroyed, only transported by the flow), so it satisfies the continuity equation familiar from fluid dynamics:

$$\frac{\partial \rho}{\partial t} + \nabla\cdot(\rho\, v) = 0, \qquad v = (\dot q, \dot p).$$

Expand the divergence of the product using $\nabla\cdot(\rho v) = v\cdot\nabla\rho + \rho\,(\nabla\cdot v)$:

$$\frac{\partial \rho}{\partial t} + \underbrace{\sum_{i=1}^{n}\left(\dot q_i\frac{\partial \rho}{\partial q_i} + \dot p_i\frac{\partial \rho}{\partial p_i}\right)}_{v\cdot\nabla\rho} + \rho\underbrace{\sum_{i=1}^{n}\left(\frac{\partial \dot q_i}{\partial q_i} + \frac{\partial \dot p_i}{\partial p_i}\right)}_{\nabla\cdot v} = 0.$$

By the incompressibility identity (11.5) the last term vanishes. Substituting Hamilton's equations $\dot q_i = \partial H/\partial p_i$, $\dot p_i = -\partial H/\partial q_i$ into the middle term:

$$\frac{\partial \rho}{\partial t} + \sum_{i=1}^{n}\left(\frac{\partial H}{\partial p_i}\frac{\partial \rho}{\partial q_i} - \frac{\partial H}{\partial q_i}\frac{\partial \rho}{\partial p_i}\right) = 0 \quad\Longleftrightarrow\quad \frac{\partial \rho}{\partial t} = -\{\rho, H\}.$$

Finally, the total (convective) derivative along a trajectory is

$$\frac{\mathrm{d}\rho}{\mathrm{d}t} = \frac{\partial \rho}{\partial t} + v\cdot\nabla\rho = \frac{\partial \rho}{\partial t} + \{\rho, H\} = 0,$$

which is the second form of the statement: an observer riding a phase-space trajectory sees a constant local density. □

Liouville's theorem has an immediate consequence: *any density that is a function of conserved quantities only is stationary.* If $\rho(q,p) = f\big(H(q,p)\big)$ for some function f, then $\{\rho, H\} = f'(H)\,\{H, H\} = 0$, so $\partial\rho/\partial t = 0$. Equilibrium statistical mechanics is the study of which f nature chooses.

11.3 From dynamics to ensembles

An *ensemble* is a probability distribution over microstates that is stationary under the dynamics and compatible with the macroscopic constraints we impose (fixed energy, fixed temperature, and so on). Two ensembles matter for this thesis.

11.3.1 The microcanonical ensemble

For a perfectly isolated system, the energy is fixed at E and Liouville's theorem suggests the simplest choice compatible with it: a uniform density on the energy shell,

$$\rho_{\mathrm{mc}}(q,p) \;=\; \frac{1}{\Omega(E)}\,\delta\big(H(q,p)-E\big), \qquad \Omega(E) = \int_\Gamma \delta\big(H(q,p)-E\big)\,\mathrm{d}q\,\mathrm{d}p, \tag{11.7}$$

where $\Omega(E)$ counts the microstates at energy E. The postulate that all accessible microstates are equally likely (the *equal a priori probability* postulate) is motivated, not proved, by ergodicity: for ergodic dynamics, time averages along a single trajectory converge to averages over the shell, so the uniform measure is the one a long experiment effectively samples. Boltzmann's entropy attaches a number to this count,

$$S(E) \;=\; k_B \log \Omega(E), \tag{11.8}$$

with k_B the Boltzmann constant. Entropy is the logarithm of the number of microscopic ways a macroscopic state can be realised.

11.3.2 The canonical ensemble and the Boltzmann distribution

Isolated systems are an idealisation; the physically ubiquitous situation is a system exchanging energy with a much larger environment (a *heat bath*) at temperature T. The stationary distribution of the system alone is then no longer uniform: it is the *Boltzmann–Gibbs distribution*,

$$\boxed{\rho_\beta(x) \;=\; \frac{1}{Z(\beta)}\,e^{-\beta H(x)}, \qquad Z(\beta) = \int e^{-\beta H(x)}\,\mathrm{d}x, \qquad \beta = \frac{1}{k_B T},} \tag{11.9}$$

where $x=(q,p)$ collects the microstate and $Z(\beta)$, the *partition function*, normalises the density. Equation (11.9) can be derived in two independent ways, and both derivations are instructive enough to be carried out in full: the first (physical) traces the exponential to the entropy of the bath; the second (information-theoretic) characterises it as the least-committal distribution compatible with a fixed mean energy, a perspective due to Jaynes [29] that returns, almost verbatim, in modern machine learning.

Toolbox 11.2 — The Boltzmann distribution from the heat bath

Let the total system (system S together with bath B) be isolated with total energy E_{tot}, and let the coupling be weak, so $H_{\mathrm{tot}} = H_S + H_B$. Apply the microcanonical postulate to the total system: the probability that S is in a *specific* microstate x with energy $H_S(x)=\epsilon$ is proportional to the number of bath microstates carrying the remaining energy,

$$\mathbb{P}(x) \;\propto\; \Omega_B\big(E_{\mathrm{tot}}-\epsilon\big) \;=\; \exp\Big[\tfrac{1}{k_B} S_B\big(E_{\mathrm{tot}}-\epsilon\big)\Big],$$

using the definition (11.8) of the bath entropy. Since the bath is huge, $\epsilon \ll E_{\mathrm{tot}}$, expand S_B to first order:

$$S_B(E_{\mathrm{tot}}-\epsilon) = S_B(E_{\mathrm{tot}}) - \epsilon\,\frac{\partial S_B}{\partial E}\Big|_{E_{\mathrm{tot}}} + O\big(\epsilon^2/E_{\mathrm{tot}}\big).$$

The derivative of the bath entropy with respect to its energy *defines* the bath temperature, $\partial S_B/\partial E = 1/T$. Substituting,

$$\mathbb{P}(x) \;\propto\; \underbrace{e^{S_B(E_{\text{tot}})/k_B}}_{\text{constant in } x} \cdot e^{-\epsilon/(k_B T)} \;\propto\; e^{-\beta H_S(x)}.$$

The quadratic remainder is suppressed by the bath's heat capacity and vanishes in the thermodynamic limit. Normalising yields (11.9). Note what happened: the exponential form is nothing but the first-order Taylor expansion of the bath's entropy. The Boltzmann factor is the price, in bath microstates, of lending energy ϵ to the system. □

Toolbox 11.3 — The Boltzmann distribution from maximum entropy

Forget physics; ask, with Jaynes [29]: among all densities ρ with a given mean energy $\mathbb{E}_\rho[H] = U$, which one assumes the least beyond the constraint? "Least" is measured by the Gibbs–Shannon entropy $S[\rho] = -k_B \int \rho \log \rho$. Maximise $S[\rho]$ subject to $\int \rho = 1$ and $\int \rho H = U$ with Lagrange multipliers λ_0, λ_1:

$$\mathcal{L}[\rho] = -k_B \int \rho \log \rho \,\mathrm{d}x \;-\; \lambda_0\Big(\int \rho \,\mathrm{d}x - 1\Big) \;-\; \lambda_1\Big(\int \rho H \,\mathrm{d}x - U\Big).$$

Take the functional derivative and set it to zero pointwise:

$$\frac{\delta \mathcal{L}}{\delta \rho(x)} = -k_B\big(\log \rho(x) + 1\big) - \lambda_0 - \lambda_1 H(x) = 0 \;\;\Longrightarrow\;\; \rho(x) = \exp\Big[-1 - \tfrac{\lambda_0}{k_B} - \tfrac{\lambda_1}{k_B} H(x)\Big].$$

The density is an exponential of the constrained quantity. This is completely general: constraining $\mathbb{E}[\phi_j(x)]$ for features ϕ_j produces $\rho \propto \exp[-\sum_j \beta_j \phi_j(x)]$, the exponential family. The multiplier λ_0 is fixed by normalisation and produces Z; renaming $\lambda_1/k_B = \beta$ and identifying $\beta = 1/(k_B T)$ through thermodynamics recovers (11.9) exactly. Second-order check: $\delta^2\mathcal{L}/\delta\rho^2 = -k_B/\rho < 0$, so the stationary point is the unique maximum (the entropy is strictly concave). □

The second derivation deserves a remark. The Boltzmann distribution is not a statement about molecules but about *inference under constraints*. Any time we posit a model of the form $p(x) \propto e^{-E(x)}$, as we will for Markov random fields (Chapter 4) and energy-based models (Appendix 13), we are doing statistical mechanics, whether the variables are spins, pixels, or the frames of a video.

11.4 The partition function and free energy

The partition function $Z(\beta)$ in (11.9) looks like a mere normaliser; it is in fact the central computational object of the theory, and the reason approximate inference exists as a field. Its logarithm defines the *Helmholtz free energy*,

$$F(\beta) \;=\; -\frac{1}{\beta} \log Z(\beta). \tag{11.10}$$

Toolbox 11.4 — Everything is a derivative of $\log Z$

Differentiate $\log Z$ with respect to β, carrying the derivative through the integral:

$$-\frac{\partial \log Z}{\partial \beta} = -\frac{1}{Z}\frac{\partial}{\partial \beta}\int e^{-\beta H} = \frac{\int H\, e^{-\beta H}}{\int e^{-\beta H}} = \mathbb{E}_{\rho_\beta}[H] \;=\; U,$$

the mean energy. Differentiate once more:

$$\frac{\partial^2 \log Z}{\partial \beta^2} = \frac{\partial}{\partial \beta}\left(-\frac{\int He^{-\beta H}}{Z}\right) = \frac{\int H^2 e^{-\beta H}}{Z} - \left(\frac{\int He^{-\beta H}}{Z}\right)^2$$
$$= \mathbb{E}[H^2] - \mathbb{E}[H]^2 \;=\; \mathrm{Var}(H) \;\geq\; 0.$$

So $\log Z$ is convex in β, its first derivative gives the mean energy, its second the energy fluctuations; the latter equals $k_B T^2 C_V$ with C_V the heat capacity. This is a *fluctuation–dissipation* relation: how a system responds to heating is determined by its spontaneous equilibrium fluctuations.

The same identity holds for any observable coupled linearly to a field. If the Gibbs weight carries a factor $e^{h\cdot a}$, then $Z(h) = \int \nu(a)e^{h\cdot a}\, \mathrm{d}a$ is a cumulant generating function and

$$\frac{\partial \langle a_k\rangle}{\partial h_j} = \frac{\partial^2 \log Z}{\partial h_j \partial h_k} = \mathrm{Cov}(a_k, a_j), \tag{11.11}$$

so a susceptibility is a covariance. Susceptibilities defined this way are the standard probe of phase transitions, and they are the instrument used by Bachtis et al. [30] to locate the transitions discussed in Section 14.6. Chapter 10 shows that the response of a diffusion score to its own observations is exactly of the form (11.11), with the observation playing the role of the field. Equation (11.11) is a static identity within one fixed measure, and is not the two-time dynamical fluctuation–dissipation theorem of non-equilibrium statistical mechanics. Finally, rewriting the Gibbs–Shannon entropy of ρ_β:

$$S[\rho_\beta] = -k_B \mathbb{E}\big[\log \rho_\beta\big] = -k_B \mathbb{E}\big[-\beta H - \log Z\big] = \frac{U}{T} + k_B \log Z,$$

and multiplying by $-T$: $-TS = -U + F$, i.e.

$$F = U - TS.$$

The free energy is the exact trade-off between energy and entropy: at low T minimising F means minimising energy (ordered states win), at high T it means maximising entropy (disordered states win). Phase transitions are the non-analyticities of F where the winner changes. □

There is one more characterisation of F worth recording, because it is the statistical-mechanics form of the variational principle that underlies both the training objective of diffusion models (Chapter 16) and the Bethe free energies of message passing. For *any* trial density μ,

$$\underbrace{\mathbb{E}_\mu[H] - \tfrac{1}{\beta} S[\mu]/k_B}_{\text{variational free energy } F[\mu]} \;=\; F \;+\; \tfrac{1}{\beta}\, D_{\mathrm{KL}}(\mu \,\|\, \rho_\beta) \;\;\geq\;\; F, \tag{11.12}$$

with equality iff $\mu = \rho_\beta$: the Boltzmann distribution is the unique minimiser of the variational free energy, and the gap is exactly a Kullback–Leibler divergence. Every variational method in machine learning is a restriction of (11.12) to a tractable family of trial densities.

11.5 The Hopfield dynamics descends the energy

Chapter 14 uses this statement; the two lines of algebra behind it are not the point there, so they are here.

Let $h_i = \sum_{j \neq i} J_{ij}\sigma_j$ be the local field on neuron i and suppose the update (14.6) flips it, so $\sigma_i' = -\sigma_i$. Only the terms of H containing σ_i change, hence

$$\Delta H \;=\; H(\sigma') - H(\sigma) \;=\; -\,(\sigma_i' - \sigma_i)\,h_i \;=\; -\,2\,\sigma_i'\,h_i.$$

The update sets $\sigma_i' = \mathrm{sign}(h_i)$, so $\sigma_i' h_i = |h_i| \geq 0$ and $\Delta H \leq 0$, strictly whenever a flip actually occurs. H is bounded below on a finite configuration space, so the dynamics terminates in a local minimum and *every* initial condition falls into an attractor.

The single-pattern case $P = 1$, the Mattis model [31], is gauge-equivalent to the uniform ferromagnet: the relabelling $\sigma_i \to \xi_i\sigma_i$ maps the stored pattern onto the all-up state. It returns in Section 16.4 as the exactly solvable teacher in a modern analysis of how energy-based models learn.

11.6 First- and second-order transitions

The order of a transition is the order of the derivative of the free energy that first misbehaves.

At a *first-order* transition the first derivative jumps: the order parameter is discontinuous, latent heat is absorbed, the two phases coexist at the transition, and correlation lengths stay finite. At a *second-order* transition the first derivative is continuous and the second diverges: the order parameter grows continuously from zero, the susceptibility (11.11) and the correlation length diverge, and observables acquire power laws.

The distinction is what makes the learning cascade of Section 16.4 detectable through susceptibilities rather than through jumps, and it is also why the finite models of this thesis cannot exhibit either in the strict sense: both notions require the thermodynamic limit, and a correlation length that grows until it reaches the system size and then stops is a finite-size ceiling.

11.7 The learning cascade in detail

Supporting Section 16.4. The chapter states that RBM training is a cascade of second-order transitions and what that means for this thesis; the mapping that establishes it, and the empirical evidence, are here.

gradient (14.7). The central idea is to track training through the singular value decomposition of the weight matrix,

$$W \;=\; \sum_{\gamma=1}^{r} w_\gamma\, u_\gamma \bar{u}_\gamma^{\mathsf{T}}, \tag{11.13}$$

whose modes $(u_\gamma, \bar{u}_\gamma)$ act as candidate order parameters: the magnetisation of the visible layer along mode γ, $m_\gamma = u_\gamma \cdot v/\sqrt{N_v}$, measures whether the learned Gibbs measure has developed structure in that direction. At initialisation the weights are small and the model is a paramagnet: no direction in configuration space is preferred. The claim of Bachtis et al. [30] is that training is an *effective cooling*: as the singular values grow, the model crosses a sequence of genuine second-order phase transitions, one for each feature it learns, each signalled by a diverging susceptibility and falling in the mean-field (Curie–Weiss) universality class. The mechanism is visible in a two-line calculation.

Toolbox 11.5 — The retrieval transition of a one-mode RBM

Following Bachtis et al. [30], take a single Gaussian hidden unit of variance $1/N_v$ coupled to N_v binary visibles through a weight vector w: the energy is $E(v,h) = -h\, w \cdot v + \frac{N_v}{2} h^2$. Integrating out the Gaussian h gives the visible marginal

$$p(v) \;\propto\; \exp\Big[\tfrac{1}{2N_v}\,(w \cdot v)^2\Big] \;:$$

a Mattis/Hopfield measure (Section 14.4) whose single pattern is w itself, made extensive by the $1/N_v$ variance scaling. Write $w = \bar{w}\,\xi$ with $\xi \in \{-1,+1\}^{N_v}$ and let $m = \frac{1}{N_v}\sum_i \xi_i v_i$ be the overlap; then $p(v) \propto \exp[\frac{N_v \bar{w}^2}{2} m^2]$. Counting configurations at fixed overlap ($\asymp \exp[N_v H(\frac{1+m}{2})]$ with H the binary entropy) gives the free-entropy density

$$\phi(m) \;=\; \frac{\bar{w}^2}{2}\,m^2 + H\Big(\frac{1+m}{2}\Big) \;=\; \log 2 + \frac{\bar{w}^2 - 1}{2}\,m^2 - \frac{m^4}{12} + O(m^6).$$

This is a Landau expansion: the quadratic coefficient changes sign at $\bar{w}^2 = \|w\|^2/N_v = 1$. Below the threshold the maximiser is $m = 0$ (paramagnet); above it two symmetric maximisers $\pm m^*$ appear continuously (retrieval). The transition is second order with mean-field exponents, and the susceptibility along w diverges as $\|w\|^2/N_v \uparrow 1$. Growing a singular value during training is therefore literally a quench through a Curie–Weiss critical point. $\square$

Why a cascade. With data generated by a Hopfield teacher whose two patterns are correlated combinations $\xi^{1,2} = \eta_1 \pm \eta_2$ of orthogonal directions, the data covariance is rank two with unequal strengths, and the early-time gradient flow of the student's SVD amplitudes along η_1 and η_2 is exponential with *two different rates*. The stronger direction (the centre of mass of the clusters) reaches its critical magnitude first: the learned measure splits into two lumps along $\pm\eta_1$. The weaker direction keeps growing at its own rate and crosses criticality later, splitting each lump again: four modes, the full teacher, retrieved in hierarchical order. Each crossing is a distinct second-order transition with its own critical time, its own diverging susceptibility, and hysteresis below it. Learning proceeds coarse-to-fine not by accident of optimisation but because the critical times are ordered by the spectrum of the data.

Evidence on real data. On MNIST, CelebA, and human-genome data, Bachtis et al. [30] track the susceptibilities of individual SVD modes during training of binary–binary RBMs and observe the same signatures: successive susceptibility peaks that sharpen with system size, finite-size-scaling collapses consistent with mean-field exponents $(\gamma, \nu) = (1, \frac{1}{2})$ at upper critical dimension four, and hysteresis, the operational fingerprints of genuine transitions rather than smooth crossovers. This confirms and sharpens the earlier picture of RBM learning as progressive alignment with the data's principal components [32], of feature emergence as condensation [33], and of sequential mode learning in linear networks [34]; a broad review of the EBM–physics interface is Carbone [35].

Chapter 12

Stochastic Calculus and the Diffusion Channel

The Itô calculus, the Fokker–Planck equation, sampling by dynamics, and the AR(1) chain in full, supporting Chapter 14.

12.1 The AR(1) chain: the running example of the thesis

We now introduce the specific Markov chain that the research chapters solve completely: the first-order autoregressive (AR(1)) process, the canonical Gaussian Markov chain [36]. Let $\alpha \in (-1, 1)$ and let (η_k) be i.i.d. $\mathcal{N}(0, \sigma_\eta^2)$ innovations; define

$$a_{k+1} \;=\; \alpha\, a_k + \eta_k, \qquad k = 0, 1, \ldots, K-2. \tag{12.1}$$

The transition kernel is Gaussian, $M(a'|a) = \mathcal{N}(a'; \alpha a, \sigma_\eta^2)$, and the joint law of the trajectory follows from (15.2). Throughout the thesis we use the stationary normalisation $\sigma_\eta^2 = 1 - \alpha^2$ and start the chain in its stationary law, $a_0 \sim \mathcal{N}(0, 1)$, so that every frame has unit marginal variance. A trajectory $a = (a_0, \ldots, a_{K-1})$ is *one* data point, the prototype of a video or a time series; its frame index k is an internal time, to be kept distinct from the diffusion time t introduced in Section 15.3. Everything about this chain is computable in closed form; the next toolbox derives the three facts used throughout.

Toolbox 12.1 — Stationary distribution and autocovariance of AR(1)

(i) Stationary variance. Suppose $a_k \sim \mathcal{N}(0, \sigma^2)$ and impose that a_{k+1} has the same variance. Since η_k is independent of a_k,

$$\operatorname{Var}(a_{k+1}) = \alpha^2 \operatorname{Var}(a_k) + \sigma_\eta^2 \stackrel{!}{=} \operatorname{Var}(a_k) \quad \Longrightarrow \quad \sigma_\infty^2 = \frac{\sigma_\eta^2}{1-\alpha^2}.$$

The fixed point exists precisely because $|\alpha| < 1$; the map $\sigma^2 \mapsto \alpha^2\sigma^2 + \sigma_\eta^2$ is a contraction, so *any* initial variance converges to σ_∞^2 geometrically. Since Gaussianity is preserved by the affine recursion, the stationary law is $\pi = \mathcal{N}(0, \sigma_\infty^2)$, which equals $\mathcal{N}(0, 1)$ under the stationary normalisation.

(ii) Autocovariance. In the stationary regime, for $d \geq 1$, unroll the recursion d times: $a_{k+d} = \alpha^d a_k + \sum_{j=0}^{d-1} \alpha^j \eta_{k+d-1-j}$. All innovation terms are independent of a_k, hence

$$\operatorname{Cov}(a_k, a_{k+d}) = \alpha^d \operatorname{Var}(a_k) = \alpha^d\, \sigma_\infty^2,$$

and by symmetry $\mathrm{Cov}(a_i, a_j) = \alpha^{|i-j|}\sigma_\infty^2$ for all i, j: correlations decay *geometrically* with the lag, with rate α. The covariance matrix of K consecutive frames,

$$(\Sigma_0)_{ij} = \sigma_\infty^2\, \alpha^{|i-j|},$$

is symmetric Toeplitz, as stationarity requires (Section 15.1).
(iii) Detailed balance. With $\pi = \mathcal{N}(0, \sigma_\infty^2)$ and $M(a'|a) = \mathcal{N}(a'; \alpha a, \sigma_\eta^2)$, both sides of (15.3) are proportional to

$$\exp\left[-\frac{a^2}{2\sigma_\infty^2} - \frac{(a'-\alpha a)^2}{2\sigma_\eta^2}\right] = \exp\left[-\frac{a^2 + (a')^2 - 2\alpha\, aa'}{2\sigma_\eta^2}\right],$$

where the exponent on the right is *symmetric* in (a, a'). Hence the stationary AR(1) chain is reversible: statistically, the movie of the chain looks the same played in either direction. □

The AR(1) chain is thus a working miniature of everything this chapter discusses: a Markov kernel with an explicit stationary law, geometric convergence, reversibility, and, decisive for us, exact Gaussian algebra at every step.

12.2 Sampling by dynamics: MCMC and Langevin

Sampling inverts the logic of the previous sections. There, a kernel M was given and we asked for its stationary law π. In sampling problems we are given π, typically a Boltzmann distribution $\pi(x) \propto e^{-\beta H(x)}$ known only up to its partition function, and we must *design* a kernel whose stationary law is π; the ergodic theorem (15.4) then converts one long trajectory into expectation values. This is how the negative phase of Boltzmann-machine learning (14.7) is estimated in practice, and the idea originates, fittingly, in computational statistical mechanics [37].

The *Metropolis–Hastings* chain [37, 38] moves from x by proposing $x' \sim g(\cdot|x)$ from any tractable proposal kernel and accepting the move with probability

$$A(x \to x') \;=\; \min\left\{1,\; \frac{\pi(x')\, g(x \mid x')}{\pi(x)\, g(x' \mid x)}\right\}, \tag{12.2}$$

staying at x otherwise. The target enters only through the *ratio* $\pi(x')/\pi(x)$, so the intractable partition function cancels: Metropolis–Hastings samples from $e^{-\beta H}/Z$ while only ever evaluating energy differences. The acceptance rule is engineered so that the resulting kernel satisfies detailed balance (15.3) with respect to π; the verification is a short computation [39, Chapter 7]. When x is high-dimensional but each conditional $\pi(x_i|x_{\setminus i})$ is tractable, the *Gibbs sampler* [40] cycles through the coordinates resampling each from its conditional, a Metropolis–Hastings move whose acceptance probability is identically one. Gibbs sampling thrives exactly when the model has sparse conditional structure: for a Markov chain, $\pi(x_k|x_{\setminus k})$ depends only on the two neighbours, and for the bipartite RBM of Section 14.5 each layer updates in one parallel step. Inference cost follows graph structure, a principle that Chapter 4 makes systematic.

For continuous x and differentiable energy, a different design principle uses gradients. Consider the discrete-time update

$$x_{k+1} \;=\; x_k + \epsilon\, \nabla \log \pi(x_k) \;+\; \sqrt{2\epsilon}\, \xi_k, \qquad \xi_k \sim \mathcal{N}(0, I), \tag{12.3}$$

the (unadjusted) *Langevin algorithm*. The drift pushes towards high probability; the noise maintains exploration. Two observations prepare what follows. First, (12.3) requires only the *score* $\nabla \log \pi$; the normalising constant is never needed. This is precisely why the score is the natural object for generative modelling (Chapter 16), and why so much of this thesis is devoted to computing scores exactly. Second, as $\epsilon \to 0$ the recursion (12.3) becomes a stochastic differential equation, the Langevin diffusion, whose stationary law is exactly π. Making that limit precise requires the Itô calculus and Fokker–Planck theory that occupy the rest of the chapter, where the claim is verified by direct computation.

12.3 From ODEs to SDEs: the Itô calculus

An ordinary differential equation $\dot{x} = f(x, t)$ moves a point along a deterministic velocity field. Physical systems, however, feel rapid, unresolved forces (collisions, thermal agitation) whose cumulative effect is well modelled by adding noise:

$$\mathrm{d}X_t \;=\; \underbrace{f(X_t, t)\,\mathrm{d}t}_{\text{drift}} \;+\; \underbrace{g(t)\,\mathrm{d}W_t}_{\text{diffusion}}. \tag{12.4}$$

Here W_t is a *Wiener process* (standard Brownian motion): the unique process with $W_0 = 0$, independent increments, $W_t - W_s \sim \mathcal{N}(0, t - s)$, and continuous paths. Equation (12.4) is shorthand for the integral equation $X_t = X_0 + \int_0^t f(X_s, s)\,\mathrm{d}s + \int_0^t g(s)\,\mathrm{d}W_s$, and the whole subtlety lives in the last term: Brownian paths are nowhere differentiable, so $\int g\,\mathrm{d}W$ cannot be a Riemann–Stieltjes integral. Itô's construction [41] defines it as the limit of left-endpoint sums,

$$\int_0^t g(s)\,\mathrm{d}W_s \;=\; \lim_{n\to\infty} \sum_{j=0}^{n-1} g(s_j)\,\big(W_{s_{j+1}} - W_{s_j}\big), \tag{12.5}$$

where the evaluation at the *left* endpoint s_j makes the integrand independent of the coming increment. Two consequences follow immediately: the Itô integral is a martingale (zero mean), and its variance is given by the *Itô isometry*, $\mathbb{E}\big[(\int_0^t g\,\mathrm{d}W)^2\big] = \int_0^t g(s)^2\,\mathrm{d}s$, both because cross terms $\mathbb{E}[g(s_j)(W_{s_{j+1}} - W_{s_j})]$ vanish by independence.

It is worth recording why no convention-free definition is available, since the same two sums reappear in Chapter 10. Fix $[0, t]$ and a partition of mesh tending to zero. The *quadratic variation* $Q_n = \sum_j (W_{s_{j+1}} - W_{s_j})^2$ has mean $\sum_j \Delta s_j = t$ and variance $2\sum_j (\Delta s_j)^2$, which vanishes in the limit, so $Q_n \to t$ in mean square. The *total variation* $V_n = \sum_j |W_{s_{j+1}} - W_{s_j}|$ instead diverges, because $Q_n \le (\max_j |W_{s_{j+1}} - W_{s_j}|)\,V_n$ and the maximum increment tends to zero by continuity: a bounded V_n would force $Q_n \to 0$, contradicting $Q_n \to t > 0$. A Riemann–Stieltjes integral $\int f\,\mathrm{d}g$ requires g to have bounded variation, precisely so that the limit does not depend on where inside each subinterval f is evaluated. For Brownian motion that guarantee is absent, different evaluation points give genuinely different limits, and a convention must be fixed.

The price of the construction is a new chain rule. Its origin is the quadratic variation just computed: increments scale as $\mathrm{d}W_t \sim \sqrt{\mathrm{d}t}$, so second-order terms in $\mathrm{d}W$ are first-order in $\mathrm{d}t$ and cannot be discarded.

Toolbox 12.2 — Itô's lemma, and where the extra term comes from

Let $Y_t = \phi(X_t, t)$ with ϕ twice differentiable and X_t solving (12.4). Taylor-expand ϕ to second order:

$$\mathrm{d}Y = \frac{\partial \phi}{\partial t}\,\mathrm{d}t + \frac{\partial \phi}{\partial x}\,\mathrm{d}X + \frac{1}{2}\frac{\partial^2 \phi}{\partial x^2}\,(\mathrm{d}X)^2 + \ldots$$

Substitute $\mathrm{d}X = f\,\mathrm{d}t + g\,\mathrm{d}W$ and expand the square:

$$(\mathrm{d}X)^2 = f^2(\mathrm{d}t)^2 + 2fg\,\mathrm{d}t\,\mathrm{d}W + g^2(\mathrm{d}W)^2.$$

Now use the multiplication table of Itô calculus, which is nothing but the scaling $\mathrm{d}W \sim \sqrt{\mathrm{d}t}$ made systematic:

$$(\mathrm{d}t)^2 = 0, \qquad \mathrm{d}t\,\mathrm{d}W = 0, \qquad (\mathrm{d}W)^2 = \mathrm{d}t.$$

The last identity is the rigorous statement that the quadratic variation of W over $[0,t]$ equals t: for a partition of mesh $\to 0$, $\sum_j (W_{s_{j+1}} - W_{s_j})^2 \to t$ in mean square, since the sum has mean $\sum_j (s_{j+1} - s_j) = t$ and variance $2\sum_j (s_{j+1} - s_j)^2 \to 0$. Keeping the surviving terms:

$$\mathrm{d}Y = \left(\frac{\partial \phi}{\partial t} + f\,\frac{\partial \phi}{\partial x} + \frac{g^2}{2}\,\frac{\partial^2 \phi}{\partial x^2}\right)\mathrm{d}t \;+\; g\,\frac{\partial \phi}{\partial x}\,\mathrm{d}W.$$

The half-Laplacian term, absent from the classical chain rule, is the engine of everything that follows: it is why densities spread, why the Fokker–Planck equation has a second-order term, and why exponential transformations of SDEs pick up variance corrections. □

12.4 The Fokker–Planck equation: transporting densities

The SDE (12.4) moves *samples*; the corresponding motion of the *density* $p_t(x)$ is governed by the Fokker–Planck (or forward Kolmogorov) equation. This change of viewpoint, from trajectories to densities, is the same one Liouville's theorem performed for Hamiltonian flow, and the derivation is parallel, with the Itô correction supplying a new, diffusive term.

Toolbox 12.3 — Derivation of the Fokker–Planck equation

Let ϕ be an arbitrary smooth test function of compact support, and compute $\frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}[\phi(X_t)]$ in two ways.

Way 1: through the samples. By Itô's lemma,

$$\mathrm{d}\phi(X_t) = \left(f\,\phi' + \tfrac{g^2}{2}\,\phi''\right)\mathrm{d}t + g\,\phi'\,\mathrm{d}W_t,$$

and taking expectations kills the martingale term:

$$\frac{\mathrm{d}}{\mathrm{d}t}\,\mathbb{E}[\phi(X_t)] = \mathbb{E}\Big[f(X_t,t)\,\phi'(X_t)\Big] + \frac{g(t)^2}{2}\,\mathbb{E}\big[\phi''(X_t)\big].$$

Way 2: through the density. $\mathbb{E}[\phi(X_t)] = \int \phi(x)\,p_t(x)\,\mathrm{d}x$, so

$$\frac{\mathrm{d}}{\mathrm{d}t}\,\mathbb{E}[\phi(X_t)] = \int \phi(x)\,\frac{\partial p_t(x)}{\partial t}\,\mathrm{d}x.$$

Equate the two, and integrate Way 1 by parts to move all derivatives off ϕ (boundary terms vanish by compact support): once for the drift term, twice for the diffusion term,

$$\int \phi\,\frac{\partial p_t}{\partial t} = \int \phi\left[-\frac{\partial}{\partial x}(f\,p_t) + \frac{g^2}{2}\,\frac{\partial^2 p_t}{\partial x^2}\right]\mathrm{d}x.$$

Since ϕ is arbitrary, the integrands must agree pointwise:

$$\boxed{\frac{\partial p_t}{\partial t} = -\frac{\partial}{\partial x}\big(f(x,t)\,p_t(x)\big) + \frac{g(t)^2}{2}\,\frac{\partial^2 p_t(x)}{\partial x^2}.}$$

The first term transports probability along the drift (as in Liouville); the second spreads it diffusively. The diffusive term is the macroscopic footprint of the Itô correction. □

In vector form, and for later reference, the Fokker–Planck equation of (12.4) reads

$$\partial_t p_t = -\nabla \cdot \big(f\, p_t\big) + \tfrac{1}{2} g^2\, \nabla^2 p_t. \tag{12.6}$$

12.4.1 Stationary law of Langevin dynamics: closing the loop

Apply the Fokker–Planck equation to the *Langevin SDE* associated with a potential U (the continuous-time limit of the algorithm (12.3)):

$$\mathrm{d}X_t = -\nabla U(X_t)\ \mathrm{d}t + \sqrt{2\beta^{-1}}\ \mathrm{d}W_t. \tag{12.7}$$

A stationary density solves $0 = \nabla\cdot(\nabla U\, p + \beta^{-1}\nabla p)$, i.e. the probability current $J = -(\nabla U)\, p - \beta^{-1}\nabla p$ is divergence-free; imposing the natural (decaying) boundary conditions forces $J \equiv 0$, hence $\nabla p/p = -\beta\nabla U$, hence

$$p_\infty(x)\ \propto\ e^{-\beta U(x)}\ : \tag{12.8}$$

Langevin dynamics equilibrates to the Boltzmann distribution of Chapter 14. This is the two-line proof of the claim left pending in Section 12.2, and it explains the deep kinship of sampling and physics: a Langevin sampler *is* an overdamped physical system at temperature $1/\beta$. Note also that the drift of (12.7) is (up to β) the *score* of the target, $-\nabla U = \nabla \log p_\infty$: dynamics that sample a distribution are driven by its score.

For the OU process, $U(x) = x^2/2$ and $\beta = 1$: the stationary law is $\mathcal{N}(0,1)$, confirming the $t \to \infty$ limit of Section 15.3.

Chapter 13

Derivations for Diffusion Models

Energy-based models, the DDPM variational bound, score matching in its two equivalent forms, and the probability-flow ODE, supporting Chapter 14.

13.1 Energy-based models

Modern generative modelling asks a single question: given samples of an unknown distribution p_0, build a machine that produces new ones. The last decade produced several answers: variational autoencoders [42], adversarial networks [43], normalizing flows [44], diffusion models. It is easy to present them as unrelated tricks. The energy-based view of LeCun et al. [45] is the antidote: it is the direct continuation of the Boltzmann-machine programme of Section 14.5, and, as this chapter unfolds, diffusion models will appear as the member of the energy-based family that finally solved its oldest problem.

An *energy-based model* (EBM) over $x \in \mathcal{X}$ assigns to each configuration a scalar energy $E_\theta(x)$ measuring how badly x fits the model's idea of the data: low energy for plausible configurations, high for implausible ones. In LeCun et al.'s formulation the energy is the primitive object: inference means finding low-energy configurations, and learning means shaping the landscape, pushing the energy of observed data down and the energy of everything else up. A probabilistic model is recovered whenever one normalises with the Gibbs measure of Chapter 14,

$$p_\theta(x) \; = \; \frac{e^{-E_\theta(x)}}{Z(\theta)}, \qquad Z(\theta) = \int_{\mathcal{X}} e^{-E_\theta(x)} \, \mathrm{d}x, \tag{13.1}$$

the Boltzmann distribution (11.9) with β absorbed into the energy and the physics replaced by a parametric family. The appeal of (13.1) is that E_θ is unconstrained: any function defines a valid unnormalised density. The price is the partition function, and every basic operation hits it. *Evaluating* $p_\theta(x)$ requires $Z(\theta)$, intractable in general (Section 14.2). *Sampling* requires the MCMC machinery of Section 12.2, whose mixing time can be exponential in the presence of metastability. *Learning* by maximum likelihood requires the gradient

$$-\nabla_\theta \log p_\theta(x) = \nabla_\theta E_\theta(x) - \mathbb{E}_{p_\theta}\big[\nabla_\theta E_\theta\big], \tag{13.2}$$

the "push down on data, pull up on the model's own samples" rule, the continuous-variable form of the Boltzmann-machine rule (14.7), whose second term again demands sampling from p_θ. LeCun et al. [45] frame the alternatives systematically: either fight the negative phase with *contrastive* losses that pull up only on well-chosen offending configurations (contrastive divergence [46] being the MCMC-flavoured member of the family), or design architectures and regularisers that bound the volume of low-energy space so that no pull-up is needed. The lesson of the tutorial is that the partition function is not an accounting detail; it is *the* design constraint of the field.

There are two classical escape routes, and this thesis walks both. The first is the *score*: since $\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$, the gradient of the log-density in x is free of Z, because differentiating in x kills the constant. Sections 13.3–16.2 build the modern edifice standing on this observation. The second is *structure*: when the energy decomposes into local terms, marginals and even Z itself become computable by dynamic programming; that route (Markov random fields, factor graphs, message passing) is developed in Chapter 4.

13.2 Denoising diffusion probabilistic models

The discrete-time formulation [47, 48] discretises (16.1) into a Markov chain $q(x_s|x_{s-1}) = \mathcal{N}(x_s; \sqrt{1-\beta_s}\, x_{s-1}, \beta_s I)$ with a variance schedule $\beta_1, \ldots, \beta_S$, and learns a reverse chain $p_\theta(x_{s-1}|x_s)$. Training maximises a variational lower bound on the data log-likelihood; the physicist will recognise the Gibbs variational principle (11.12) in modern dress.

Toolbox 13.1 — The evidence lower bound of a diffusion model

For any latent-variable model with joint $p_\theta(x_{0:S})$ and any inference distribution $q(x_{1:S}|x_0)$, Jensen's inequality gives

$$\begin{aligned}\log p_\theta(x_0) &= \log \int p_\theta(x_{0:S})\ \mathrm{d}x_{1:S} = \log\, \mathbb{E}_q\left[\frac{p_\theta(x_{0:S})}{q(x_{1:S}|x_0)}\right] \\ &\geq\ \mathbb{E}_q\left[\log \frac{p_\theta(x_{0:S})}{q(x_{1:S}|x_0)}\right] \ =:\ \mathrm{ELBO}.\end{aligned}$$

Insert the two Markov factorisations $p_\theta(x_{0:S}) = p(x_S)\prod_s p_\theta(x_{s-1}|x_s)$ and $q(x_{1:S}|x_0) = \prod_s q(x_s|x_{s-1})$, then regroup telescopically using Bayes' rule $q(x_s|x_{s-1}) = q(x_{s-1}|x_s, x_0)\, q(x_s|x_0)/q(x_{s-1}|x_0)$:

$$\begin{aligned}-\mathrm{ELBO} \ &= D_{\mathrm{KL}}(q(x_S|x_0)\,\|\,p(x_S)) \ -\ \mathbb{E}_q \log p_\theta(x_0|x_1) \\ &\quad + \sum_{s=2}^{S} \mathbb{E}_q\, D_{\mathrm{KL}}(q(x_{s-1}|x_s, x_0)\,\|\,p_\theta(x_{s-1}|x_s))\,,\end{aligned}$$

one denoising step per summand. The decisive point: because the forward chain is Gaussian and conditioned on *both* endpoints, each target $q(x_{s-1}|x_s, x_0)$ is an explicit Gaussian. Matching it with a Gaussian p_θ reduces each KL term to a weighted regression of the posterior mean; concretely, to predicting the noise ε that was added [48]. The thermodynamic reading of the ELBO gap as an entropy-production bound goes back to Sohl-Dickstein et al. [47]. □

The practical upshot of Ho et al. [48] is the loss

$$\mathcal{L}(\theta) = \mathbb{E}_{s,\, a\sim p_0,\, \varepsilon}\Big\|\varepsilon - \varepsilon_\theta\big(\underbrace{a\, e^{-t_s} + \sqrt{\Delta_{t_s}}\,\varepsilon}_{x_{t_s}},\ s\big)\Big\|^2, \tag{13.3}$$

a denoising regression, and the empirical discovery that this simplified objective, with each noise level weighted equally and all the variational bookkeeping dropped, trains photorealistic image models. Its connection with the score is not incidental, as the next section shows: predicting the noise *is* estimating $\nabla \log p_t$ up to scale.

13.3 Score matching

Suppose we model the score directly by a function $s_\theta(x)$ and wish it to match $\nabla \log p(x)$ for data from p. The naive objective $\mathbb{E}_p\|s_\theta - \nabla \log p\|^2$ seems to require knowing the very score we lack. Two classical results remove the obstruction.

Toolbox 13.2 — Implicit score matching (Hyvärinen)

Expand the square and keep only θ-dependent terms:

$$\mathbb{E}_p \|s_\theta - \nabla \log p\|^2 = \mathbb{E}_p \|s_\theta\|^2 - 2\,\mathbb{E}_p \langle s_\theta, \nabla \log p\rangle + \text{const}.$$

The cross term integrates by parts. Componentwise, using $p\,\partial_i \log p = \partial_i p$ and assuming $p\,s_{\theta,i} \to 0$ at infinity:

$$\int p\ s_{\theta,i}\ \partial_i \log p\ \ \mathrm{d}x = \int s_{\theta,i}\ \partial_i p\ \ \mathrm{d}x = -\int p\ \partial_i s_{\theta,i}\ \ \mathrm{d}x.$$

Therefore

$$\mathbb{E}_p \|s_\theta - \nabla \log p\|^2 = \mathbb{E}_p\Big[\ \|s_\theta(x)\|^2 + 2\,\nabla \cdot s_\theta(x)\Big] + \text{const}:$$

an objective involving only the model and data samples [49]. Elegant, but the divergence term is expensive for large networks, which is why diffusion models use the denoising variant instead. □

Toolbox 13.3 — Denoising score matching (Vincent)

Fix t and consider the noised marginal (16.2), written as $p_t(x) = \int p_0(a)\,q_t(x|a)\ \mathrm{d}a$ with Gaussian kernel $q_t(x|a) = \mathcal{N}(x; ae^{-t}, \Delta_t I)$. Then

$$\nabla_x \log p_t(x) = \frac{\int p_0(a)\,\nabla_x q_t(x|a)\ \mathrm{d}a}{p_t(x)} = \int \underbrace{\frac{p_0(a)\,q_t(x|a)}{p_t(x)}}_{=\,p(a|x)} \nabla_x \log q_t(x|a)\ \mathrm{d}a,$$

so that

$$\nabla_x \log p_t(x)\ \ =\ \ \mathbb{E}\big[\nabla_x \log q_t(x|a)\ \big|\ x\big]\ :$$

the score of the *marginal* is the posterior average of the score of the *kernel*, and the latter is trivial, $\nabla_x \log q_t(x|a) = -(x - ae^{-t})/\Delta_t$. A standard L^2-projection argument turns this into a regression: for any s_θ,

$$\mathbb{E}_{x\sim p_t} \|s_\theta(x) - \nabla \log p_t(x)\|^2 = \mathbb{E}_{a,x} \|s_\theta(x) - \nabla_x \log q_t(x|a)\|^2 + \text{const},$$

because conditional expectation is exactly the L^2 projection onto functions of x [50]. Substituting the Gaussian kernel score and $x = ae^{-t} + \sqrt{\Delta_t}\varepsilon$ gives $\nabla_x \log q_t = -\varepsilon/\sqrt{\Delta_t}$: *learning the score of noisy data is learning to predict the noise*, and the DDPM loss (13.3) is denoising score matching in disguise. □

The continuous-time synthesis [51, 52] trains one network $s_\theta(x,t)$ against denoising score matching at all noise levels and samples with the reverse SDE (15.8); an accessible exposition is Song [53], and Karras et al. [54] systematise the design space. This is the resolution, promised in Section 13.1, of the energy-based model's oldest problem: Song and Ermon replace the energy by its gradient, the equilibrium sampler by a finite-time reverse SDE, and the intractable likelihood by a regression whose targets are exact.

13.4 Flows: the deterministic side of the same coin

Anderson's theorem has a deterministic twin. As noted in Section 15.4, the marginals p_t of the forward SDE are also transported by the noise-free *probability-flow ODE* with velocity $f - \frac{g^2}{2}\nabla \log p_t$ [52]: the same score, consumed by an ODE instead of an SDE, turns generation into a smooth, invertible map from Gaussian noise to data. This places diffusion models inside an older family. *Normalizing flows* [44, 55] construct an invertible map T_θ and evaluate likelihoods through the change-of-variables formula; their continuous-time version parametrises the velocity field of an ODE directly [56]. What diffusion brought to this family is a *simulation-free training signal*: *flow matching* [57] and stochastic interpolants [58] show that one can regress the velocity field of any prescribed interpolation between noise and data (the OU path (16.1) being one choice among many) with the same conditional trick as denoising score matching, never integrating the ODE during training; a systematic introduction is Holderrieth and Erives [59]. The unifying statement is worth recording: DDPMs, score SDEs, and flow-matching models all learn a vector field whose ground truth is a *conditional expectation under the noising channel*; they differ in the path of marginals and in whether sampling is stochastic or deterministic. Every exact result of this thesis therefore speaks to the whole family at once: the object we compute in closed form, the conditional expectation behind the field, is the training target of all of them.

Part VIII

The background chapters as first written

Chapter 14

Statistical Mechanics and the Origins of Learning Machines

Statistical mechanics is the discipline that explains how probabilistic laws emerge from deterministic dynamics. It is also the field in which the neural network was invented. This chapter serves both facts, and its two halves have distinct jobs. The first half builds the small set of physical tools that the rest of the thesis uses, beginning with the principle of stationary action — the variational pattern that recurs as free-energy minimisation, as the training objective of a diffusion model, and as maximum likelihood — and then (the Boltzmann distribution, the partition function and free energy, and the Ising energy), with the derivations carried out in full: the Boltzmann distribution is the stationary law of Langevin dynamics (Chapter 15), the free energy is the variational principle behind the diffusion-model training objective (Chapter 16), and sampling by simulating a physical process is statistical mechanics turned into an algorithm. The second half is the part of the literature review that most histories of machine learning skip: how spin glasses became the Hopfield network, how the Hopfield network became the Boltzmann machine, and how the statistical mechanics of these systems grew into the modern theory of learning. This is the tradition, recognised by the 2024 Nobel Prize in Physics, in which this thesis works. The physical presentation follows classical references [60, 61].

14.1 The principle of stationary action

Mechanics can be built from a single variational statement, and it is worth starting there rather than with Hamilton's equations, because the same statement — *a physical trajectory is a stationary point of a functional* — returns three more times in this thesis: as the free-energy principle of §11.4, as the variational bound that trains a diffusion model (Chapter 16), and as the maximum-likelihood problem solved by expectation–maximisation in Chapter 8. The objects change; the shape of the argument does not.

Let a system be described by generalised coordinates $q(t)$ and let the *Lagrangian* $L(q, \dot{q}, t)$ be the difference between kinetic and potential energy, $L = T - U$. For a path $q(t)$ joining fixed endpoints $q(t_1) = q_1$ and $q(t_2) = q_2$, define the *action*

$$S[q] \;=\; \int_{t_1}^{t_2} L\big(q(t), \dot{q}(t), t\big)\ \mathrm{d}t. \tag{14.1}$$

The action assigns one number to a whole path. Hamilton's principle states that the path actually taken is a *stationary point* of S: for any smooth variation $\delta q(t)$ vanishing at the endpoints, $\delta S = 0$.

Remark 14.1 (Stationary, not minimal). It is usually called the principle of *least* action, and that is slightly wrong. The physical path is stationary, not necessarily minimal — it can be a saddle. The same caution applies everywhere the pattern recurs: expectation–maximisation converges to a stationary point of the likelihood, not to a global maximum (Chapter 8), and it is a recurring source of false confidence to read "the algorithm converged" as "the algorithm found the best answer".

Carrying out the variation gives the Euler–Lagrange equations. Writing $q(t) \to q(t) + \delta q(t)$ and expanding to first order,

$$\delta S = \int_{t_1}^{t_2} \left(\frac{\partial L}{\partial q}\, \delta q + \frac{\partial L}{\partial \dot q}\, \delta \dot q \right) \mathrm{d}t,$$

and integrating the second term by parts, with the boundary term vanishing because $\delta q(t_1) = \delta q(t_2) = 0$,

$$\delta S = \int_{t_1}^{t_2} \left(\frac{\partial L}{\partial q} - \frac{\mathrm{d}}{\mathrm{d}t} \frac{\partial L}{\partial \dot q} \right) \delta q \;\mathrm{d}t.$$

Since this must vanish for every admissible δq, the bracket must vanish pointwise:

$$\frac{\mathrm{d}}{\mathrm{d}t} \frac{\partial L}{\partial \dot q_i} \;-\; \frac{\partial L}{\partial q_i} \;=\; 0, \qquad i = 1, \dots, n. \tag{14.2}$$

For $L = \sum_i \frac{1}{2} m_i \dot q_i^2 - U(q)$ this returns $m_i \ddot q_i = -\partial U / \partial q_i$, which is Newton's second law. The variational principle is therefore not a reformulation for its own sake: it derives the dynamics from a scalar functional, and that is the move which generalises.

From Lagrange to Hamilton. The passage to the Hamiltonian picture is a Legendre transform. Define the conjugate momentum $p_i = \partial L / \partial \dot q_i$ and set

$$H(q, p) \;=\; \sum_i p_i \dot q_i \;-\; L(q, \dot q), \tag{14.3}$$

with $\dot q$ eliminated in favour of p. For $L = T - U$ with T quadratic in the velocities this gives $H = T + U$, the total energy, which is (11.1) below; and (14.2) becomes the pair of first-order equations (11.2). The gain is not the equations themselves but the arena: the Hamiltonian formulation lives on phase space, where the state is a *point* whose evolution preserves volume, and that is precisely the structure statistical mechanics needs to speak about probability densities over states.

14.2 Interacting systems: the Ising paradigm

Everything so far holds for arbitrary H; the physics (and the difficulty) enters when the energy couples many degrees of freedom. The canonical example is the *Ising model* [62]: binary spins $\sigma_i \in \{-1, +1\}$ on the nodes of a graph $G = (V, \mathcal{E})$, with energy

$$H(\sigma) \;=\; -\sum_{(i,j) \in \mathcal{E}} J_{ij}\, \sigma_i \sigma_j \;-\; \sum_{i \in V} h_i\, \sigma_i, \tag{14.4}$$

where J_{ij} are pairwise couplings and h_i external fields. The Gibbs distribution $p(\sigma) \propto e^{-\beta H(\sigma)}$ makes neighbouring spins prefer alignment when $J_{ij} > 0$. Three lessons from this model shape the rest of the thesis.

(i) The partition function is the hard part. $Z = \sum_\sigma e^{-\beta H(\sigma)}$ runs over $2^{|V|}$ configurations. On two-dimensional lattices Onsager computed it exactly [63]; on general graphs the computation is #P-hard, and all of approximate inference (Monte Carlo sampling, variational methods, message passing) exists because of this wall. On *trees*, however, Z and all marginals are computable exactly in time linear in the system size by the transfer-matrix recursion, the same recursion that Chapter 4 presents as belief propagation. This tractable island is precisely where our research model lives.

(ii) Collective order emerges. For β beyond a critical value the two-dimensional model magnetises spontaneously: a macroscopic fraction of spins align even at $h = 0$, and the symmetric state splits into two ergodic components. This spontaneous symmetry breaking is the archetype of a *phase transition*, and its dynamical counterpart, a generation trajectory committing to one mode of a multimodal distribution, is exactly the *speciation* phenomenon of diffusion models [64, 65].

(iii) Disorder creates complexity. Making the couplings J_{ij} random (of both signs) yields spin glasses, and, as the next sections recount, spin glasses yielded neural networks. This is where the physics of this chapter stops being background and becomes the literature this thesis belongs to.

14.3 Disorder, frustration, and spin glasses

In a ferromagnet all couplings agree, and the two ground states (all spins up, all spins down) are found by inspection. Draw the couplings J_{ij} at random with both signs [66, 67] and this transparency collapses. A triangle with two positive bonds and one negative bond already cannot satisfy all three: whatever the spins do, at least one bond pays. This is *frustration*, and its large-N consequence is a free-energy landscape shattered into exponentially many valleys separated by extensive barriers: the system has many near-equally-good ways of being, none of them readable off the couplings. The mean-field theory of this phenomenon, developed through the replica and cavity methods [68], and the Thouless–Anderson–Palmer (TAP) self-consistency equations [69], took a decade to control and earned Parisi the 2021 Nobel Prize in Physics.

Two exports from this episode matter here. First, a language: rugged landscapes, metastable states, order parameters that measure similarity rather than magnetisation. Machine learning speaks this language every time it discusses loss surfaces and local minima. Second, a technology: the cavity and TAP equations are recursions on *local* quantities that solve a *global* inference problem, and they are the direct ancestors of the message-passing algorithms [70–72] that Chapter 4 develops and that the research chapters deploy on diffusion scores. The connection between disordered magnets and these algorithms is historical descent, not analogy.

14.4 The Hopfield network: memory as a phase of matter

Hopfield [73] asked a physicist's question about memory: can a system of simple binary units *store* and *retrieve* patterns as a collective effect, the way a magnet stores a magnetisation? His construction is an Ising system (14.4) in disguise. Take N binary neurons $\sigma_i \in \{-1, +1\}$ and P patterns $\xi^\mu \in \{-1, +1\}^N$ to be memorised, and wire the couplings by the *Hebb rule*, "neurons that fire together wire together" [74]:

$$J_{ij} \;=\; \frac{1}{N}\sum_{\mu=1}^{P} \xi_i^\mu \xi_j^\mu, \qquad H(\sigma) \;=\; -\frac{1}{2}\sum_{i\neq j} J_{ij}\,\sigma_i\sigma_j. \tag{14.5}$$

The dynamics is the simplest imaginable: pick a neuron, align it with its local field, repeat,

$$\sigma_i \;\leftarrow\; \mathrm{sign}\Big(\textstyle\sum_{j\neq i} J_{ij}\,\sigma_j\Big). \tag{14.6}$$

Toolbox 14.1 — The Hopfield dynamics descends the energy

Let $h_i = \sum_{j \neq i} J_{ij}\sigma_j$ be the local field on neuron i and suppose the update (14.6) flips it, $\sigma_i' = -\sigma_i$. Only the terms of H containing σ_i change, so

$$\Delta H \;=\; H(\sigma') - H(\sigma) \;=\; -\,(\sigma_i' - \sigma_i)\,h_i \;=\; -\,2\,\sigma_i'\,h_i.$$

The update sets $\sigma_i' = \mathrm{sign}(h_i)$, hence $\sigma_i' h_i = |h_i| \geq 0$ and $\Delta H \leq 0$, with strict decrease whenever a flip actually occurs. Since H is bounded below on a finite configuration space, the dynamics must terminate in a local minimum: *every* initial condition falls into an attractor. Memory retrieval is this fall: initialise at a corrupted pattern, descend, and read off the fixed point. □

The patterns are, by construction, near the bottoms of the energy (14.5): retrieval works because each memory is an attractor with a basin. How many memories fit is a genuine phase-transition question, answered with spin-glass tools by Amit et al. [75]: for random patterns with $P = \alpha N$, retrieval survives only below a critical load $\alpha_c \approx 0.138$, beyond which the memories are crushed into a useless glassy phase. In this model, memory is a phase of matter in the technical sense: it is entered and left through sharp transitions. The single-pattern case $P = 1$, known as the Mattis model [31], is gauge-equivalent to the uniform ferromagnet: flipping $\sigma_i \to \xi_i\sigma_i$ maps the stored pattern onto the all-up state. This observation is worth keeping in mind: the Mattis model returns in Section 16.4 as the exactly solvable teacher in a modern analysis of how energy-based models learn.

In 2024 the Nobel Prize in Physics went to John Hopfield and Geoffrey Hinton “for foundational discoveries and inventions that enable machine learning with artificial neural networks” [76]: formal recognition that the lineage traced in this section is physics, not just inspired by it.

14.5 From Hopfield to Boltzmann machines: learning enters

The Hopfield network stores what its designer wires into it. The decisive step towards machine *learning* was to make the couplings themselves the unknowns. Promote the zero-temperature dynamics (14.6) to stochastic updates at temperature $T = 1/\beta$ (each neuron aligns with its field only with probability given by the Boltzmann factor); the stationary law of this dynamics is exactly the Gibbs measure $p(\sigma) \propto e^{-\beta H(\sigma)}$ of the Ising energy. A *Boltzmann machine* [77] is this object run backwards: fix a dataset, and ask for the couplings J that make the Gibbs measure reproduce it. Gradient ascent on the log-likelihood gives a rule of striking symmetry,

$$\frac{\partial}{\partial J_{ij}}\,\mathbb{E}_{\mathrm{data}}\big[\log p(\sigma)\big] \;=\; \beta\,\Big(\underbrace{\langle\sigma_i\sigma_j\rangle_{\mathrm{data}}}_{\text{positive phase}} \;-\; \underbrace{\langle\sigma_i\sigma_j\rangle_{\mathrm{model}}}_{\text{negative phase}}\Big), \tag{14.7}$$

and learning stops when the model’s correlations match the data’s. This is the maximum-entropy principle of Toolbox 11.3 turned into an algorithm: matching prescribed moments with an exponential-family measure. The rule also contains, in miniature, the central obstruction of the field. The positive phase is an average over data, which is cheap. The negative phase is an average over the *model’s* Gibbs measure: a partition-function computation, the wall of Section 14.2, approachable only through sampling (Section 12.2).

The *restricted* Boltzmann machine (RBM) tames this cost by architecture [78]: split the units into a visible layer v and a hidden layer h with couplings only across layers, $E(v,h) = -v^{\mathsf{T}}Wh - b^{\mathsf{T}}v - c^{\mathsf{T}}h$. On this bipartite graph each layer is conditionally independent given the other, so Gibbs sampling alternates two exact, fully parallel steps, and the contrastive-divergence approximation to (14.7) made training practical [46]. Stacked RBMs, the deep belief networks, delivered the first working deep architectures and relaunched neural networks as a field [79]. The lineage then split. The supervised branch, powered by backpropagation [80], scaled through convolutional networks to the modern deep-learning era [81, 82] and largely left equilibrium physics behind. The *generative* branch, the one this thesis lives in, kept the Gibbsian core: a model is an energy, learning is moment matching, sampling is dynamics. Chapter 16 follows that branch from energy-based models to diffusion.

14.6 The statistical mechanics of learning

Physics did not stop contributing machines; it also built the theory of their behaviour. Gardner [83] showed that learning itself can be treated as an equilibrium problem: fix a set of examples and compute, by replica methods, the volume of coupling space compatible with them; capacity, generalisation, and their transitions then come out as thermodynamics in *weight* space. This programme, extended through teacher–student settings, matured into the modern statistical physics of inference [71], whose central lesson recurs throughout this thesis: high-dimensional inference problems have phases, and algorithms fail or succeed at sharp boundaries.

Two strands of this literature frame our research questions directly. The first is algorithmic: Mézard [84] derived the mean-field message-passing (TAP) equations of the Hopfield model and its generalisations, exhibiting the equivalence between Hopfield networks and RBMs and showing that retrieval can be organised as a message-passing computation. It is the same machinery, transplanted from memories to diffusion scores, that the research chapters run on our model. The second is dynamical: training itself undergoes phase transitions. In linear networks, gradient descent learns the singular modes of the data covariance sequentially [34]; in RBMs, the weight matrix's singular vectors align with the data's principal components in the early phase of learning [32], and hidden units develop compositional receptive fields through condensation transitions [33]. The sharpest result in this line, Bachtis et al. [30], examined in depth in Section 16.4, shows that RBM training is an entire *cascade* of second-order phase transitions, one per learned feature. Learning, like memory, turns out to be a sequence of symmetry breakings.

The order of a transition is the order of the derivative of the free energy that first misbehaves, and the distinction matters for reading these results. At a *first-order* transition the first derivative jumps: the order parameter is discontinuous, latent heat is absorbed, the two phases coexist at the transition, and correlation lengths stay finite. At a *second-order* transition the first derivative is continuous and the second diverges: the order parameter grows continuously from zero, the susceptibility (11.11) and the correlation length diverge, and observables acquire power laws. The cascade above is of the second kind, which is why it is detected through susceptibilities rather than through jumps. Both notions require the thermodynamic limit; in the finite models of Chapters 6 to 10 a correlation length that grows until it reaches the size of the system and then stops is a finite-size ceiling, not a divergence.

14.7 What this thesis takes from physics

A *complex system* is, operationally, a system whose macroscopic behaviour is not readable off its microscopic rules: interactions generate collective phenomena (order, phases, transitions, slow dynamics) that must be derived, not postulated. Statistical mechanics is the toolbox for that derivation, and after this chapter's story its objects map one-to-one onto the objects of modern generative modelling:

Statistical mechanics	**Generative modelling**
microstate x	data point / latent configuration
energy $H(x)$	negative log-density $-\log p(x)$ (up to Z)
Boltzmann distribution $e^{-\beta H}/Z$	model distribution
partition function Z	normalising constant / evidence
free energy F	negative ELBO (up to constants)
Hebbian couplings, attractors	stored memories, retrieval
moment matching (14.7)	maximum-likelihood training
Langevin dynamics	sampling / reverse diffusion
phase transition	feature condensation, speciation
cavity / TAP equations	belief propagation / AMP

The next chapter supplies the probabilistic dynamics, Markov chains and their continuous-time limits, that turn the static ensembles of this chapter into processes that move probability around, which is what a diffusion model ultimately is.

Chapter 15

Stochastic Processes, SDEs, and the Ornstein–Uhlenbeck Channel

Chapter 14 answered the question "which distribution?": Boltzmann's. This chapter answers "how does probability move?", in discrete and in continuous time. Its material enters the thesis in three places. Markov chains are the *data* whose diffusion score we compute exactly (the AR(1) sequence of Section 12.1 is the running example of every research chapter); stochastic differential equations are the *corruption and generation processes* of diffusion models (the Ornstein–Uhlenbeck channel of Section 15.3 appears on every page of the research chapters); and the Fokker–Planck and time-reversal theorems of Sections 12.4–15.4 are the mathematical basis of the generative programme reviewed in Chapter 16. Standard references are Brockwell and Davis [36], Robert and Casella [39], Øksendal [85], Gardiner [86], Risken [87].

15.1 Stochastic processes and stationarity

A *stochastic process* is a collection of random variables indexed by time, $(X_k)_{k\geq 0}$ in discrete time or $(X_t)_{t\geq 0}$ in continuous time, defined on a common probability space; equivalently, a probability law on the space of trajectories. Two notions of time homogeneity are used constantly in this thesis and are worth stating precisely [36].

Definition 15.1 (Strict and weak stationarity)**.** A process (X_k) is *strictly stationary* if all its finite-dimensional laws are invariant under time shifts: for every n, every set of indices $k_1, \ldots, k_n$, and every shift h,

$$(X_{k_1}, \ldots, X_{k_n}) \stackrel{d}{=} (X_{k_1+h}, \ldots, X_{k_n+h}).$$

It is *weakly (second-order) stationary* if its mean is constant and its autocovariance depends only on the lag, $\mathrm{Cov}(X_k, X_{k+d}) = \gamma(d)$.

Strict stationarity implies weak stationarity whenever second moments exist; for *Gaussian* processes the two notions coincide, because a Gaussian law is determined by its first two moments. Stationarity has an immediate matrix consequence that the research chapters exploit: the covariance matrix of K consecutive observations of a weakly stationary process, $(\Sigma)_{ij} = \gamma(i-j)$, is constant along diagonals, a symmetric *Toeplitz* matrix. Whenever a covariance in this thesis is Toeplitz, stationarity is the reason.

15.2 Markov chains: kernels, stationarity, reversibility

Definition 15.2 (Markov chain)**.** A sequence of random variables $(X_k)_{k\geq 0}$ with values in a state space $\mathcal{X}$ is a *Markov chain* if for every k and every measurable $A \subseteq \mathcal{X}$,

$$\mathbb{P}\big(X_{k+1} \in A \,\big|\, X_k, X_{k-1}, \ldots, X_0\big) \;=\; \mathbb{P}\big(X_{k+1} \in A \,\big|\, X_k\big). \tag{15.1}$$

The conditional law is encoded by a *transition kernel* $M(x' \,|\, x)$: a density (or matrix, for discrete $\mathcal{X}$) with $\int M(x'|x)\, \mathrm{d}x' = 1$ for every x.

The Markov property (15.1) is exactly a statement about *factorisation*: the joint law of the first K states is

$$p(x_0, x_1, \ldots, x_{K-1}) \;=\; p_0(x_0) \prod_{k=0}^{K-2} M(x_{k+1} \,|\, x_k). \tag{15.2}$$

This factorisation is the most heavily used equation of the thesis. It says that the joint distribution of a Markov trajectory is a product of *local* terms, each touching two consecutive variables: the structure that makes the chain a tree in the factor-graph sense of Chapter 4, that makes its precision matrix tridiagonal in the Gaussian case of Chapter 6, and that makes belief propagation exact.

Marginal distributions evolve by integrating the kernel: $p_{k+1}(x') = \int M(x'|x)\, p_k(x)\, \mathrm{d}x$, compactly $p_{k+1} = M p_k$. A distribution π is *stationary* (invariant) for M if $\pi = M\pi$; a chain started in π stays in π forever, and the trajectory is then a strictly stationary process in the sense of Definition 15.1. The kernel satisfies *detailed balance* with respect to π if

$$\pi(x)\, M(x' \,|\, x) \;=\; \pi(x')\, M(x \,|\, x') \qquad \text{for all } x, x' : \tag{15.3}$$

the equilibrium probability flux from x to x' equals the reverse flux, so the chain, watched in equilibrium, is statistically indistinguishable run forwards or backwards. Detailed balance implies stationarity (integrate (15.3) over x), and its power is practical: it is a local, pointwise condition, checkable without computing any integral.

Stationarity alone does not guarantee that the chain *finds* π; for that one needs the chain to be able to reach everywhere (*irreducibility*) without getting trapped in cycles (*aperiodicity*). Under these conditions the fundamental convergence theorem holds [39]: the marginal law of X_k converges to π from any initial condition, and time averages converge to expectations under π (the ergodic theorem),

$$\frac{1}{N} \sum_{k=1}^{N} f(X_k) \;\xrightarrow[N\to\infty]{\text{a.s.}}\; \mathbb{E}_\pi[f]. \tag{15.4}$$

Equation (15.4) is the rigorous form of the ergodic postulate invoked in Section 11.3, with the physical dynamics replaced by an algorithmic one.

15.3 The Ornstein–Uhlenbeck process: the corruption channel

The noising process used by diffusion models, and by every result of this thesis, is the *Ornstein–Uhlenbeck process* [88]: Brownian motion pulled back to the origin by a linear spring,

$$\mathrm{d}X_t \;=\; -X_t\, \mathrm{d}t \;+\; \sqrt{2}\, \mathrm{d}W_t, \qquad X_0 = a. \tag{15.5}$$

(The drift and diffusion constants are a normalisation choice, the variance-preserving convention of the diffusion literature [52]; any other choice rescales time.) The OU process is the unique Gaussian, Markov, stationary process in continuous time, and it is exactly solvable.

Toolbox 15.1 — Exact solution of the OU SDE by integrating factor

Multiply (15.5) by e^t and consider $Y_t = e^t X_t$. By Itô's lemma with $\phi(x,t) = e^t x$ (here $\partial^2\phi/\partial x^2 = 0$, so no correction term arises):

$$\mathrm{d}Y_t = e^t X_t\,\mathrm{d}t + e^t\big(-X_t\,\mathrm{d}t + \sqrt{2}\,\mathrm{d}W_t\big) = \sqrt{2}\,e^t\,\mathrm{d}W_t.$$

The drift cancels exactly; that is what the integrating factor is for. Integrate from 0 to t and undo the substitution:

$$X_t \;=\; a\,e^{-t} \;+\; \sqrt{2}\int_0^t e^{-(t-s)}\,\mathrm{d}W_s.$$

The integral is Gaussian (limit of Gaussian sums), with mean zero and variance given by the Itô isometry:

$$\mathrm{Var}(X_t) = 2\int_0^t e^{-2(t-s)}\,\mathrm{d}s = 2\,e^{-2t}\,\frac{e^{2t}-1}{2} = 1 - e^{-2t}.$$

Hence the *transition kernel* of the OU channel is

$$p(x_t \,|\, X_0 = a) \;=\; \mathcal{N}\big(x_t;\; a\,e^{-t},\; \Delta_t\big), \qquad \boxed{\Delta_t \;=\; 1 - e^{-2t}}.$$

□

The boxed kernel is used on every page of the research chapters, so we fix its reading now. Corrupting a data point a for a time t produces

$$x \;=\; a\,e^{-t} \;+\; \sqrt{\Delta_t}\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0,1): \tag{15.6}$$

the signal is *contracted* by e^{-t} while isotropic noise of variance Δ_t fills in. At $t = 0$: $x = a$ (no corruption); as $t \to \infty$: $x \sim \mathcal{N}(0,1)$ regardless of a (all information destroyed, the equilibrium of the spring). The signal-to-noise ratio e^{-2t}/Δ_t sweeps continuously from ∞ to 0, and it diverges as $1/2t$ for small t, a factor that reappears throughout the thesis as an error amplifier. The OU channel interpolates between the data distribution and a standard Gaussian, which is precisely what a diffusion model needs.

Two structural remarks, both load-bearing later. First, the OU kernel is Gaussian *in a as well as in x*: viewed as a function of the clean variable, $\mathcal{N}(x; ae^{-t}, \Delta_t) \propto \exp[-(e^{-2t}/2\Delta_t)(a - xe^t)^2]$, a Gaussian likelihood factor of precision e^{-2t}/Δ_t centred at the deconvolved observation $e^t x$. This is the fact that makes the posteriors of Chapter 6 jointly Gaussian and the messages of the BP computation closable. Second, if a random vector $a \sim \mathcal{N}(\mu, \Sigma_0)$ is pushed through (15.6) coordinate-wise with *independent* noises, the output is Gaussian with

$$\mu_t = e^{-t}\mu, \qquad \Sigma_t \;=\; e^{-2t}\,\Sigma_0 \;+\; \Delta_t\,I, \tag{15.7}$$

since means and covariances transform linearly and the added noise is white. Equation (15.7), shrink the covariance and add a multiple of the identity, is the entire forward process of this thesis in one line.

15.4 Time reversal: Anderson's theorem

One last piece of SDE theory converts everything above into a generative method. Suppose data $X_0 \sim p_0$ are corrupted by a forward SDE (12.4) up to time T, producing marginals p_t. Can the corruption be undone? Can we find an SDE which, run from $X_T \sim p_T$ backwards, reproduces the same marginals? The answer [89] is yes, and the reverse drift needs exactly one unknown object:

$$\mathrm{d}X_t \;=\; \Big[f(X_t,t) \;-\; g(t)^2\,\nabla_x \log p_t(X_t)\Big]\;\mathrm{d}t \;+\; g(t)\;\mathrm{d}\bar{W}_t, \tag{15.8}$$

where time runs backwards from T to 0 and $\bar{W}$ is a reverse-time Wiener process. The correction to the drift is the *score* $\nabla_x \log p_t$ of the noised marginal: the object defined in (2.2) and computed exactly, for Markov-chain data, in the research chapters. A heuristic that makes (15.8) plausible follows from the Fokker–Planck equation: rewriting its diffusion term as a transport term,

$$\frac{g^2}{2}\frac{\partial^2 p_t}{\partial x^2} = \frac{\partial}{\partial x}\left(\frac{g^2}{2}\,p_t\,\frac{\partial \log p_t}{\partial x}\right),$$

one sees that the density evolution of (12.4) is *identical* to that of the noise-free ODE with velocity $f - \frac{g^2}{2}\nabla \log p_t$ (the *probability-flow ODE* of 52); running that transport backwards, and re-injecting noise consistently, yields (15.8). The rigorous statement and proof are in Anderson [89].

15.5 From algorithmic chains to data chains

A closing change of perspective, to set up the research problem. In this chapter Markov chains began as *algorithms*: their stationary law was the target, their trajectory a means to an end. From Chapter 6 onwards the chain (12.1) plays the opposite role: it is the *data-generating process*, its trajectory $(a_0, \ldots, a_{K-1})$ is one data point (a video of K frames), and the object of interest is the probability law of the whole trajectory after each frame has been independently corrupted by the OU channel (15.6). The factorisation (15.2) then stops being a computational convenience and becomes the structure to be discovered: the research programme of this thesis asks how much of (15.2) survives noising, and how an inference algorithm, or a neural network, can exploit what survives. In one sentence: the OU channel destroys data at a known rate, the Fokker–Planck equation describes the destruction at the level of densities, and Anderson's theorem says the destruction is invertible by anyone who knows the score, which is why the rest of this thesis is about computing that score exactly when the data have Markov structure.

Chapter 16

From Energy-Based Models to Diffusion and Flows

Chapter 14 left the generative branch of machine learning at the Boltzmann machine: a model is an energy, learning is moment matching, sampling is dynamics. This chapter follows that branch to the present. It reviews, in the depth their role in this thesis demands, the three ideas the research chapters build on: the energy-based view of modelling [45], the score-matching principle that removed the partition function from training [49, 50], and the diffusion construction that turned denoising into a complete generative method [47, 48, 52], together with its deterministic sibling, the flow model. The chapter then reviews the statistical-physics literature on diffusion models, from the high-dimensional analyses of Biroli and Mézard [90] to the speciation, memorisation, and generalisation results of the Biroli–Mézard line of work, and, on the training side, the cascade of phase transitions in energy-based-model learning [30]. It closes by stating the gap this thesis addresses.

16.1 The diffusion idea

Diffusion models [47, 48, 52] solve the generative problem with a two-process design inspired by non-equilibrium thermodynamics: a fixed *forward process* gradually corrupts data into pure noise, and a learned *reverse process* undoes the corruption step by step. A recent book-length survey of the family is Lai et al. [91]. The forward process used throughout this thesis is the OU channel of Section 15.3: at diffusion time t,

$$x_t \;=\; a\,e^{-t} + \sqrt{\Delta_t}\,\varepsilon, \qquad a \sim p_0, \quad \varepsilon \sim \mathcal{N}(0, I), \qquad \Delta_t = 1 - e^{-2t}, \tag{16.1}$$

so that the noised marginal is the convolution

$$p_t(x) \;=\; \int p_0(a)\,\mathcal{N}\big(x;\, ae^{-t},\, \Delta_t I\big)\; \mathrm{d}a. \tag{16.2}$$

As t grows, p_t interpolates from the data distribution to $\mathcal{N}(0, I)$. By Anderson's theorem (Section 15.4), the corruption is undone by the reverse SDE (15.8), whose only unknown ingredient is the score $\nabla_x \log p_t(x)$ of the noised marginals. Generative diffusion is therefore, at bottom, *score estimation*: whoever knows $\nabla \log p_t$ for all t can sample from p_0. Note what this design buys relative to Section 13.1: no energy is ever normalised, no equilibrium is ever awaited. The Markov-chain convergence problem of MCMC is replaced by a *fixed-horizon* integration of an SDE whose drift is learned. The partition-function problem has not been solved; it has been side-stepped, by working only with objects (scores) from which Z has already been differentiated away.

16.1.1 The thermodynamic reading, and the direction of the arrow

The appeal to non-equilibrium thermodynamics in Sohl-Dickstein et al. [47] is structural rather than decorative. Their forward process is a chain of small perturbations converting a structured distribution into a simple one, that is an annealing schedule, and the objective they derive is a variational bound of the same algebraic form as the work identities of non-equilibrium statistical mechanics: the log-likelihood is bounded by an average of log-ratios along the forward path, exactly as free-energy differences are bounded in Jarzynski-type equalities and estimated in annealed importance sampling. That analogy is what supplies the training objective, and it is the reason the field speaks of temperature, annealing and entropy at all.

The direction of the forward process is fixed by the second law, which for the Fokker–Planck equation (12.6) takes an exact form. Let p_t evolve with a gradient drift and stationary law $p_\infty \propto e^{-V}$, and consider the free energy $\mathcal{F}[p_t] = D_{\mathrm{KL}}(p_t \,\|\, p_\infty)$, which is the mean energy minus the entropy up to a constant. Writing the dynamics as $\partial_t p = \nabla \cdot \big(p \nabla \log(p/p_\infty)\big)$ and integrating by parts,

$$\frac{\mathrm{d}\mathcal{F}}{\mathrm{d}t} = \int \partial_t p \, \log \frac{p}{p_\infty} = -\int p \left\| \nabla \log \frac{p}{p_\infty} \right\|^2 \;\le\; 0, \tag{16.3}$$

with equality only at $p_t = p_\infty$. This is the H-theorem: the free energy decreases monotonically and the relative entropy is consumed at a rate given by a relative Fisher information. The forward channel therefore destroys information about the data monotonically and irreversibly, and (16.3) identifies the rate. The reverse direction is not forbidden, but it is not free either: by Anderson's theorem it requires the score of the intermediate laws to be supplied from outside. The entire modelling effort of a diffusion model is the estimation of that one object, and Chapters 6 to 10 are concerned with what that object looks like when the data are trajectories.

16.2 The score as a posterior expectation

The middle line of Vincent's derivation deserves its own section, because every result of this thesis passes through it. Rearranging $\nabla \log p_t(x) = \mathbb{E}[-(x - ae^{-t})/\Delta_t \mid x]$:

$$\boxed{\nabla_x \log p_t(x) \;=\; \frac{e^{-t}\,\mathbb{E}[\,a \,|\, x\,] - x}{\Delta_t} \qquad \Longleftrightarrow \qquad \mathbb{E}[\,a \,|\, x\,] = e^{t}\Big(x + \Delta_t \, \nabla_x \log p_t(x)\Big).} \tag{16.4}$$

We refer to (16.4) throughout as the *score–posterior identity*; in the empirical-Bayes literature it is known as Tweedie's formula [92–94]. The score of the noisy marginal and the Bayes-optimal denoiser $\mathbb{E}[a|x]$ determine each other exactly, with no assumption whatsoever on p_0. Three readings will recur:

1. *Geometric*: the score points from x towards (the contracted image of) the posterior mean of the clean data; denoising direction and score direction coincide.

2. *Statistical*: estimating the score is solving a Bayesian inference problem, namely computing a posterior expectation. The score is not *like* an inference target; it *is* one.

3. *Algorithmic*: for sequence data with a Markov prior, that inference problem lives on a tree-shaped graph, so the score is computable by message passing, the programme made precise in Chapter 4 and executed in Chapter 6.

For *vector-valued* data $a \in \mathbb{R}^K$ noised coordinate-wise, (16.4) holds componentwise with the *full* posterior:

$$\big(\nabla_x \log p_t(x)\big)_k \;=\; \frac{e^{-t}\,\mathbb{E}[\,a_k \,|\, x_0, \ldots, x_{K-1}\,] - x_k}{\Delta_t}. \tag{16.5}$$

The conditioning is on *all* coordinates: even though the noise is independent across frames, the posterior mean of frame k, and hence the k-th score component, depends on every observation, because the prior couples the frames. Equation (16.5) is the precise reason the *joint* score differs from the per-frame marginal score, a distinction that is easy to blur and whose importance for structured data was impressed on this project in supervision (Chapter 1); Chapter 6 quantifies exactly how much coupling the conditioning induces, as a function of t.

16.3 The statistical physics of generative diffusion

A line of work initiated by Biroli, Mézard, and collaborators analyses diffusion models with the tools of Chapter 14, in the regime where the score is *exact*: either computed from a solvable p_0 or from the empirical measure of n training points. Because this is the literature our research extends, we review it in some detail.

High dimensions and the cost of asymmetry. Biroli and Mézard [90] analyse generative diffusion in the high-dimensional limit for two controlled data models, a Gaussian model and the Curie–Weiss ferromagnet. In the latter, the reverse dynamics undergoes the symmetry-breaking mechanism of Section 14.2: at a critical diffusion time the trajectory commits to one of the two low-temperature states. Their sharpest quantitative point concerns data requirements: to reconstruct the *relative weights* of two asymmetric modes, the number of training samples must be much larger than the data dimension, and the paper characterises the scaling laws in dimension and sample size for efficient generation.

The three regimes of the backward dynamics. Biroli et al. [64] study the backward process when the score has been trained optimally on n samples in dimension d, and identify three dynamical regimes separated by two transitions. Starting from pure noise, the trajectory first behaves as a free (Brownian-like) process; at the *speciation* time it commits to the gross structure of the data (a class, a mode), through a mechanism analogous to spontaneous symmetry breaking; at the later *collapse* time the trajectory is attracted to an individual memorised training point, through a mechanism analogous to condensation in a glass. The speciation time is computable from a spectral analysis of the data correlation matrix, and the collapse time from an excess-entropy estimate; the dependence of the collapse time on n and d quantifies the curse of dimensionality for diffusion models. Related transition phenomena along the generation trajectory were exhibited by Raya and Ambrogioni [65].

Speciation with general class structure. Achilli et al. [95] develop the general theory of the speciation transition: where earlier analyses required classes separated by their first moments (Gaussian mixtures with distinct means), they characterise speciation times for arbitrary class structures through a free-entropy difference criterion, recovering the known Gaussian-mixture results and extending them to classes that differ only through higher-order or collective features, with successive speciation events for multiple classes. A hierarchical counterpart is Sclocchi et al. [96]: in a hierarchical generative model of data, the backward process started from time t exhibits a phase transition at a threshold time at which the probability of preserving the high-level class drops abruptly, while low-level features evolve smoothly; beyond the transition a regenerated sample can retain low-level elements of the original image while its class has changed. The statistical-physics treatment of generative diffusion in solvable models is developed at book length in the doctoral thesis of Achilli [97].

Memorisation and generalisation. When the score is learned from finitely many samples rather than computed exactly, the question becomes when generation generalises and when it memorises. Bonnaire et al. [98] identify two timescales in the training dynamics: an early time τ_{gen} at which the model begins to generate high-quality samples, and a later time τ_{mem} beyond which memorisation of training data emerges; τ_{mem} grows linearly with the training-set size n while τ_{gen} stays constant, opening a widening window in which overparameterised models generalise, a form of implicit dynamical regularisation. Garnier-Brun et al. [99] refine the picture by identifying a phase of *biased* generalisation in between: the test loss still decreases while the model increasingly favours samples anomalously close to training points, so the standard early-stopping criterion does not certify genuine novelty. Ghio et al. [100] map generative samplers (flows, diffusions, autoregressive networks) onto statistical-physics sampling problems and delimit where each is efficient, importing the algorithmic-hardness picture of spin glasses [71].

The exactly solvable programme, and what it holds fixed. The common method of these works is to choose p_0 rich enough to be interesting and simple enough that $\nabla \log p_t$ is available in closed form, then study the exact generation dynamics. What none of them vary is the *internal structure of a single data point*: their coordinates are exchangeable or i.i.d. conditional on a mode. There is no ordering and no dynamics *inside* the sample. That axis is precisely where this thesis works.

16.4 Learning as a cascade of phase transitions

The works above put phase transitions on the *sampling* side of generative modelling. A complementary result, Bachtis et al. [30], which this section examines in detail, puts them on the *training* side, and completes the argument of Section 14.6 that learning itself is a sequence of symmetry breakings. The object of study is the restricted Boltzmann machine of Section 14.5, the minimal trainable EBM, with energy $E(v,h) = -v^\top W h - b^\top v - c^\top h$ and the moment-matching gradient (14.7). The central idea is to track training through the singular value decomposition of the weight matrix,

$$W \;=\; \sum_{\gamma=1}^{r} w_\gamma\, u_\gamma \bar{u}_\gamma^\top, \tag{16.6}$$

whose modes $(u_\gamma, \bar{u}_\gamma)$ act as candidate order parameters: the magnetisation of the visible layer along mode γ, $m_\gamma = u_\gamma \cdot v/\sqrt{N_v}$, measures whether the learned Gibbs measure has developed structure in that direction. At initialisation the weights are small and the model is a paramagnet: no direction in configuration space is preferred. The claim of Bachtis et al. [30] is that training is an *effective cooling*: as the singular values grow, the model crosses a sequence of genuine second-order phase transitions, one for each feature it learns, each signalled by a diverging susceptibility and falling in the mean-field (Curie–Weiss) universality class. The mechanism is visible in a two-line calculation.

Toolbox 16.1 — The retrieval transition of a one-mode RBM

Following Bachtis et al. [30], take a single Gaussian hidden unit of variance $1/N_v$ coupled to N_v binary visibles through a weight vector w: the energy is $E(v,h) = -h\, w\cdot v + \frac{N_v}{2}h^2$. Integrating out the Gaussian h gives the visible marginal

$$p(v) \;\propto\; \exp\!\left[\tfrac{1}{2N_v}\,(w\cdot v)^2\right] :$$

a Mattis/Hopfield measure (Section 14.4) whose single pattern is w itself, made extensive by the $1/N_v$ variance scaling. Write $w = \bar{w}\,\xi$ with $\xi \in \{-1,+1\}^{N_v}$ and let $m = \frac{1}{N_v}\sum_i \xi_i v_i$ be the overlap; then $p(v) \propto \exp[\frac{N_v \bar{w}^2}{2} m^2]$. Counting configurations at fixed overlap ($\asymp$

$\exp[N_v H(\frac{1+m}{2})]$ with H the binary entropy) gives the free-entropy density

$$\phi(m) \;=\; \frac{\bar{w}^2}{2}\,m^2 + H\Big(\frac{1+m}{2}\Big) \;=\; \log 2 + \frac{\bar{w}^2-1}{2}\,m^2 - \frac{m^4}{12} + O(m^6).$$

This is a Landau expansion: the quadratic coefficient changes sign at $\bar{w}^2 = \|w\|^2/N_v = 1$. Below the threshold the maximiser is $m = 0$ (paramagnet); above it two symmetric maximisers $\pm m^*$ appear continuously (retrieval). The transition is second order with mean-field exponents, and the susceptibility along w diverges as $\|w\|^2/N_v \uparrow 1$. Growing a singular value during training is therefore literally a quench through a Curie–Weiss critical point. □

Why a cascade. With data generated by a Hopfield teacher whose two patterns are correlated combinations $\xi^{1,2} = \eta_1 \pm \eta_2$ of orthogonal directions, the data covariance is rank two with unequal strengths, and the early-time gradient flow of the student's SVD amplitudes along η_1 and η_2 is exponential with *two different rates*. The stronger direction (the centre of mass of the clusters) reaches its critical magnitude first: the learned measure splits into two lumps along $\pm\eta_1$. The weaker direction keeps growing at its own rate and crosses criticality later, splitting each lump again: four modes, the full teacher, retrieved in hierarchical order. Each crossing is a distinct second-order transition with its own critical time, its own diverging susceptibility, and hysteresis below it. Learning proceeds coarse-to-fine not by accident of optimisation but because the critical times are ordered by the spectrum of the data.

Evidence on real data. On MNIST, CelebA, and human-genome data, Bachtis et al. [30] track the susceptibilities of individual SVD modes during training of binary–binary RBMs and observe the same signatures: successive susceptibility peaks that sharpen with system size, finite-size-scaling collapses consistent with mean-field exponents $(\gamma, \nu) = (1, \frac{1}{2})$ at upper critical dimension four, and hysteresis, the operational fingerprints of genuine transitions rather than smooth crossovers. This confirms and sharpens the earlier picture of RBM learning as progressive alignment with the data's principal components [32], of feature emergence as condensation [33], and of sequential mode learning in linear networks [34]; a broad review of the EBM–physics interface is Carbone [35].

What this means for the thesis. Phase transitions now appear on both faces of generative diffusion: in *sampling*, as speciation events along the reverse trajectory [64, 95], and in *training*, as the cascade just described. Both are hierarchies of symmetry breakings ordered by the data's spectrum. Our research adds the axis that both literatures hold fixed: the structure of the inference problem *inside a single data point*, and how the diffusion time t deforms it. The chain we solve is the minimal model in which that question is non-trivial and answerable exactly.

16.5 The gap, restated

Real sequential data (video, audio, trajectories) have exactly the structure the solvable programme leaves out: an internal (frame) time with a Markovian dependence along it. For such data the score (16.5) is a smoothing posterior on a chain, an object with seventy years of inference theory behind it. Yet the questions a diffusion modeller asks about it are not answered by the classical literature, which fixes the noise level, nor by the diffusion literature, which ignores internal structure: how does the coupling structure of $\nabla \log p_t$ deform with t? Can it be computed locally? What survives a Gaussian parametrisation of the messages when the data are not Gaussian? What limited guidance does that give for score architectures? Answering these questions, in the fully solvable setting of the AR(1) chain and its non-Gaussian perturbations, is the contribution of Chapters 6 and 7, with the inference tools prepared in Chapter 4 and the path that led to the precise formulation recorded in Chapter 1.

Bibliography

[1] Brian D. O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982.

[2] Ulrich G. Haussmann and Etienne Pardoux. Time reversal of diffusions. *The Annals of Probability*, 14(4):1188–1205, 1986.

[3] Herbert Robbins. An empirical Bayes approach to statistics. In *Proceedings of the Third Berkeley Symposium on Mathematical Statistics and Probability, Volume 1*, pages 157–163, 1956.

[4] Koichi Miyasawa. An empirical Bayes estimator of the mean of a normal population. *Bulletin of the International Statistical Institute*, 38(4):181–188, 1961.

[5] Bradley Efron. Tweedie's formula and selection bias. *Journal of the American Statistical Association*, 106(496):1602–1614, 2011.

[6] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.

[7] Aapo Hyvärinen. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6:695–709, 2005.

[8] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems*, 2019.

[9] Frank R. Kschischang, Brendan J. Frey, and Hans-Andrea Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.

[10] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems.* Morgan Kaufmann, 1988.

[11] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation.* Oxford University Press, 2009.

[12] Martin J. Wainwright and Michael I. Jordan. Graphical models, exponential families, and variational inference. *Foundations and Trends in Machine Learning*, 1(1–2):1–305, 2008.

[13] Leonard E. Baum, Ted Petrie, George Soules, and Norman Weiss. A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains. *The Annals of Mathematical Statistics*, 41(1):164–171, 1970.

[14] Lawrence R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.

[15] Rudolph E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[16] Herbert E. Rauch, F. Tung, and Charlotte T. Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8):1445–1450, 1965.

[17] Olivier Cappé, Eric Moulines, and Tobias Rydén. *Inference in Hidden Markov Models*. Springer, 2005.

[18] Song Mei. U-Nets as belief propagation: efficient classification, denoising, and diffusion in generative hierarchical models, 2024.

[19] Steven M. Kay. *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993.

[20] Erich L. Lehmann and George Casella. *Theory of Point Estimation*. Springer, 2nd edition, 1998.

[21] Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 39(1):1–38, 1977.

[22] C. F. Jeff Wu. On the convergence properties of the EM algorithm. *The Annals of Statistics*, 11(1):95–103, 1983.

[23] Xiao-Li Meng and Donald B. Rubin. Maximum likelihood estimation via the ECM algorithm: a general framework. *Biometrika*, 80(2):267–278, 1993.

[24] Ronald A. Fisher. Theory of statistical estimation. *Mathematical Proceedings of the Cambridge Philosophical Society*, 22(5):700–725, 1925.

[25] Thomas A. Louis. Finding the observed information matrix when using the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 44(2):226–233, 1982.

[26] Henry Teicher. Identifiability of finite mixtures. *The Annals of Mathematical Statistics*, 34(4):1265–1269, 1963.

[27] Sidney J. Yakowitz and John D. Spragins. On the identifiability of finite mixtures. *The Annals of Mathematical Statistics*, 39(1):209–214, 1968.

[28] Jérôme Garnier-Brun, Marc Mézard, Emanuele Moscato, and Luca Saglietti. How transformers learn structured data: insights from hierarchical filtering, 2024.

[29] Edwin T. Jaynes. Information theory and statistical mechanics. *Physical Review*, 106(4): 620–630, 1957.

[30] Dimitrios Bachtis, Giulio Biroli, Aurélien Decelle, and Beatriz Seoane. Cascade of phase transitions in the training of energy-based models. In *Advances in Neural Information Processing Systems 37*. 2024.

[31] Daniel C. Mattis. Solvable spin systems with random interactions. *Physics Letters A*, 56 (5):421–422, 1976.

[32] Aurélien Decelle, Giancarlo Fissore, and Cyril Furtlehner. Spectral dynamics of learning in restricted Boltzmann machines. *EPL (Europhysics Letters)*, 119(6):60001, 2017.

[33] Jérôme Tubiana and Rémi Monasson. Emergence of compositional representations in restricted Boltzmann machines. *Physical Review Letters*, 118:138301, 2017.

[34] Andrew M. Saxe, James L. McClelland, and Surya Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations*. 2014.

[35] Davide Carbone. Hitchhiker's guide on the relation of energy-based models with other generative models, sampling and statistical physics: a comprehensive review. *Transactions on Machine Learning Research*, 2025.

[36] Peter J. Brockwell and Richard A. Davis. *Time Series: Theory and Methods*. Springer, New York, 2 edition, 1991.

[37] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

[38] W. Keith Hastings. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika*, 57(1):97–109, 1970.

[39] Christian P. Robert and George Casella. *Monte Carlo Statistical Methods*. Springer, New York, 2 edition, 2004.

[40] Stuart Geman and Donald Geman. Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 6(6):721–741, 1984.

[41] Kiyosi Itô. Stochastic integral. *Proceedings of the Imperial Academy, Tokyo*, 20(8):519–524, 1944.

[42] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*. 2014.

[43] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems 27*. 2014.

[44] Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 1530–1538, 2015.

[45] Yann LeCun, Sumit Chopra, Raia Hadsell, Marc'Aurelio Ranzato, and Fu Jie Huang. A tutorial on energy-based learning. In Gökhan Bakır, Thomas Hofmann, Bernhard Schölkopf, Alexander J. Smola, Ben Taskar, and S.V.N. Vishwanathan, editors, *Predicting Structured Data*. MIT Press, Cambridge, MA, 2006.

[46] Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002.

[47] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 2256–2265, 2015.

[48] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 6840–6851, 2020.

[49] Aapo Hyvärinen. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6:695–709, 2005.

[50] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.

[51] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[52] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[53] Yang Song. Generative modeling by estimating gradients of the data distribution (blog post). `https://yang-song.net/blog/2021/score/` [last access: 14/07/26], 2021.

[54] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 26565–26577, 2022.

[55] George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021.

[56] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems 31*. 2018.

[57] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*. 2023.

[58] Michael S. Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations*. 2023.

[59] Peter Holderrieth and Ezra Erives. An introduction to flow matching and diffusion models, 2025. arXiv:2506.02070; lecture notes for MIT course 6.S184.

[60] Herbert Goldstein, Charles P. Poole, and John L. Safko. *Classical Mechanics*. Addison-Wesley, San Francisco, 3 edition, 2002.

[61] Lev D. Landau and Evgeny M. Lifshitz. *Statistical Physics, Part 1*. Pergamon Press, Oxford, 3 edition, 1980.

[62] Ernst Ising. Beitrag zur Theorie des Ferromagnetismus. *Zeitschrift für Physik*, 31(1): 253–258, 1925.

[63] Lars Onsager. Crystal statistics. I. A two-dimensional model with an order-disorder transition. *Physical Review*, 65(3-4):117–149, 1944.

[64] Giulio Biroli, Tony Bonnaire, Valentin de Bortoli, and Marc Mézard. Dynamical regimes of diffusion models. *Nature Communications*, 15:9957, 2024. doi: 10.1038/s41467-024-54281-3.

[65] Gabriel Raya and Luca Ambrogioni. Spontaneous symmetry breaking in generative diffusion models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[66] Samuel F. Edwards and Philip W. Anderson. Theory of spin glasses. *Journal of Physics F: Metal Physics*, 5(5):965–974, 1975.

[67] David Sherrington and Scott Kirkpatrick. Solvable model of a spin-glass. *Physical Review Letters*, 35(26):1792–1796, 1975.

[68] Marc Mézard, Giorgio Parisi, and Miguel A. Virasoro. *Spin Glass Theory and Beyond*. World Scientific, Singapore, 1987.

[69] David J. Thouless, Philip W. Anderson, and Richard G. Palmer. Solution of 'Solvable model of a spin glass'. *Philosophical Magazine*, 35(3):593–601, 1977.

[70] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation*. Oxford University Press, Oxford, 2009.

[71] Lenka Zdeborová and Florent Krzakala. Statistical physics of inference: thresholds and algorithms. *Advances in Physics*, 65(5):453–552, 2016.

[72] Florent Krzakala and Lenka Zdeborová. Statistical physics methods in optimization and machine learning: An introduction to replica, cavity and message-passing techniques, 2024. Lecture notes.

[73] John J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982.

[74] Donald O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. Wiley, New York, 1949.

[75] Daniel J. Amit, Hanoch Gutfreund, and Haim Sompolinsky. Storing infinite numbers of patterns in a spin-glass model of neural networks. *Physical Review Letters*, 55(14): 1530–1533, 1985.

[76] The Royal Swedish Academy of Sciences. The Nobel Prize in Physics 2024. `https://www.nobelprize.org/prizes/physics/2024/summary/`, 2024. Last accessed 19 July 2026.

[77] David H. Ackley, Geoffrey E. Hinton, and Terrence J. Sejnowski. A learning algorithm for Boltzmann machines. *Cognitive Science*, 9(1):147–169, 1985.

[78] Paul Smolensky. Information processing in dynamical systems: Foundations of harmony theory. In David E. Rumelhart and James L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Volume 1: Foundations*, pages 194–281. MIT Press, Cambridge, MA, 1986.

[79] Geoffrey E. Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7):1527–1554, 2006.

[80] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[81] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems 25*. 2012.

[82] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–444, 2015.

[83] Elizabeth Gardner. The space of interactions in neural network models. *Journal of Physics A: Mathematical and General*, 21(1):257–270, 1988.

[84] Marc Mézard. Mean-field message-passing equations in the Hopfield model and its generalizations. *Physical Review E*, 95:022117, 2017.

[85] Bernt Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. Springer, Berlin, 6 edition, 2003.

[86] Crispin W. Gardiner. *Stochastic Methods: A Handbook for the Natural and Social Sciences*. Springer, Berlin, 4 edition, 2009.

[87] Hannes Risken. *The Fokker–Planck Equation: Methods of Solution and Applications*. Springer, Berlin, 2 edition, 1996.

[88] George E. Uhlenbeck and Leonard S. Ornstein. On the theory of the Brownian motion. *Physical Review*, 36(5):823–841, 1930.

[89] Brian D. O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982.

[90] Giulio Biroli and Marc Mézard. Generative diffusion in very large dimensions. *Journal of Statistical Mechanics: Theory and Experiment*, 2023(9):093402, 2023.

[91] Chieh-Hsin Lai, Yang Song, Dongjun Kim, and Yuki Mitsufuji. The principles of diffusion models: From origins to advances, 2025. arXiv:2510.21890.

[92] Herbert Robbins. An empirical Bayes approach to statistics. In *Proceedings of the Third Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 157–163, 1956.

[93] Koichi Miyasawa. An empirical Bayes estimator of the mean of a normal population. *Bulletin of the International Statistical Institute*, 38(4):181–188, 1961.

[94] Bradley Efron. Tweedie's formula and selection bias. *Journal of the American Statistical Association*, 106(496):1602–1614, 2011.

[95] Beatrice Achilli, Marco Benedetti, Giulio Biroli, and Marc Mézard. Theory of speciation transitions in diffusion models with general class structure, 2026.

[96] Antonio Sclocchi, Alessandro Favero, and Matthieu Wyart. A phase transition in diffusion models reveals the hierarchical nature of data. *Proceedings of the National Academy of Sciences*, 122(1):e2408799121, 2025.

[97] Beatrice Achilli. *Statistical Physics of Generative Diffusion*. PhD thesis, Bocconi University, 2026.

[98] Tony Bonnaire, Raphaël Urfin, Giulio Biroli, and Marc Mézard. Why diffusion models don't memorize: the role of implicit dynamical regularization in training. In *Advances in Neural Information Processing Systems*, 2025. Oral presentation; arXiv:2505.17638.

[99] Jérôme Garnier-Brun, Luca Biggio, Davide Beltrame, Marc Mézard, and Luca Saglietti. Biased generalization in diffusion models, 2026. Spotlight, International Conference on Machine Learning.

[100] Davide Ghio, Yatin Dandi, Florent Krzakala, and Lenka Zdeborová. Sampling with flows, diffusion, and autoregressive neural networks from a spin-glass perspective. *Proceedings of the National Academy of Sciences*, 121(27):e2311810121, 2024.